

SECOND EDITION

ACING The Certified Kubernetes Administrator Exam

Chad M. Crowell
Rafael Brito



MEAP



MANNING

SECOND EDITION

ACING
The Certified
Kubernetes
Administrator
Exam

Chad M. Crowell
Rafael Brito

MEAP

 MANNING



Acing the Certified Kubernetes Administrator Exam, Second Edition

1. [welcome](#)
2. [1 First steps](#)
3. [2 Kubernetes cluster](#)
4. [3 Identity and access management](#)
5. [4 Deploying applications in Kubernetes](#)
6. [5 Running applications in Kubernetes](#)

welcome

Thank you for your interest in the MEAP edition of *Acing the Certified Kubernetes Administrator Exam, Second Edition*. This edition has been revised with updated study materials for the new CKA Exam and it reflects the changes to the domains and objectives. Also, Rafael Brito joins as my co-author, bringing his many years of valuable industry knowledge and experience to you, the reader.

To some, Kubernetes is considered overly complex and difficult to learn because it abstracts away so many of the components of a Linux system. I'm not going to pretend that's not true, but I think we've done a great job in this book of explaining each component, to pull back the curtain, so to speak, and provide greater context, helping you better grasp concepts through real-world examples. Although this book is focused on the exam, I think it's also a good resource to learn the fundamentals of Kubernetes. However, if you'd like to dive deeper into a specific topic, luckily there are a lot of other valuable resources out there. For example, the book *Kubernetes in Action*, by Marko Lukša, is a book I recommend a lot.

Rafael and I relate to you, as we were learning Kubernetes back in 2016 and 2017, respectively, and failed to find easy-to-understand study materials. I will say, however, that the concepts in this book will come easier to you if you have experience working with Linux and container technology. The great thing about Kubernetes is that it builds on existing technology, such as iptables, DNS, cgroups and namespaces. It's not re-inventing the wheel; it's simply using existing technology to create efficient ways of decoupling components and distributing load to complement whatever applications you're running on it.

This book largely follows the CKA exam objectives, but it's broad enough to cover a wide spectrum of Kubernetes primitives. This includes networking, identity, storage, troubleshooting and running applications on Kubernetes. We think you'll enjoy the practical examples, and how we will quickly get a Kubernetes cluster up and running, so that building a cluster is not a barrier to

success as you follow along in this book. This book does have a lot of examples that will allow you to practice what you learn. This is crucial, because the CKA exam is solely hands-on and the curriculum was revamped in 2025 with new domains, which this new edition of this book covers.

We aim to give you the necessary knowledge to ace the exam and deliver the content in a way that's easy to absorb. We like to present the material as if we were all sitting down together and having a conversation, and I hope it comes across that way. You've taken the initiative to purchase and read this book, so we take your time seriously and truly want you to succeed.

Your feedback is essential to creating the best version of this book possible and will help so many others trying to study for this exam. We hope that you'll leave comments in the [liveBook Discussion forum](#), as it will let us know how you understand the material in the book, and improve how we relay that information for all readers in the future.

—Chad M. Crowell

In this book

[welcome](#) [1 First steps](#) [2 Kubernetes cluster](#) [3 Identity and access management](#)
[4 Deploying applications in Kubernetes](#) [5 Running applications in Kubernetes](#)

1 First steps

This chapter covers

- An introduction to the Certified Kubernetes Administrator Exam
- What the Kubernetes Administrator role entails
- Kubernetes and the problems it solves
- The Kubernetes API and its architecture
- Kubernetes components and Linux backend processes
- Declarative and Imperative commands in Kubernetes

1.1 Introducing the CKA Exam

Welcome to *Acing the Certified Kubernetes Administrator Exam, 2_{nd} edition*. If you've purchased this book, chances are you've already researched the exam, know what it's about and perhaps even have it on your schedule. If not, don't worry, we'll talk about what the exam is and how to get signed up as soon as possible. For those of you who want to get right down to it, go ahead and skip this section, as it will most likely be reviewed for you. You can skip to section 2, where we get into the meat and potatoes of the exam curriculum.

For those who are still getting introduced to the CKA exam, let's go over what the exam is and what it entails. First, let us start by stating that there are now five Kubernetes exams in total, the CKA being the first and the longest-running exam. The others are, KCNA (Kubernetes and Cloud Native Associate), KCSA (Kubernetes and Cloud Native Security Associate), CKAD (Certified Kubernetes Application Developer), and CKS (Certified Kubernetes Security Specialist). There's even a new program called "Kubestronauts" that awards you a special jacket when you collect them all! Here's how the CKA (Certified Kubernetes Administrator) fits in with the other exams:

- The CKA is a hands-on exam, which means it does not offer multiple-

- choice or fill-in-the-blank questions. More on that later in this chapter.
- The CKA covers the background services (kubelet, kubeadm, and systemd) that help Kubernetes run.
- The CKA exam was the first Kubernetes exam ever and was first introduced in November 2017.

I'm glad you've decided to join us on this journey to getting certified in Kubernetes. The Certified Kubernetes Administrator (CKA) certification will be quite an accomplishment and will help hugely advance your career. Also, you'll be a part of a large group of individuals who hold this certification, including over 70,000 people worldwide (as of 2025). You might be asking yourself if it's worth it, and to that, I would say yes, and here's why:

- Kubernetes and distributed systems are going to be around for a long time.
- Kubernetes skills are in high demand.
- Getting certified will help solidify your understanding and show that you are well-rounded and versed in Kubernetes.

Now, let's get into what the exam is all about. The Certified Kubernetes Administrator (CKA) exam is a competency test like no other. Instead of multiple-choice or fill-in-the-blank type questions, this exam is entirely executed from within a Linux terminal (Ubuntu XFCE) in a remote desktop environment provided by PSI Services (the exam provider). Yes, that's right, they'll give you a set of tasks to complete, and you'll execute the solution by typing in the commands inside the terminal within the provided remote desktop environment. This is the entire exam experience, and you will have two hours to complete 15-20 tasks of this nature. Once complete, you are graded on the outcome of your tasks, no matter which path you took to achieve the outcome. This means, there may be more than one way to solve a given task. Throughout this book, you will learn different ways of getting the same results, giving you more tools in your tool belt, to achieve a passing grade on this exam, which is a 66% or higher. You can also receive partial credit on any task, so this will help your success rate as well.

Throughout this book, we'll address tips and tricks available for each topic within each chapter. This will tie in nicely to the application of your learnings, and give you the necessary skills to approach the exam with

confidence. These exam tips, in combination with your determination and repeated practice, will lead to success in passing the CKA exam. I can't emphasize enough how much muscle memory and putting in the practice will help your brain retain and access the appropriate Kubernetes when time comes for the exam. It is truly an exercise, so if you're determined to pass the exam, I wouldn't take too long of breaks in-between studying, especially if you aren't working with Kubernetes daily. Don't let this deter you though, we're going to do this Kubernetes workout together!

The CKA exam is provided by the Linux Foundation and Kubernetes is maintained by the Cloud Native Computing Foundation (CNCF). The exam costs \$445 US dollars as of the writing of this book, but check the updated pricing sheet for current prices and prices in other currencies by visiting the Linux Foundation website at <https://training.linuxfoundation.org>. This price may be a bit higher than similar certifications of its class, but they do allow one free retake as well one additional browser tab opened to the following sites and their subdomains during the exam:

- <https://kubernetes.io/docs/>
- <https://kubernetes.io/blog/>
- <https://helm.sh/docs>

In order to take the exam, you will need a computer running Windows 10 (64-bit), Windows 11 (64-bit), macOS 13.x or higher, or Ubuntu 22.04. The exam is taken using the PSI Secure Browser (a web browser created to guarantee a secure exam delivery over a virtual connection). When you begin the exam, you'll be instructed to download and install a new PSI-secured browser, automatically granting you access to the PSI's proctoring platform called the "PSI Bridge" (a remote desktop environment). The remote desktop environment will include links to open the terminal and the provided Firefox browser (you must use Firefox) so you can browse to the authorized sites listed earlier. You will also need a webcam that has at least 640 x 480 pixel resolution as the exam proctors (PSI) will require a 360 view of the room (external camera required for desktops) in order to monitor you for the duration of the exam. Your computer screen must be 1920 x 1080 or higher in 32-bit color (dual monitors are not supported), you must have a functional microphone, and your internet bandwidth speed must be at least 300kbps to

download and upload. The room where you take the exam must be quiet and well-lit. Public spaces such as coffee shops or stores are not allowed. Your desk must be cleared of all papers and other electronics, and you must ensure you can be seen clearly in the center frame of your webcam.

On exam day, you'll sit down at your desk with your computer, make sure it's plugged in, then go the Linux Foundation portal to start your exam. Before you click the "begin exam" button, make sure that all browser tabs are closed and all other applications are not running in the background (the proctor will check this as well).

EXAM TIP

Regardless of your machine's Operating System, before starting the exam, the PSI will close prohibited programs, such as display mirroring subsystems. We recommend connecting to the exam 30 minutes before the scheduled time in case you have to troubleshoot these programs to be stopped because some of them might respawn. Some students reported that they could only start the exam on macOS after disabling Bluetooth. Some other students could not pass this check and failed the exam automatically.

Once you click on the button to begin the exam, you'll immediately be introduced to an exam proctor. This exam proctor will check your environment to make sure that your desk is cleared and there aren't any papers or unauthorized electronics around. So, using your camera, you'll pan around the room to get a full 360 view, and you'll wait for their approval. They'll also periodically check your hands and wrists. I know this is kind of annoying, but at frequent intervals, the proctor will first check your hands and wrists before you start the exam, then they'll also stop you in frequent intervals in the middle of your exam and ask you to show both sides of your hands and wrists. Before they release the exam to you, they will also ask you for your government issued ID, which you must hold up to the camera. Once the proctor is done verifying your identity and checking your workspace, they will release the exam, which means they will allow you to enter the exam and view the first question. You'll notice that each question has a similar format, including the context you must use, and then the tasks(s) which you must execute via the command-line in order to solve. This exam is a race against

the clock, if you come to a question that you can't answer, my advice is to skip it, you can flag it and come back to it at any time during the exam. It will be clear how to flag a question for review, as they show you in a brief automated walkthrough at the beginning of the exam. For each task, you will also see the percentage of points that that task is worth. If you're stumped on something, look at how much it's worth. If it's worth, let's say, 5%, then go ahead and skip it.

EXAM TIP

If you are copying and pasting text back and forth from the Firefox browser and the terminal, use the keyboard shortcut CTRL + SHIFT + C to copy and CTRL + SHIFT + V to paste.

The core competencies that you will be tested on for the exam are cluster architecture, installation and configuration, workloads and scheduling, services and networking, and storage and troubleshooting. We'll cover all these areas in this book. Within the cluster architecture competency, which will consist of 25% of the questions on the exam, you will be tested on role-based access control, using kubeadm to update control plane components, using Helm to install cluster components, and installing and configuring operators. In the workloads and scheduling competency, which will consist of 15% of the questions on the exam, expect to be tested on performing rolling updates and rollbacks, auto-scaling applications, pod admission with requests and limits, and using ConfigMaps and secrets. In the services and networking competency, which consists of 20% of the questions on the exam, you'll be tested on creating and updating services in Kubernetes, enforcing network policies, using ingress controllers and ingress resources with Gateway API, managing DNS, and the container network interface in Kubernetes. In the storage competency, which consists of 10% of the questions on the exam, you'll need to understand storage classes, persistent volumes, and volume modes in Kubernetes. Then, in the troubleshooting competency, which is 30% of the questions on the exam, you will be expected to know how to get the logs from a Kubernetes cluster, as well as monitoring and repair of core cluster components. Figure 1.1 shows a table outlining these domains and their competencies.

Table 1.1 Exam competencies and percentages in which they will be tested on the exam

Cluster Architecture, Installation & Configuration – 25%
Manage role based access control (RBAC) Implement and configure a highly-available control plane Create and manage Kubernetes clusters using kubeadm Manage the lifecycle of Kubernetes clusters Prepare underlying infrastructure for installing a Kubernetes cluster Use Helm and Kustomize to install cluster components Understand extension interfaces like CNI, CSI, and CRI Install and configure operators with an understanding of CRDs
Workloads & Scheduling – 15%
Understand application deployments and how to perform rolling update and rollbacks Use ConfigMaps and Secrets to configure applications Configure workload autoscaling Understand the primitives used to create robust, self-healing, application deployments Configure pod admission and scheduling (requests, limits, node affinity, etc.)
Services & Networking - 20%
Understand host networking configuration on pods and nodes Define and enforce network policies Understand ClusterIP, NodePort, LoadBalancer service types and endpoints Know how to use Ingress controllers and Ingress resources Use Gateway API to manage ingress traffic Know how to configure and use CoreDNS Define and enforce network policies
Storage – 10%
Implement storage classes, and dynamic volume provisioning Configure volume types, access modes and reclaim policies Manage persistent volumes and persistent volume claims
Troubleshooting – 30%
Troubleshoot cluster and node component failure Understand how to monitor cluster and application resource usage Manage and evaluate container logs from stdout & stderr

Troubleshoot application failure Troubleshoot cluster component failure Troubleshoot services and networking
--

You will have access to different clusters during the exam, which you will be required to ssh to each control plane node depending on the question. Usually, each question will ask you to perform the task on a different cluster than the previous question. They will provide instructions at the beginning of each question for which node to ssh to, so don't worry too much about memorizing the cluster names and commands to switch between clusters. They will also have the alias for `kubectl` set to `k`. An *alias* is a command you run that pertains to another command. For example, a common alias that exists for most Linux operating systems is `l='ls -lah'`. This means when you type the command `l`, it's really the same thing as typing `ls -lah`. Similarly for the CKA exam, when you type the command `k`, it's the same thing as the command `kubectl`. From the command-line on your computer, you can type `alias` to list all the existing aliases on your computer.

So many people have registered for the CKA exam that it has become one of the most popular Linux Foundation certifications to date. This partially speaks to the demand for the certification, but also to the credibility of the certification once received. The certificate is valid for two years, and the process to recertify is the same process in which you originally went through the first time you sat the exam.

I suggest scheduling your exam now, so you have an end goal and a deadline for completing it. This will keep you motivated to finish this book and will provide the necessary timeline to keep your knowledge top of mind, and also appropriately fitted to sit the exam. If you are working with Kubernetes at your job today, and you are typing `kubectl` commands on a daily basis, then schedule your exam in one month from today. If you are just approaching the topic of Kubernetes, having never heard of it before, go ahead and schedule your exam for the furthest date the exam scheduler will allow (usually this is three months, but it may be less.) You can always reschedule, but the point is to give yourself two things. 1.) A deadline, so that you can take this exam seriously and achieve your intended result. 2.) Daily practice to implement into your pre-frontal cortex (where memories are stored). This is exactly what

you'll need to stay fresh and ready for the exam.

1.2 What's in store

Throughout this book, I'll be inserting exercises and exam-like scenarios for you to practice `kubectl` commands to best prepare you for the exam. To make it as easy as possible for you to practice I've included instructions for creating your own local cluster. In Appendix A, I'll walk you through the steps to create a Kubernetes cluster using `kind`. `Kind` Kubernetes (<https://kind.sigs.k8s.io/>) is a free, lightweight, and easy to use tool for creating a cluster right on your existing laptop or desktop computer. The only requirement is `Docker`, and with one command, you'll have a cluster up and running in seconds. This eliminates the barrier to entry for practicing for the CKA. I advise you to utilize this method, as you can waste a lot of time trying to build a cluster manually, and because the exam will have clusters already prepared for you, I believe this is the best way to study and follow along with this book. There are other methods to create a local cluster which you are welcome to use, such as `minikube` or `microk8s`. The idea is similar, but those tools might not align directly with the scenarios in this book. For example, in chapter 5 we'll prepare a cluster with a missing node. The steps to stage the environment are unique to `kind`, so reproducing the environment in `minikube` will be different.

Within each chapter of this book, there will be a scenario that resembles the real exam, which we'll work through together, followed by additional practice exercises at the end of the chapter which you'll be tested to complete on your own. In appendix D, you'll be able to look at hints for solving these practice exercises. You can use these to both test your knowledge of the book's content, and to gauge your readiness for the CKA exam.

Before we begin, I'd like to clear some things up and let you know what this book is and what it isn't. This is not an introduction to Kubernetes book, so I expect you to have a background in containers, and an understanding of what problem Kubernetes solves as it will not be covered in this book. The CKA exam is not an introductory exam either, therefore, it will take quite a bit of experience with using and navigating the Linux operating system. People who understand `cgroups` and `namespaces`, which provide containers in Linux

will have an easier time following this book and subsequently passing the exam. Also, most who sit this exam already are working with the technology in their job today, by either getting introduced to it via a new project, or cluster administration is already a primary function within their role. I'm not saying that people who are not working with Kubernetes won't pass the exam, but getting direct exposure to real-world scenarios and using your knowledge in a hands-on fashion daily is more likely to lead to success.

1.2.1 What is a Kubernetes administrator?

The role of a Kubernetes Administrator is twofold. A Kubernetes Administrator knows the inner workings of Kubernetes and knows how to translate that into value for the end-users. Here's what a job posting for a Kubernetes Administrator might say: "The responsibility of a Kubernetes Administrator is to ensure that the company's services meet the needs of the customers within the desired levels of reliability, performance, and availability by developing continuous improvements, tools, and automation. You must know how to install, configure, deploy, update, and patch each of the components of the Kubernetes infrastructure to ensure that services and underlying systems are properly monitored and have adequate observability. This includes identifying and monitoring proper KPIs (Key Performance Indicators) to ensure service health, minimizing MTTA (Mean Time to Acknowledge) and MTTR (Mean Time to Repair)."

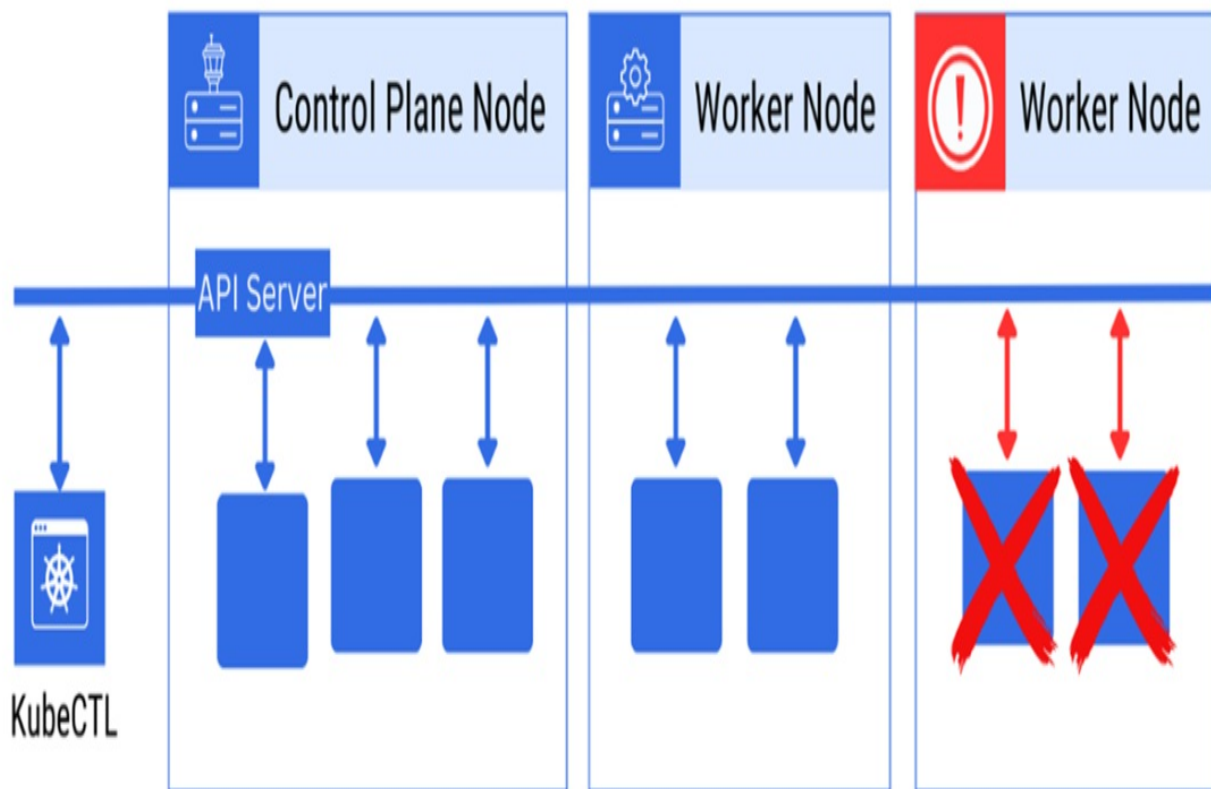
Let's not sugar coat it, the difficulty level of the exam remains high, hence the need for this book. But you will start to realize the complexities of Kubernetes after reading this book, not only how to envision the complexities and how it relates to what you already know about engineering and technology, but also how to dig through the complexities to troubleshoot and determine the root cause for the actions within a Kubernetes Cluster.

1.3 Meeting Kubernetes

That brings me to the next point, the Kubernetes Cluster: What is it? The Kubernetes cluster is called a *cluster* because it is a series of machines working together, in conjunction, as in figure 1.2. Just like a server farm, the machines are interconnected and housed within the same facility or network.

But, the key difference in Kubernetes is that the connected servers not only distribute load appropriately, but also easily exchange data to eliminate a single point of failure. This means if one of the nodes fails, this doesn't bring down the entire cluster. From now on in this book, we'll call servers "nodes". *Node* is a term that's specific to clusters, which tends to indicate that the server is a part of a larger system like a cluster. We'll talk more about the anatomy of a cluster in Chapter 2.

Figure 1.1 Servers (called "nodes") working together in a cluster. When node failure occurs, you can remove it from the cluster to be repaired without impacting running workloads.



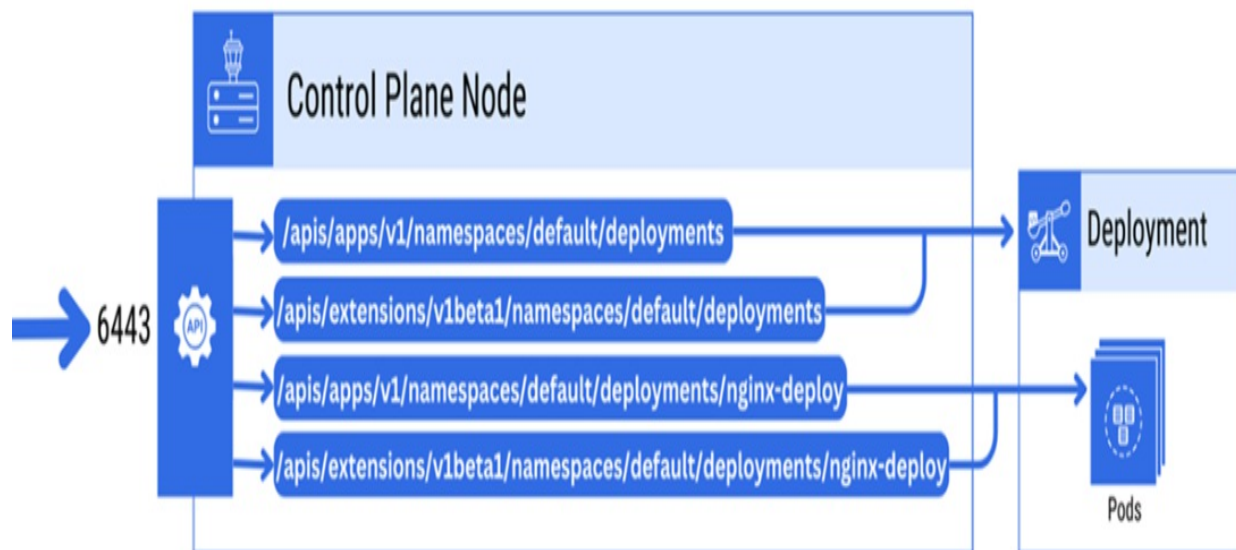
NOTE

Kubernetes is often referred to as K8s for short (pronounced *_kates_*), where the '8' between the K and the S represent the number of characters between the K and the S ("ubernete" = 8 characters).

Kubernetes is nothing more than a piece of software that you interact with via

REST (Representational State Transfer) API. A RESTful API is a well-defined, highly scalable, loosely coupled application architecture, which favors communication over a network. More importantly, the act of transferring or communicating the representation of the state of a resource over a network. I can't stress enough how important it is to remember that Kubernetes is behind an API (A RESTful One), which is a collection of resources that can be manipulated through that API. Resources is the key word here. Resources are how we address objects in Kubernetes. Sometimes in the Kubernetes community, we use the word "resources" and "objects" interchangeably, but there is a fundamental difference. Objects can have more than one resource address; for example, deployments can have multiple URIs (Uniform Resource Identifiers) based on the API version or based on the deployment name, as you can see in figure 1.3. *Deployments* in Kubernetes are objects to offer more automated control over pods and ReplicaSets that comprise your application running on Kubernetes. *ReplicaSets* are another type of object in Kubernetes, which provide a control loop running a specified number of pods (i.e. pod replicas). You can list all available API resources with the command `kubectl api-resources` from any Kubernetes cluster.

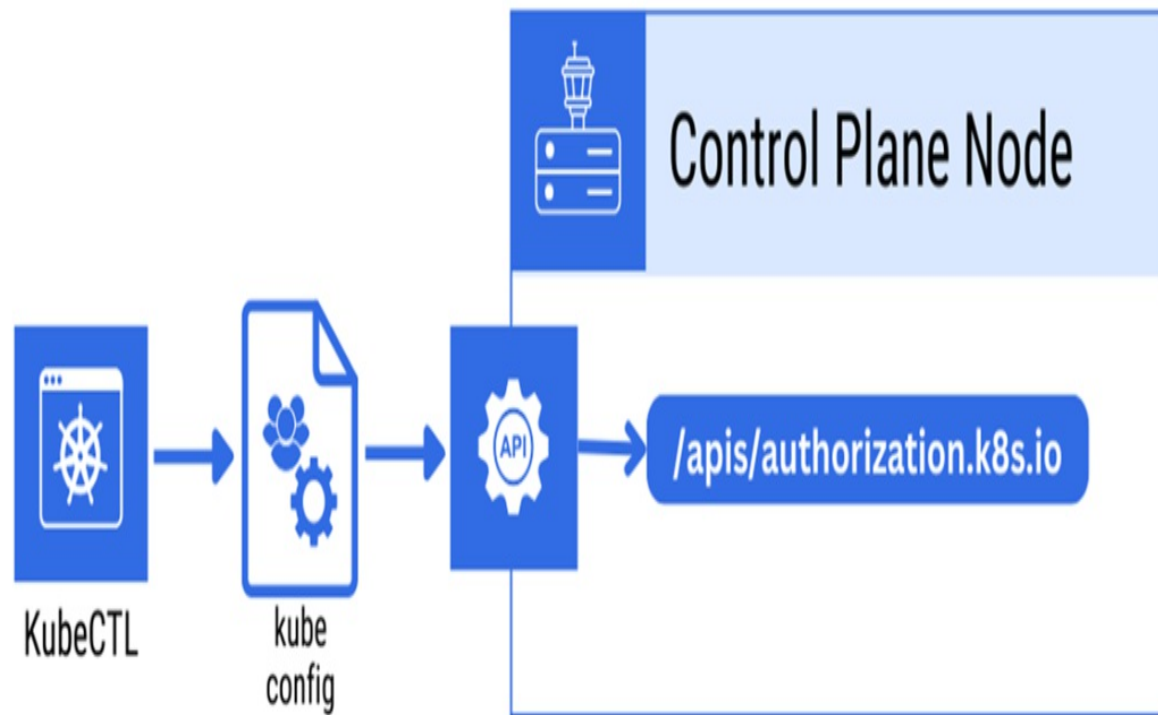
Figure 1.2 Kubernetes addresses objects by their resource URI, where multiple URIs can point to a single object



We commonly interact with the Kubernetes API, and the objects within, by

using a command-line tool called *kubectl*. We will be using *kubectl* solely throughout this book, as this will be the command-line tool used on the exam to interface with the Kubernetes API. Having the *kubectl* command-line tool, in addition to a certificate is needed to create, read, update, and delete Kubernetes resources. Depicted in figure 1.4, a certificate that provides us with the role-based access that we need to perform certain operations in the Kubernetes cluster.

Figure 1.3 Kubectl is the tool used to access Kubernetes objects, given a valid certificate inside the kubeconfig

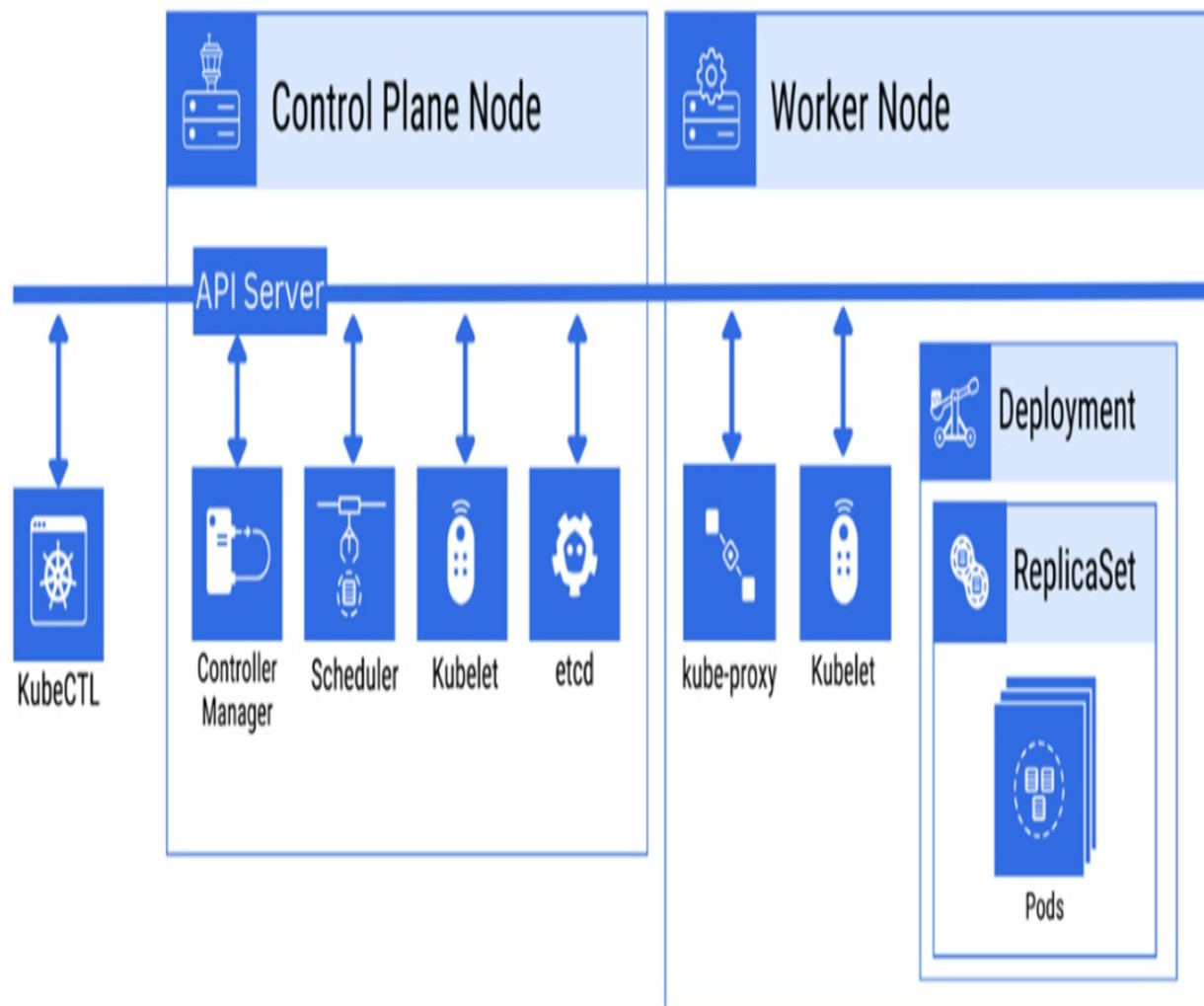


1.3.1 Cluster architecture, installation, and configuration

As previously mentioned, the architecture of a cluster is built out of nodes, and on those nodes are running *pods*. A pod is the smallest deployable unit in Kubernetes, and contains one or more containers. In an abstraction of the API, resources like deployments are created that are comprised of ReplicaSets, which in turn are running one or more pods across multiple nodes. In Kubernetes there are two types of nodes, the *control plane node* and

the *worker node*. The control plane node runs the API server, DNS, the controller manager, the scheduler, kube-proxy, and etcd datastore. All these parts and their relationships can be seen in figure 1.5. Communication to the control plane happens through the API, on the control plane node. The worker nodes, by way of the kubelet, get instructions from the control plane to fulfill their duty to not only to run the containers themselves but to report back on the health, providing a constant status back to the control plane. The worker carries the brunt of the load, which makes their role very important for running applications in Kubernetes.

Figure 1.4 Kubernetes architecture showing nodes, pods, deployments and ReplicaSets



1.3.2 Workloads and scheduling

Resources like `ReplicaSets` (abbreviated “RS”) and *Deployments* (abbreviated “Deploy”) are essential to developing stateless application workloads running on Kubernetes. Without them, we wouldn’t be able to automatically scale pods, which provides load balancing, making our applications more readily available when running on Kubernetes. We call the process of deploying these applications to Kubernetes “scheduling”. The term scheduling comes from the *scheduler*, which is a component of the control plane. We’ll talk more about the control plane components later in this chapter. Historically, if you had created an application that runs on some hardware, it must be maintained and updated from time to time. This sometimes causes outages and/or limitations to what that hardware can do (it gets old). Kubernetes (the application) abstracts this hardware away and creates a common interface (an API) between it and everything else (hardware alike). This allows you to switch out hardware on the fly, as well as incorporate hardware from different vendors and mix and match.

My favorite way of driving home this point is to look at an example to get a solid understanding about resources inside of an API. Let’s look at an API that’s been rendered through the browser at <https://swapi.dev>. So, if you go to SWAPI (Star Wars API), you’ll see a way that you can request different facts about the Star Wars movies. You make this request via a GET request to the API. Because the API is built on top of the HTTP protocol, we can perform HTTP methods to take action on the API endpoints. Those actions can be Create, Read, Update, or Delete, usually put into an acronym: *CRUD*. In this case, though, the purpose of the SWAPI is to make a request to the API and “GET” back some data. So, at the top of the page, in the request field, type in `people/1/` and down below you should see the result, shown in figure 1.6.

Figure 1.5 SWAPI website allows you to access objects the same way Kubernetes does

← → ↻ ⬆ <https://swapi.dev>

<https://swapi.dev/api/people/1/>

request

Result:

```
{
  "name": "Luke Skywalker",
  "height": "172",
  "mass": "77",
  "hair_color": "blond",
  "skin_color": "fair",
  "eye_color": "blue",
  "birth_year": "19BBY",
```


The result is “Luke Skywalker”, but the result doesn’t really matter, the point is that you just got back data (in JSON format) from the API. The data that you received is the resource URI we’ve been talking about. The object located at that URI was “Luke Skywalker”, and that object contains data such as height, hair color, eye color, etc. Relating this back to Kubernetes, the people object in SWAPI is much like the deployment object in Kubernetes. And you can access the Kubernetes API in a similar way. The Kubernetes Dashboard even offers the API rendered through a browser just like SWAPI. Figure 1.7 shows some Kubernetes resource URIs.

Figure 1.6 Objects in Kubernetes accessed according to their resource URI. Nodes, for example, are located in /apis/apps/v1/nodes.



Control Plane Node

→ 6443



/apis/apps

/apis/storage.k8s.io

/apis/networking.k8s.io

/apis/authorization.k8s.io

/api/v1/pods

/api/v1/nodes

/api/v1/namespaces

/api/v1/serviceaccounts

There are different API calls (called HTTP *methods*) in order to perform certain actions against the API (given you have already authenticated to the API). They are GET, POST, PATCH, PUT and DELETE. A GET HTTP method is exactly what we've been doing with the SWAPI, which is to retrieve or view information about the resources in our API. A POST HTTP method is used to create a new resource in the API. A PATCH HTTP method is used to update existing resources. A PUT HTTP method is used to replace existing resources in the API. Finally, a DELETE HTTP method is used to delete a resource from the API.

1.3.3 Services and networking

To have the end user interact with your application running on Kubernetes, we create an object called a *service*. These services provide a load balancing feature to the pods they serve. They also provide a single IP address and DNS name, which we'll get to in later chapters. Speaking of DNS, there's a component in Kubernetes called *CoreDNS*, which provides names to IP translations. This means that resources in Kubernetes can talk to each other using common names as opposed to IP addresses. The three types of services are "ClusterIP", "NodePort", and "LoadBalancer", in which we will talk more about in Chapter 6.

An *Ingress resource* is another Kubernetes object that provides path-based routing to a service in Kubernetes. For example, you would create an ingress resource to offer a layer 7 (application layer) route to a service based on a URL path.

1.3.4 Storage

While managing ephemeral objects such as pods in Kubernetes, you can't rely on storage to be tied to a single pod. *Storage*, meaning the data that an application uses to store and organize files in a filesystem such as NTFS, XFS, or ext4. A *filesystem* is an instance of that organization of storage that takes place in an operating system such as Linux. In Kubernetes, there's this concept of *persistent volumes*, which aren't tied to an individual pod, but instead to an NFS, EFS or iSCSI volume. Having the storage decoupled from

the ephemeral applications creates that persistence of data.

In addition, from a developer's perspective, you no longer have to manage the underlying storage. To Kubernetes, it seems as just one large storage layer. The developer can utilize an object called a *persistent volume claim* to reserve a persistent volume, so it cannot be used for other applications running in Kubernetes. This still requires the persistent volume to exist, so in the case that it doesn't, the developer may access storage dynamically using an object called storage *class*. Storage class pertains to the different classes that they come in, for example, slow versus fast.

Volumes can also be used by multiple pods at the same time as well as reused, specifying certain access modes and certain reclaim policies respectively. The access modes allow read and write capability of a volume to one or many pods. The *reclaim* policy will allow or deny the access from other pods in a Kubernetes cluster.

1.3.5 Troubleshooting

Kubernetes clusters aren't perfect; they can get quite complex the more resources and objects are created in them. Troubleshooting when problems arise is essential to limit the downtime of running applications but also detect problems to optimize and have Kubernetes performing at its best. Being able to decipher logs from a running container, analyze failures, and propose a solution to that failure is key to becoming a great Kubernetes administrator. *Logs* are time-based behavioral data sent via text to a directory on the filesystem in order to give you verbose information about a problem that's occurring. All the redundancy in the world will not fix a cluster that has been poorly constructed and not well maintained.

Applications running on Kubernetes also have an added maintenance responsibility. They contain logs from `STDOUT` and `STDERR` that are important for detecting when communication problems arise or an application failure is imminent.

Practice Exercises

Perform the command to list all API resources in your Kubernetes cluster. Save the output to a file named “resources.csv”.

List the background processes on your Linux operating system that are associated with Kubernetes.

1.4 Control plane node

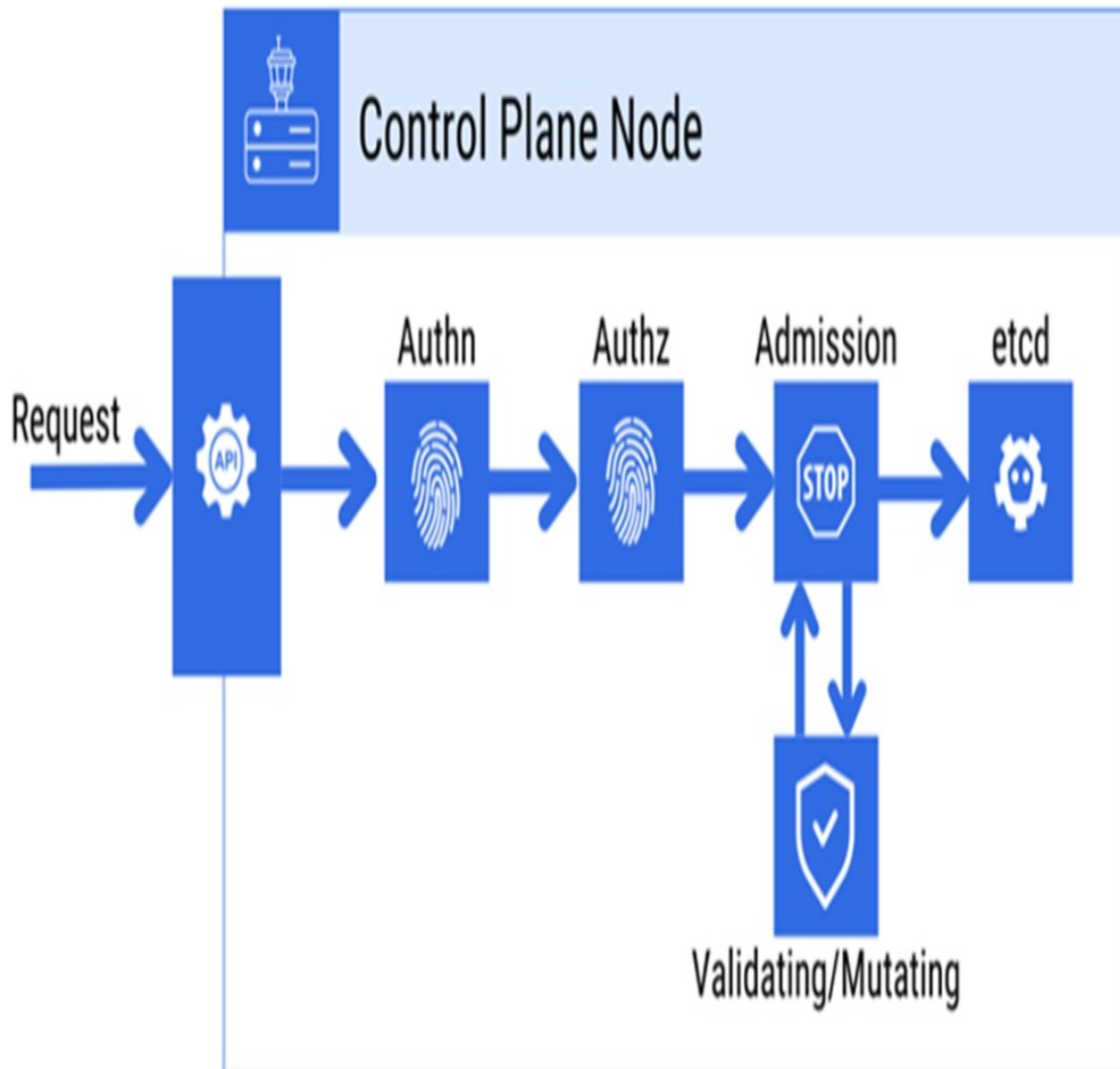
The services and components on the control plane that make it the control plane are:

- Controller Manager
- API Server
- Scheduler
- etcd datastore

The *controller manager* is the manager for all other controllers in Kubernetes. This includes deployments, ReplicaSets, etc. A controller in Kubernetes is responsible for matching the current state to the desired state. For example, the controller will automatically scale a deployment if the pods do not match the desired number of replicas. The controller also creates the default accounts and API access tokens for new namespaces. That’s why you’ll see when we get to creating a Kubernetes cluster in the next chapter that there’s a default namespace, service account, and secret already created for you to start deploying your resources (applications) into. This makes it easy to start running containerized applications on Kubernetes right away. When referring to a *namespace*, think of it as a dedicated virtual environment for your Kubernetes resources. So, the default namespace will be isolated within the scope of that namespace and can have its own grouping of Kubernetes resources.

The *API server* is exactly what it sounds like, it’s the component that exposes the Kubernetes API (the RESTful API that we talked about earlier). It’s the entry-point for the cluster, so all communication to the cluster passes through it in order to access the cluster components. Think of it like a gate, that will open only if opened with the right key using authentication, as in figure 1.8. We’ll talk more about authentication in chapter three.

Figure 1.7 In order to access the Kubernetes API, use a key to authenticate



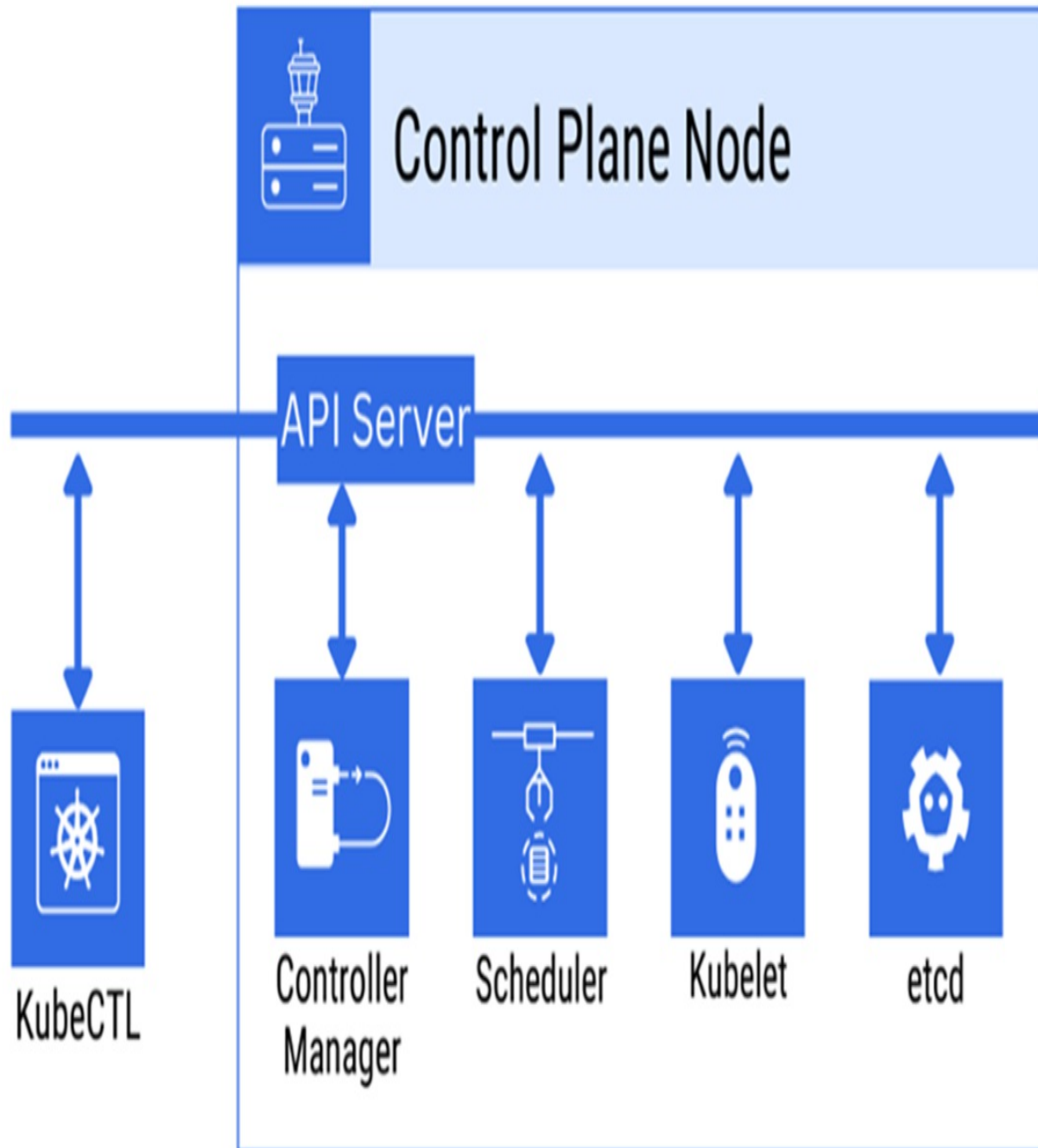
The *scheduler* is the component that selects the nodes for the pods to run on. You will probably hear the word scheduling a lot throughout this book, so from here on out when I say “scheduling”, it just means that the pods are being placed (scheduled) on the nodes for running them. If there are multiple nodes that already have pods running on them, then the scheduler is going to

place the pods on a different node, taking into consideration the available resources and other rules/specifications that have been placed on that node.

There are certain services (called *daemons*) that run on the control plane, in which are critical for the Kubernetes cluster to operate. They are so critical, in fact, that in a production scenario, they are replicated for high availability. Replicating the control plane components is beyond the scope of this book and the exam because the Kubernetes clusters on the exam are not going to have more than one control plane node. In general, replicating the control plane works by doing two things. One is ensuring the control plane components have only one instance receiving requests at a time. This is achieved by electing a leader. This leader will act as the primary control plane and relay its status to the other followers. Secondly, since the control plane relies on etcd for storing all the Kubernetes configuration data (the state of resources and how they are running in Kubernetes), you'll create multiple redundant copies of etcd as well. If you want to learn more about replicating the control-plane for high availability in Kubernetes, you can read more here: <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability>

Speaking of the *etcd datastore*, it's one of those critical control plane components we were talking about earlier. Etcd stores all the Kubernetes configuration data (the state of Kubernetes). Etcd is a consistent distributed *key-value store* designed to hold small amounts of data that can fit entirely in memory. This is important due to the fact that data can be retrieved more quickly than other traditional databases. See figure 1.9. The most important thing to remember about the etcd datastore is that losing it (etcd failure) is catastrophic, as it contains all the configuration information for resources running inside a Kubernetes cluster. Backing up etcd is no longer a requirement for the CKA exam, as of February 18th, 2025, so we don't cover this, as we have in the first version of this book.

Figure 1.8 The etcd datastore resides in a Kubernetes pod on the control plane node



1.5 Worker nodes

Now that we've talked about the control plane components in great detail,

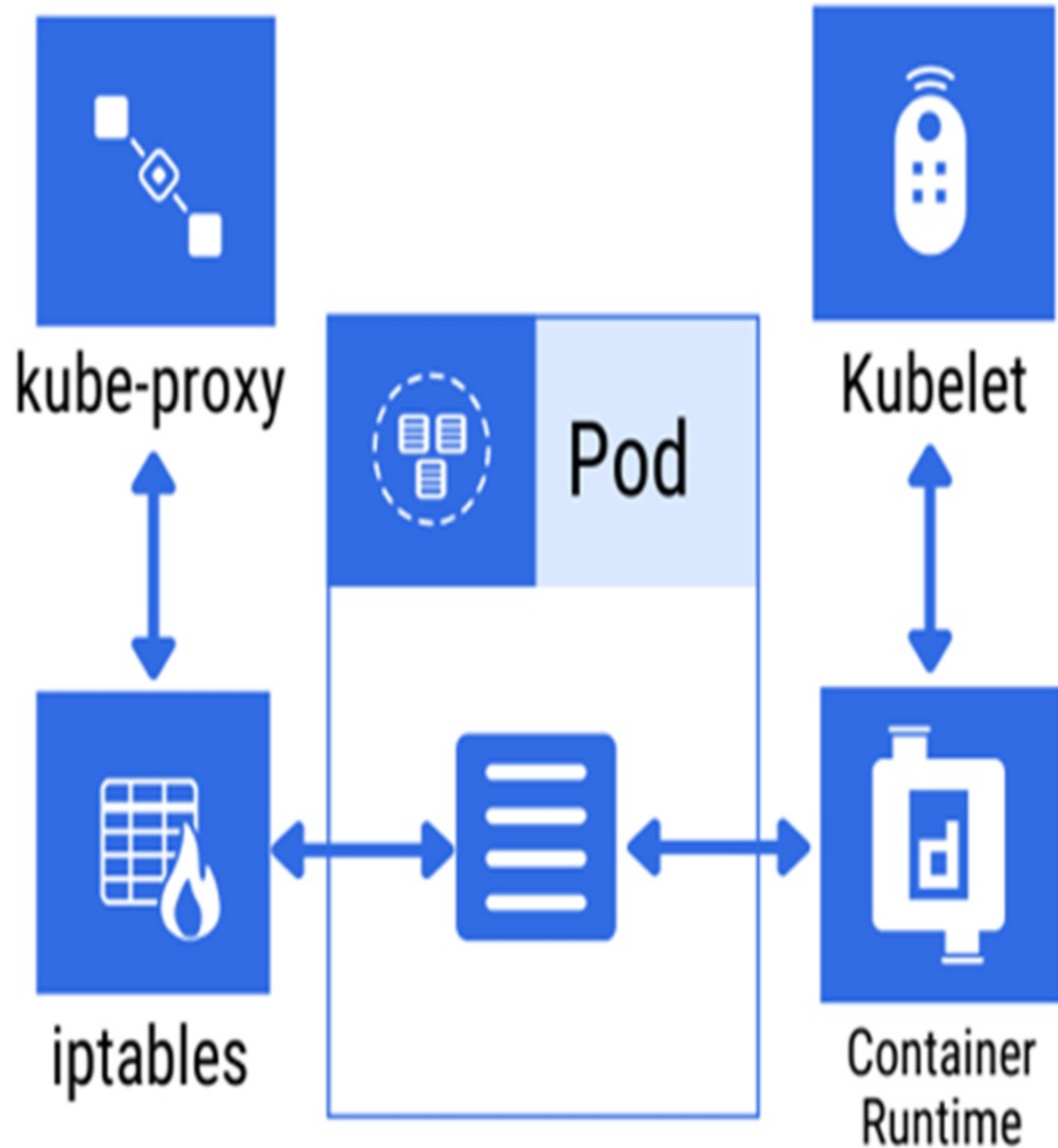
let's get to the components that reside on a worker node, which are shown in figure 1.10. The components that run on a worker node are different from the components on the control plane, because the worker nodes play a different role in the Kubernetes landscape. You will most certainly have more than one node inside of a Kubernetes cluster, and each one of the following components is installed on each node:

- - kubelet
- - kube-proxy
- - container Runtime

Figure 1.9 The role of the worker node is to run the application workloads. It also contains kubelet, kube-proxy and the container runtime (e.g. containerd)



Worker Node



The *kubelet* service that runs on each worker node makes sure that containers are running in a pod. However, while it's not aware of containers that aren't managed by the scheduler, kubelet can detect when a container (within a pod) is failing and take corrective action to ensure that the container restarts in the way that is specified in the *YAML manifest*, the set of instructions for configuring a pod in YAML (YAML ain't markup language) format. It's like a set of declarative instructions (with the end state in mind) for the kubelet to follow, in order to maintain high availability for the application running on Kubernetes. If the kubelet can't get the container to run, it will report the status of the pod and container to the Kubernetes API server and you will see it in the pod events, which are viewed by performing the command `kubectl describe po nginx` (nginx is the name of the pod), and you'll see a similar output here:

```
$ kubectl describe po nginx
Name:          nginx
Namespace:     default
Priority:       0
Node:          host01/172.17.0.33
Start Time:    Tue, 01 Feb 2022 16:49:36 +0000
Labels:        run=nginx
Annotations:    <none>
Status:        Running
IP:            192.168.0.4
IPs:
  IP: 192.168.0.4
Containers:
  nginx:
    Container ID:  containerd://4de3efffd3a6f1ec49c968d7fde95e8e
➔8055cca33a197499879
    Image:         nginx
    Image ID:      docker.io/library/nginx@sha256:2834dc507516af
➔cbe38b8ed5d0f4837f16e78d00deb7e7767
    Port:         <none>
    Host Port:    <none>
    State:        Running
      Started:    Tue, 01 Feb 2022 16:49:44 +0000
    Ready:        True
    Restart Count: 0
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-
➔api-access-5ffvj (ro)
```

```

Conditions:
  Type           Status
  Initialized     True
  Ready          True
  ContainersReady True
  PodScheduled   True
Volumes:
  kube-api-access-5ffvj:
    Type: Projected (a volume that contains
    ↪ injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    ConfigMapOptional: <nil>
    DownwardAPI: true
QoS Class:           BestEffort
Node-Selectors:      <none>
Tolerations:         node.kubernetes.io/not-ready:
    ↪ NoExecute op=Exists for 300s
                    node.kubernetes.io/unreachable:
    ↪ NoExecute op=Exists for 300s
Events:
  Type        Reason      Age   From          Message
  ----        -
  Normal      Scheduled   17s   default-scheduler
    ↪ Successfully assigned default/nginx to host01
  Normal      Pulling     16s   kubelet
    ↪ Pulling image "nginx"
  Normal      Pulled      9s    kubelet
    ↪ Successfully pulled image "nginx" in 6.993614669s
  Normal      Created     9s    kubelet
    ↪ Created container nginx
  Normal      Started     9s    kubelet
    ↪ Started container nginx

```

When you look at the events after describing the pod, you see that there are different states of a container (seen as the status) including *Waiting*, *Running*, and *Terminated*. You will also see the state of the container by running the command `kubectl get po`.

The *Kube-proxy* service running on each worker node is the communication mechanism of a Kubernetes service. A service is another resource type in Kubernetes, and is responsible for allocating traffic to the various pods within that service. We'll talk about services in-depth in chapter six. These are basic network rules for the node to follow in the event traffic needs to get to the pods in Kubernetes. There's a fascinating video that describes this in greater

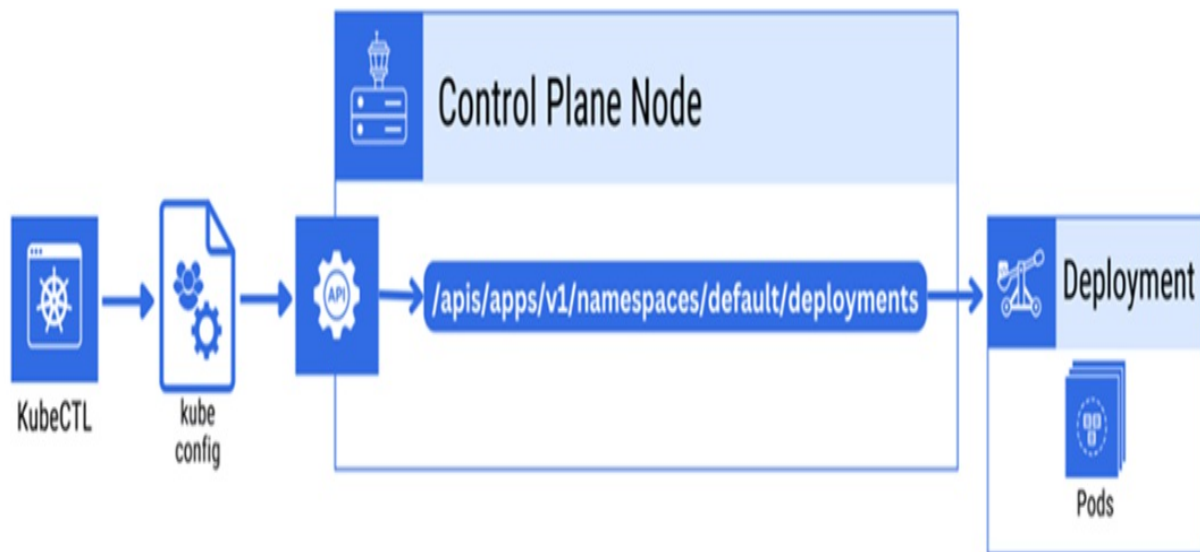
detail called “Life of a packet”. I highly encourage you to watch it here: <https://youtu.be/0Omvgd7Hg1I>

Finally, running on each node in the Kubernetes cluster is the *container runtime*. For the CKA exam, this will be containerd (<https://containerd.io/>). This is a necessary component in order for Kubernetes to work, as it is the engine that runs the containers themselves. Interestingly, though, it is an external dependency, meaning that it’s one of the only components necessary to a Kubernetes cluster that Kubernetes doesn’t apply or install itself. This means that you have choices when it comes to container engines that you have to install. Kubernetes supports any container runtime that follows the CRI requirements (<https://github.com/kubernetes/community/blob/master/contributors/devel/sig-node/container-runtime-interface.md>). Some common options are cri-containerd (<https://containerd.io/docs/>), or CRI-O (<https://cri-o.io/#what-is-cri-o>). Containerd is a lightweight and portable container runtime that is also CRI-compliant. CRI-O is another lightweight container runtime that supports the Open Container Initiative (OCI). So, it’s a community-driven, open-source project that supports multiple image formats, including Docker images.

1.6 API model and PKI

We’ll talk more about Role-based Access Control (RBAC) and identity management in Kubernetes in Chapter 3, but for now, let’s just talk about the public key infrastructure that comprises the client to server relationship in Kubernetes. As we have seen, in order to manipulate objects in Kubernetes, we need two things. The first is the command-line tool `kubectl`. The second is a client certificate, as you can see in figure 1.11.

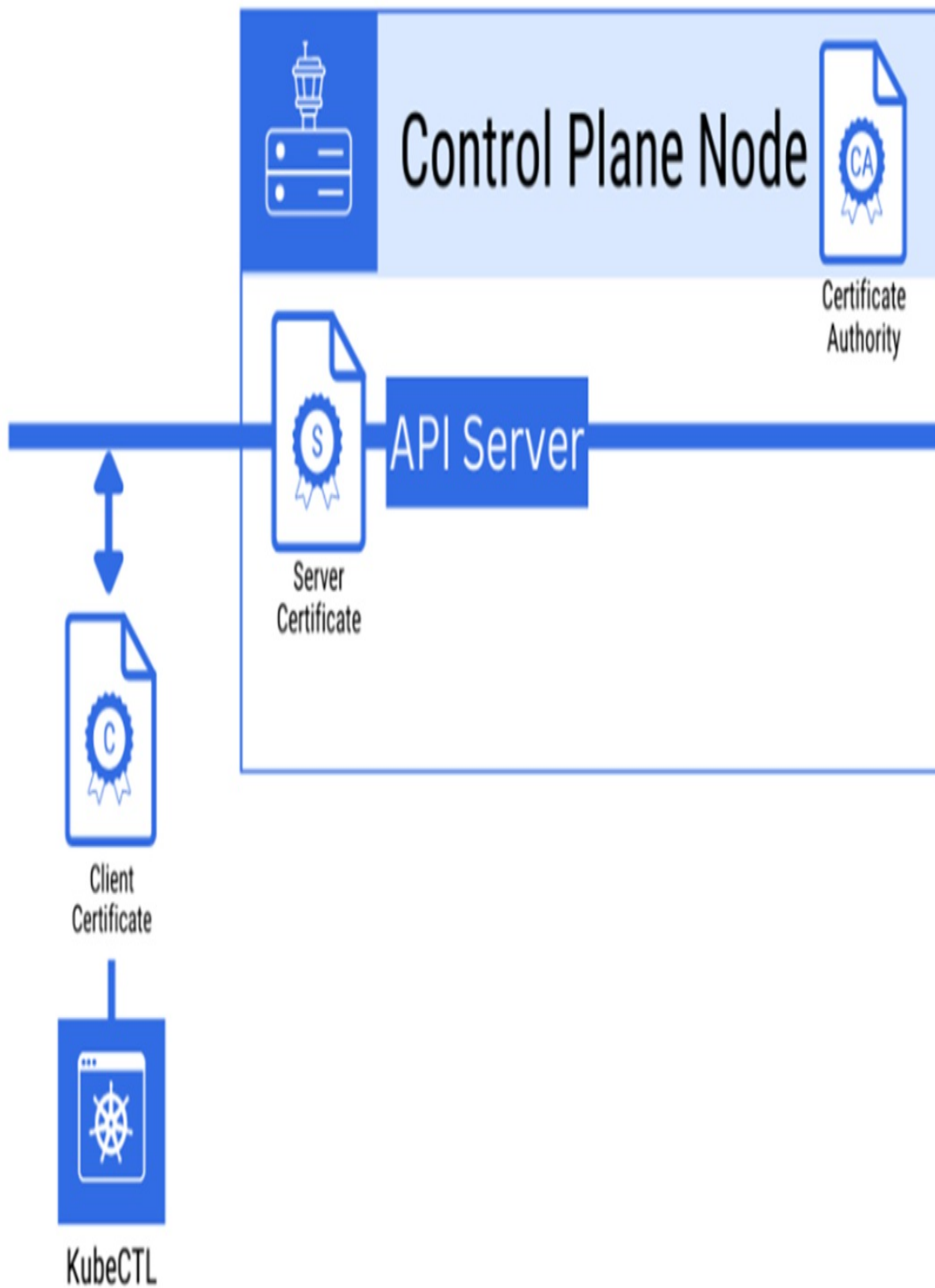
Figure 1.10 In order to manipulate Kubernetes objects, we must authenticate with a client certificate



Public Key Infrastructure (PKI) is a very common client/server communication pattern, and it is the usual way our computers securely communicate with websites via web servers. At a high-level, PKI enables secure communication over the web by ensuring that the website that you're visiting is actually the website that you intended to visit (via cryptographic signature). This ensures that you're visiting the right website (not an imposter's website), but also ensures that nobody is snooping or intercepting the traffic back and forth. If you think about accessing your bank over the web, this PKI infrastructure is absolutely critical to ensuring that bank accounts are safe and the people accessing them are indeed the owners of the accounts and the bank you are going to is indeed the bank you intend to go to. The fundamental pieces of the Public Key Infrastructure (PKI) are three things, which are shown in figure 1.12. First and foremost is the *Certificate Authority (CA)*. The CA is the source of truth, and signs the server certificate (in the case of one used with the API) and subsequently, the client is able to tell whether the server is valid. On the web, for example, the common certificate authorities are DigiCert, Symantec, and Thawte and are already trusted by the browser (the client) as a way for your browser to quickly verify a website's identity (e.g. whether google.com is actually google). The second and third pieces of the PKI puzzle are the server and client, both relying on the certificate authority, the client trying to verify the identity of the server, and the server trying to authenticate and to prove to the certificate authority

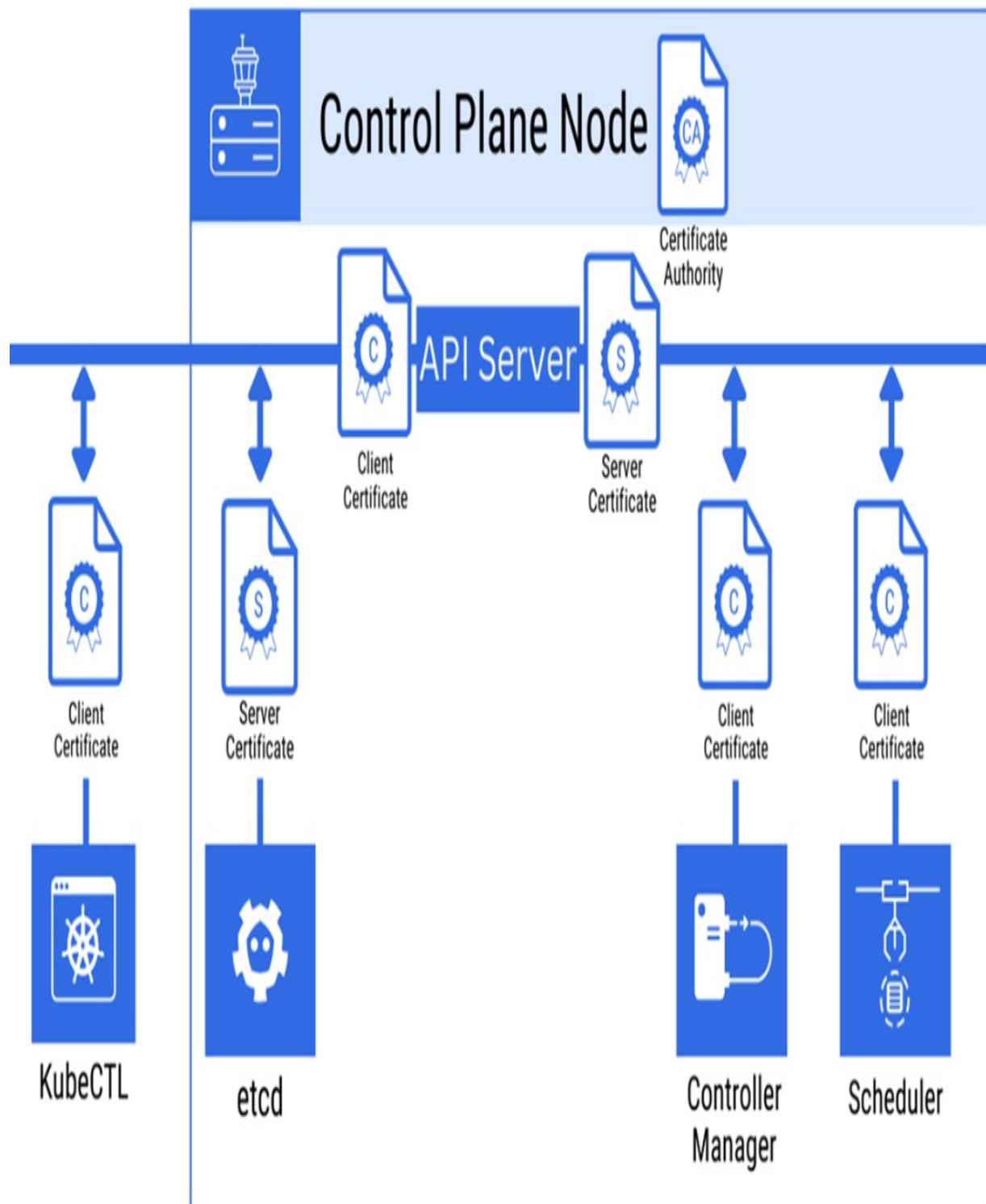
that they are who they say they are. Just like the PKI for the web, the exact same PKI model is applied to Kubernetes.

Figure 1.11 The control plane node serves as the certificate authority, which signs the certificate and provide authentication to client and server communication



Taking the same three pieces of the PKI puzzle and applying them to Kubernetes, Kubernetes is its own Certificate Authority and will be the source of truth for the other components within a Kubernetes cluster. The clients in Kubernetes (the ones checking whether the servers are who they say they are) are the kubelet, the scheduler, the controller manager, and the etcd datastore. The same Kubernetes component can be a client and a server at the same time. The servers in Kubernetes (the ones trying to prove their identity to the certificate authority) are the kubelet, the Kubernetes API, and the etcd datastore. See figure 1.13.

Figure 1.12 The Kubernetes API has multiple responsibilities in the PKI infrastructure of both client and server. The CA is generating certificates, and servers are proving their identity.



Luckily, you don't have to worry about creating a certificate authority, client, or server certificate. *kubeadm* will do all of this for you. *kubeadm* is another

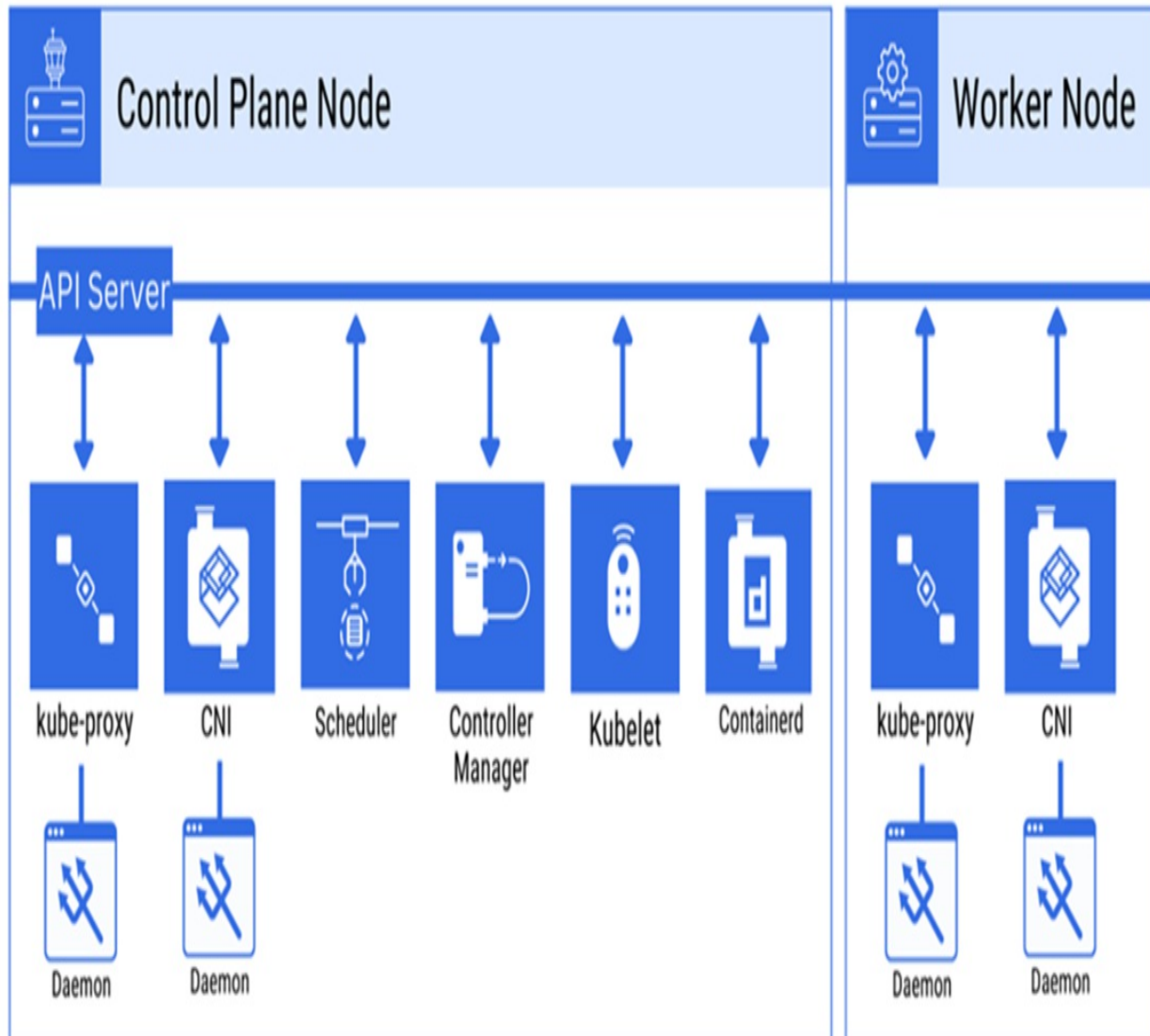
command-line tool like `kubectl`, but its purpose is for creating the necessary components that make up our Kubernetes cluster (including the certificate authority and certificates) and sometimes we call this “bootstrapping the cluster”. We’ll talk more about `kubeadm` and the location of all of these certificates in chapter 3.

1.7 Linux background processes

With all this talk about processes and components of both the control plane and the worker nodes, we should mention Linux Background Processes.

Linux Background Processes are a grouping of files on the Linux operating system that provide a constantly running software program that runs in the background. If you remember, we referred to certain processes that run on the control plane as Daemons. Daemons have been used in the Linux world for ages. If you are more familiar with Windows computers, this same thing is called a “service”. Not to be confused with a *DaemonSet* in Kubernetes, which makes sure the daemon is running on each node as a pod in the Kubernetes cluster at all times, as shown in figure 1.14.

Figure 1.13 *DaemonSets ensure that a single pod is running on each node in the cluster. Kube-proxy and CNI both run as a DaemonSets.*



For that matter, the control plane components that we spoke about earlier are a good example of that. You can have a DaemonSet running on each node that provides the kube-proxy component, for example. See for yourself by running the command `kubectl get ds -A`.

```
$ kubectl get ds -A
```

NAMESPACE	NAME	DESIRED	CURRENT	READY	UP-TO
kube-system	kube-flannel-ds	1	1	1	1
kube-system	kube-proxy	1	1	1	1

There is one background process, however, that you don't want Kubernetes to manage (as a DaemonSet), and that is kubelet. *Kubelet* is one of the only

background processes in the context of Kubernetes that will be located on the Linux system itself (as a Linux background process). The reason why I mention this is because you must know this for the exam. This is Because the Linux process may be down, presenting a scenario on the exam for you to repair it. Knowledge about where this process is and how to repair it will come in handy for the exam. To see a list of all background processes on your Linux system type the command `sudo systemctl list-unit-files --type service --all | grep kubelet -a6`.

```
$ sudo systemctl list-unit-files --type service --all | grep kube
ip6tables.service          enabled          enable
iptables.service          enabled          enable
irqbalance.service         enabled          enable
keyboard-setup.service     enabled          enable
kmod-static-nodes.service  static           enable
kmod.service               static           enable
kubelet.service            enabled          enable
logrotate.service          static           enable
lvm2-lvmpolld.service      static           enable
lvm2-monitor.service       enabled          enable
lvm2-pvscan@.service       static           enable
lvm2.service               masked           enable
man-db.service             static           enable
```

You will see a list of processes that will show the unit file (the process name) and the state of the process (enabled/disabled/masked/static). In this list, you will see `kubelet.service` as one of these. Once you've identified that the `kubelet` process is running on your system, you can do one of three things to it. If it is stopped, you can start it by typing `systemctl start kubelet`. If you notice that the process is not started when the node is rebooted, then you can enable it with the command `systemctl enable kubelet` so that it automatically starts up when the node starts up (or restarts). Finally, if you simply want to check the status of the process to see if its active or inactive, you can run the command `systemctl status kubelet`.

Within the list of services, you may have noticed the `systemd.journald` process. If you didn't, go ahead and run the following command to show it: `sudo systemctl list-unit-files --type service --all | grep journald.service`.

```
$ systemctl list-unit-files --type service --all | grep journald.
```

`systemd-journald.service`

`static`

`enabled`

JournalD (another Linux background process) is used to collect logs about the kubelet process on each node. Journalctl is a command-line utility for viewing logs on a Linux system collected by systemd, systemd being the primary daemon in Linux that controls all processes. This logging mechanism can prove very useful during the exam. You may find yourself digging through the logs, in order to find out why the kubelet process is failing, for example. Use the commands `sudo journalctl -u kubelet` and `sudo journalctl -u containerd` as a reference.

You could also check the `/var/log/pods` directory for pod logs, however, you can also retrieve these in the exact same fashion with the `kubectl logs` command (e.g. `kubectl logs kube-controller-manager-server1 -n kube-system`). More on troubleshooting and collecting these logs in chapter 8. For now, just know that these Linux background processes are important for collecting valuable information about the Kubelet.

Practice Exercises

List the background processes on your Linux operating system that are associated with Kubernetes.

List the status of the kubelet process running on the Kubernetes node and output the result to a file named `kubelet-status.txt`.

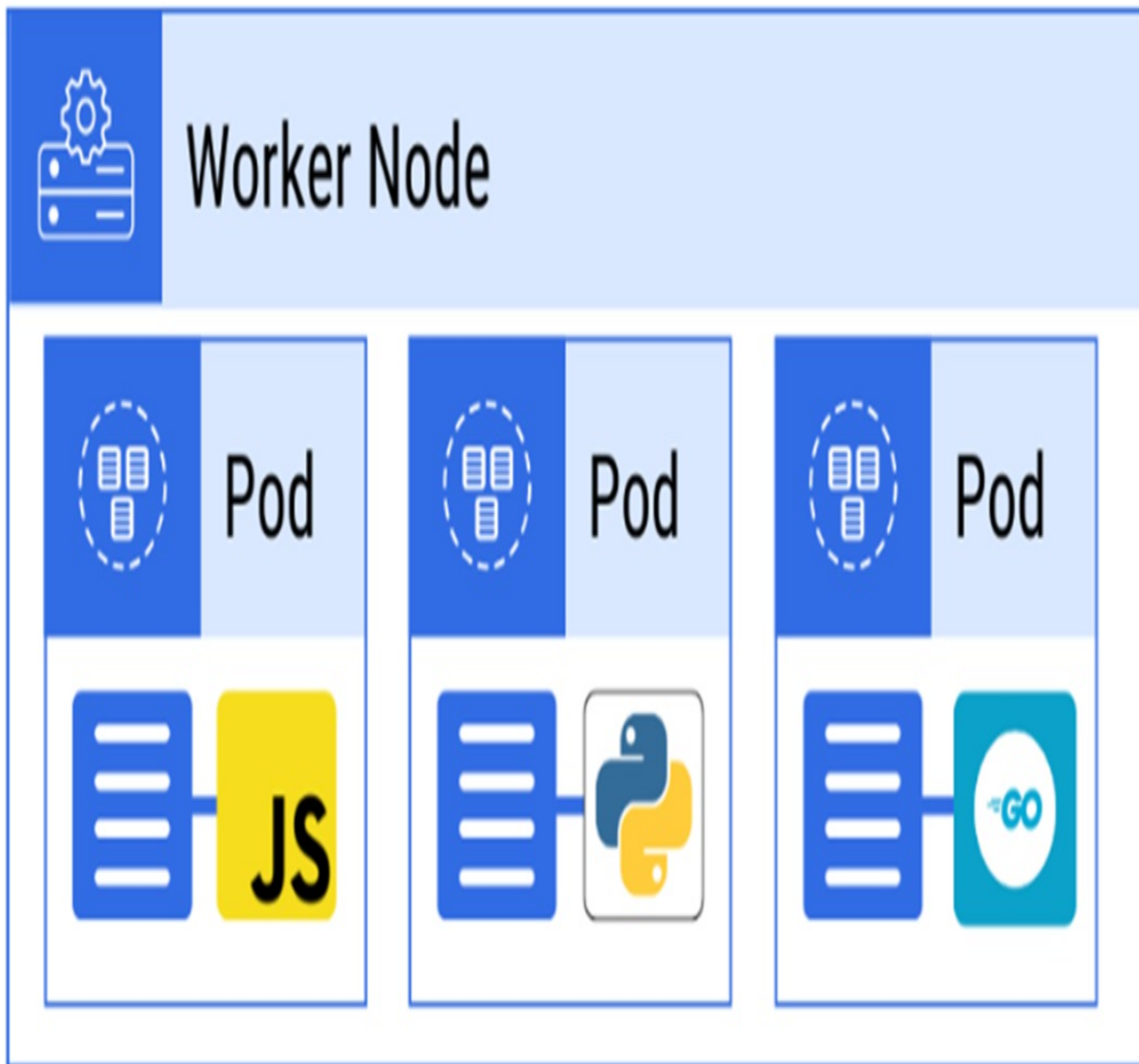
Use `systemctl` to list all the processes created on your Kubernetes control plane node.

1.8 Declarative syntax

Now that you have a good idea of what a Kubernetes cluster is composed of and its underlying processes, I would be remiss not to mention the primary purpose of Kubernetes, which is to run applications. Yes, once you have the foundation (what we've talked about so far in this chapter) of your control plane components, worker node components, Linux background processes, and the cluster is functioning as a loosely coupled unit (remember the API

model?), it's time to run your applications on top of that. After all, this is the primary function and is the reason why we're using Kubernetes in the first place. As far as running these applications on Kubernetes, Kubernetes doesn't really care. You can run a Java application on Kubernetes, just the same as a .NET application, Kubernetes doesn't care at all about the language or how the application runs, as shown in figure 1.15. It will run your containers in the same way in each and every instance and does so keeping that same foundational benefit to containers, which inherently run in the same way on every machine because the application binaries and libraries are packaged up along with the container.

Figure 1.14 In a true microservices architecture, each microservice is decoupled from other services, therefore, it can be built in the language that suits the service best. Here you can see three pods running three different applications, in JavaScript, Python, and Go.



So, if Kubernetes doesn't care about the language or the framework of the application, how do applications get deployed on Kubernetes? This brings us to a very important word in DevOps and cloud-native, that is *declarative*. You will find that building applications on Kubernetes in a declarative way is far better than imperative. Why? Imperative is nothing more than running a series of commands in a specific sequence. This makes it hard to track these commands and detect if a command failed while executing those imperative commands in sequential order. Declarative is more descriptive and succinct and has the following major benefits:

- Describing your configuration via YAML file, which can be checked into version control
- Build with the end state in mind, without considering order or specific commands to run
- The efficiency and speed to get up and running through parallel actions

YAML is a human readable language, also rendered for machines, so creating this configuration file is fairly simple. Most resources follow the same pattern and structure for the YAML and how it's made. Here's an example of a file named "my-pod-manifest.yaml":

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: nginx
    name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
```

These YAML files (called manifests) can be referred to years later, which is a good form of documenting how the application was built. Checking it into version control will allow you to track changes and work in teams to develop and enhance the configuration over time. You have the freedom to build applications running on Kubernetes with the end in mind. This is called goal-seeking. You don't have to concern yourself with the path to reach that end state, you simply submit the YAML file to the Kubernetes API, and the API chooses the best path to complete your build. Interestingly enough, submitting your file to the API is the same as performing a POST request to the API. By the nature of the API having visibility into everything running in your cluster, actions are able to be processed in parallel and reach your end state faster. Here's an example of running an imperative command (using `kubectl`) to create a pod via the YAML file declaratively:

```
$ kubectl create -f my-pod-manifest.yaml
pod/nginx created
```

There are three different ways you can submit this file to the Kubernetes API

using `kubectl`. By either using the command `kubectl create -f my-pod-manifest.yaml`, the command `kubectl apply -f my-pod-manifest.yaml`, or the command `kubectl replace -f manifest.yaml`. The *create* command expects that no resource has been created yet, and if there is, it will throw an error. The *apply* command, on the other hand, can be used to either create or update an existing resource. The *replace* command will delete the resource if its there and create an entirely new one. This means it will be as if you deleted the resource manually and then created a new one. If you are using the replace command on a deployment, for example, this will trigger a new rollout, where the pods within that deployment will be gracefully terminated and new pod replicas will be spun up. We'll talk about deployments in more detail in chapter 4, and subsequently rollouts in Chapter 5.

Idempotent is another important word to remember. The benefit to working with the Kubernetes API is that the state of the resources within will remain unchanged, no matter how many times the same call is made to the API. This creates consistency in your environment, and you know what to expect as your infrastructure and applications get increasingly complex over time.

Now that we've talked about Kubernetes to exhaustion and discussed the complexities and complications that we may or may not run into, let's dive into the next chapter and get more hands-on with our approach to learning Kuberntetes. After all, you'll have your hands on the keyboard for the exam, so why not dive into practicing and simulating the exam experience? Roll up your sleeves!

1.9 Summary

- In this chapter, you've met Kubernetes. You now know what Kubernetes looks like inside and out and what it's commonly used for. You should also now know:
- Kubernetes is an application that runs other applications and is built much like other RESTful web apps. The API is the central hub for authentication, authorization, and communication in the Kubernetes cluster
- A cluster has many different components, including two main parts; a control plane, which is handled by the control plane nodes, and the

worker nodes, of which are responsible for running the workloads (i.e. the containerized applications running on top of Kubernetes)

- Linux background processes are important in Kubernetes because they are responsible for keeping Kubernetes running on the host itself.
- You can access the cluster in two ways, programmatically or using a tool called kubectl, which both require a certificate to authenticate. This certificate is common in a PKI (Public Key Infrastructure) system in which checks in with the CA (Certificate Authority) to ensure that the certificate is valid and that communication can happen between components in the Kubernetes cluster.
- Kubernetes was built with microservices in mind, meaning, large microservices applications are able to be run more efficiently in Kubernetes, being that one service is decoupled from the overall application.
- Running microservices on Kubernetes is more efficient via declarative approach. This way, we can describe what we want the end state to be, as opposed to running a set of imperative commands to achieve the same result.

2 Kubernetes cluster

This chapter covers

- Control plane and worker node components in a multi-node cluster
- Upgrading control plane components with kubeadm
- Pod and node networking in Kubernetes
- Extension interfaces (CNI, CSI, and CRI)
- Taints and tolerations in Kubernetes

The Kubernetes cluster architecture is almost impossible to get a grasp of without getting inside an actual running cluster and discovering the components firsthand. No matter how you look at it, as Kubernetes administrators, we have to know what's going on under the hood. By the time you finish this chapter, you will feel comfortable with accessing all the components in a Kubernetes cluster, how to upgrade components when needed and backing up the cluster configuration.

The cluster architecture, installation and configuration

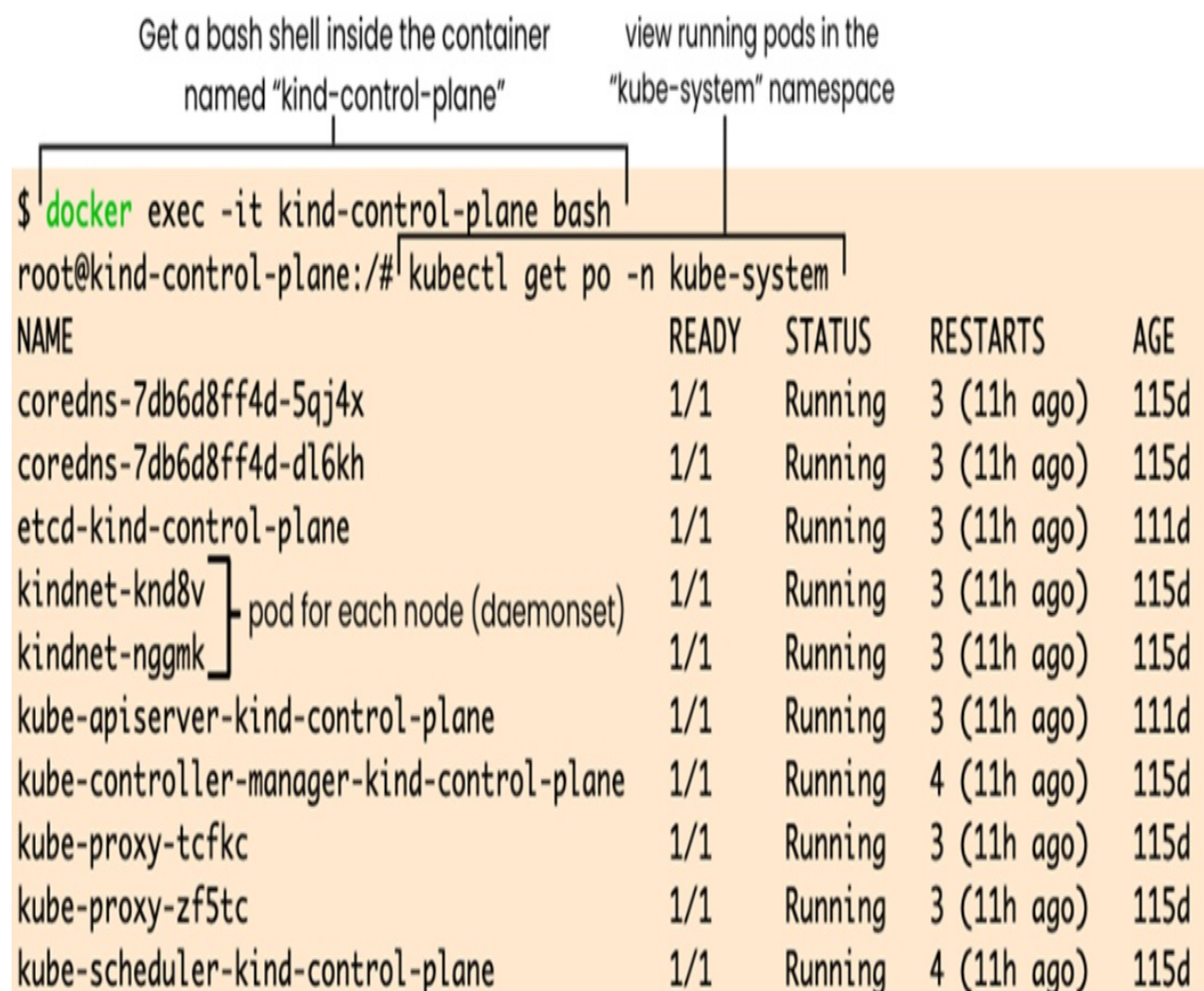
This chapter covers part of the cluster architecture, installation, and configuration domain of the CKA curriculum. This domain covers what a Kubernetes cluster consists of, including the key components and ways to maintain a healthy Kubernetes cluster. It encompasses the following competencies.

Competency	Chapter Section
Prepare underlying infrastructure for installing a Kubernetes cluster	2.1
Create and manage Kubernetes clusters using kubeadm	2.1
Manage the lifecycle of Kubernetes clusters	2.2
Understand extension interfaces	2.3

2.1 Kubernetes cluster components

The control plane pods work together to form the control plane. Those components are the API server, the controller manager, the scheduler and ETCD. In the Kubernetes clusters that you will face on exam day, these components exist as pods running on the control plane node. We can view the cluster components by viewing the pods running in the “kube-system” namespace. View those pods with the command `kubectl get po -n kube-system` and you’ll see output similar to figure 2.1. We first run the command `docker exec -it kind-control-plane bash` to get a shell to the control plane node.

Figure 2.1 View the pods in the kube-system namespace



```
$ docker exec -it kind-control-plane bash
root@kind-control-plane:/# kubectl get po -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-7db6d8ff4d-5qj4x	1/1	Running	3 (11h ago)	115d
coredns-7db6d8ff4d-dl6kh	1/1	Running	3 (11h ago)	115d
etcd-kind-control-plane	1/1	Running	3 (11h ago)	111d
kindnet-knd8v	1/1	Running	3 (11h ago)	115d
kindnet-nggmkn	1/1	Running	3 (11h ago)	115d
} pod for each node (daemonset)				
kube-apiserver-kind-control-plane	1/1	Running	3 (11h ago)	111d
kube-controller-manager-kind-control-plane	1/1	Running	4 (11h ago)	115d
kube-proxy-tcfkc	1/1	Running	3 (11h ago)	115d
kube-proxy-zf5tc	1/1	Running	3 (11h ago)	115d
kube-scheduler-kind-control-plane	1/1	Running	4 (11h ago)	115d

In the coming sections of this chapter, we'll learn how to manage these components, including how to access them, modify their configuration, back them up, and upgrade them. Now, let's now get our hands dirty and work with them in the terminal.

2.1.1 Kubernetes version upgrade

The CKA exam will test your knowledge of maintaining a Kubernetes cluster. This includes upgrading the control plane components to a certain version. For example, the exam question will say something like this:

Exam task

There's a need for company X to upgrade the Kubernetes controller to version 1.31 or higher, due to a bug that affects pod scheduling. Perform the update with minimal downtime and loss of service.

Knowing that you should use `kubeadm` to do so, will allow you to breeze through that task on the exam and feel confident and headed toward a passing score.

If you don't already have access to an existing Kubernetes cluster, creating a Kubernetes cluster with `kind` is explained in appendix A. As soon as your `KIND` cluster is built, use the `kubectl` tool pre-installed on the control plane node. You can get a bash shell to the control plane node by typing the command `docker exec -it kind-control-plane bash` and you can follow along. The beauty of having a cluster that is built with `kubeadm` is we can also use `kubeadm` to view the versions of our control plane components, as shown in figure 2.2, and upgrade the cluster. Using the command `kubeadm upgrade plan` from the control plane will get a list of the control plane components and tell you what version they are currently on in the "CURRENT" column. It will also tell you what version they can be upgraded to in the "TARGET" column.

Figure 2.2 View the control plane components and how to upgrade them if necessary

Get the current version of the control plane components
and see which version they can be upgraded to

```
root@kind-control-plane:/# kubectl upgrade plan
[preflight] Running pre-flight checks.
[upgrade/config] Reading configuration from the cluster...

Upgrade to the latest version in the v1.31 series:
```

	COMPONENT	NODE	CURRENT	TARGET
control plane components	kube-apiserver	kind-control-plane	v1.31.4	v1.31.5
	kube-controller-manager	kind-control-plane	v1.31.4	v1.31.5
	kube-scheduler	kind-control-plane	v1.31.4	v1.31.5
	kube-proxy		v1.31.4	v1.31.5
	CoreDNS		v1.11.3	v1.11.3
	etcd	kind-control-plane	3.5.15-0	3.5.15-0

```
You can now apply the upgrade by executing the following command:

kubectl upgrade apply v1.31.5
```

NOTE

The TARGET column doesn't show the latest release of Kubernetes. This is because we can only update to the current version of kubectl. If kubectl was at version 1.32.1, the target column would show "v1.32.1". To upgrade kubectl from 1.31.5 to 1.32.1, run the command `apt update; apt install -y kubectl=1.32.1-00`.

If you must upgrade kubeadm to get to the latest release, first download the GPG key, and add kubeadm to your local apt packages (still in a shell on the control plane node). To download the GPG key, run the command `sudo curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release.key | gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg`. To add kubeadm to your local apt packages, run the command

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

Having done that, rerun the command `apt update; apt install -y kubeadm=1.32.1-00` to upgrade kubeadm to version 1.32.1. When you run the `kubeadm upgrade plan` command again, and you'll notice that your TARGET column has a higher version of 1.32.1, which you can upgrade to. See the abbreviated output from this command in figure 2.3.

Figure 2.3 When you upgrade kubeadm to a higher version, you are able to upgrade your control plane components to that higher version as well


```
root@kind-control-plane:~# curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release.key | gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

Get GPG key for
Kubernetes

```
root@kind-control-plane:~# echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.32/deb/ /' | tee /etc/apt/sources.list.d/kubernetes.list
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.32/deb/ /
```

Copy the sources into
local apt repository

```
root@kind-control-plane:~# apt update; apt install -y kubeadm=1.32.1-00 ← Install version 1.32.1-00 of kubeadm
```

Upgrade to the latest version in the v1.32 series:

COMPONENT	NODE	CURRENT	TARGET
kube-apiserver	kind-control-plane	v1.32.0	v1.32.1
kube-controller-manager	kind-control-plane	v1.32.0	v1.32.1
kube-scheduler	kind-control-plane	v1.32.0	v1.32.1
kube-proxy		1.32.0	v1.32.1
CoreDNS		v1.11.3	v1.11.3
etcd	kind-control-plane	3.5.16-0	3.5.16-0

The target version has
changed because kubeadm
version has changed

You can now apply the upgrade by executing the following command:

```
kubeadm upgrade apply v1.32.1
```

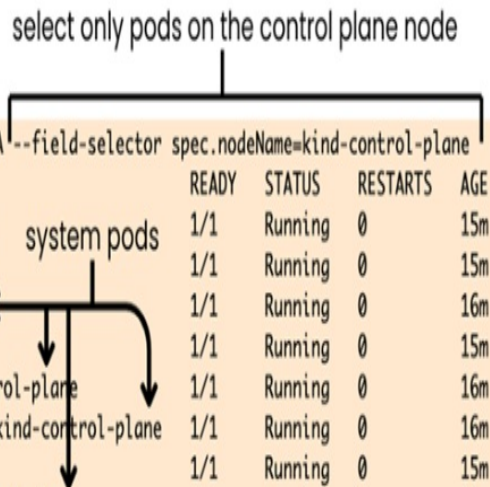
Congratulations! You’ve successfully upgraded the control plane components of a Kubernetes cluster, also known as “Upgrading Kubernetes”.

2.1.2 The control plane

The control plane and worker nodes have different responsibilities. Worker nodes carry the application workload and run the pods that contain those application containers whereas the control plane runs an initial set of pods, comprised of the control plane components we just looked at in upgrading the cluster. We refer to them as system pods, because they contain the necessary structure around the running of the entire Kubernetes system. The “kube-apiserver”, “kube-controller-manager”, “kube-scheduler”, and “etcd” all run as system pods on the control plane node. These system pods will exist

regardless of where your Kubernetes cluster resides or how it was built, as they are essential to the core of Kubernetes. You can see the system pods by running the command `k get po -o wide -A --field-selector spec.nodeName=kind-control-plane` and you'll see an output similar to figure 2.4.

Figure 2.4 Viewing system pods that reside on the control plane node

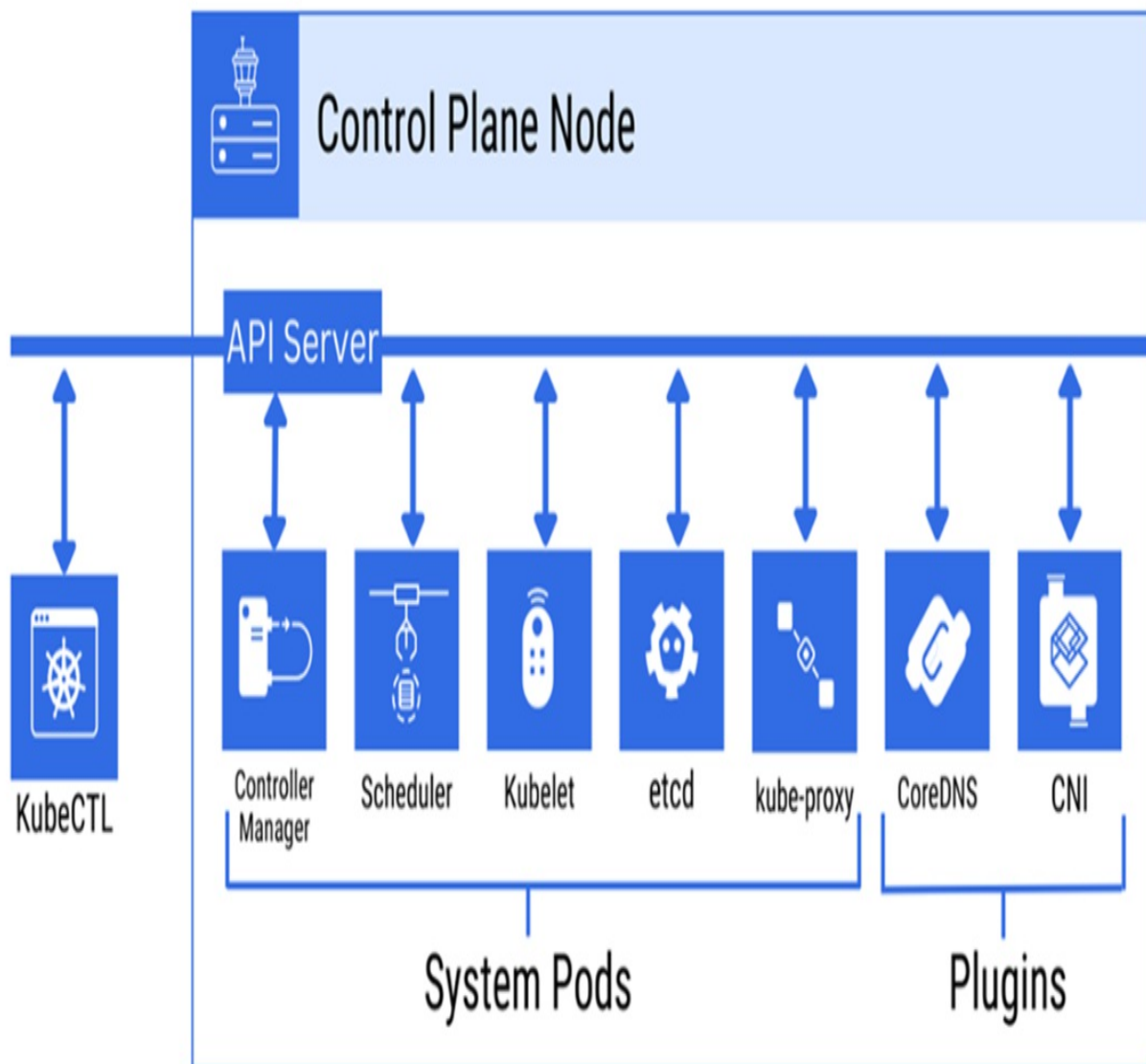


```
root@kind-control-plane:/# k get po -o wide -A --field-selector spec.nodeName=kind-control-plane
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
kube-system	coredns-668d6bf9bc-6mvtb	1/1	Running	0	15m	10.244.0.4	kind-control-plane
kube-system	coredns-668d6bf9bc-tb2rp	1/1	Running	0	15m	10.244.0.2	kind-control-plane
kube-system	etcd-kind-control-plane	1/1	Running	0	16m	172.18.0.2	kind-control-plane
kube-system	kindnet-jvxkz	1/1	Running	0	15m	172.18.0.2	kind-control-plane
kube-system	kube-apiserver-kind-control-plane	1/1	Running	0	16m	172.18.0.2	kind-control-plane
kube-system	kube-controller-manager-kind-control-plane	1/1	Running	0	16m	172.18.0.2	kind-control-plane
kube-system	kube-proxy-29rm4	1/1	Running	0	15m	172.18.0.2	kind-control-plane
kube-system	kube-scheduler-kind-control-plane	1/1	Running	0	16m	172.18.0.2	kind-control-plane
local-path-storage	local-path-provisioner-58cc7856b6-k552f	1/1	Running	0	15m	10.244.0.3	kind-control-plane

Coredns is also a pod that runs on the control plane node, but it's considered a plugin, not a core system pod. Also, the kube-proxy pod will run on all nodes, regardless of whether it's a control plane node or a worker node. Let's just focus on the core components, and look at figure 2.5 to get a better visualization of the core components of a control plane node.

Figure 2.5 System pods and plugins on the Kubernetes control plane node



Each pod plays a specific role in the operation of Kubernetes. The controller manager pod maintains cluster operations, ensuring deployments are running the correct number of replicas. The scheduler pod is responsible for detecting available resources on a node, in order to place a pod on that node. The act of assigning a pod to a node is called scheduling, so from now on when we say “a pod is scheduled to a node”, we mean that a new pod has now been started on that node. The API server pod will expose the API interface, and is the communication hub for all other components, including those components on a worker node and control plane node. Finally, the etcd pod is the etcd datastore for storing cluster configuration data, as we’ve talked about in this

chapter as well as the previous chapter.

2.1.3 Taints and tolerations

By default, application pods will not run on the control plane node because it is protecting the system pods. It's kind of like partitioning your drive into two, one for the OS and one for files and folders. In order to protect the control plane from running out of system resources like CPU and RAM which would threaten the system pods, the control plane has a special attribute assigned to it called a taint. A taint will repel work, meaning it will disable scheduling to that node, unless a certain specification exists in the YAML spec called a toleration. Let's look at what a taint looks like on our control plane node (`docker exec -it kind-control-plane bash`) by typing the command `kubectl describe node kind-control-plane | grep Taints`

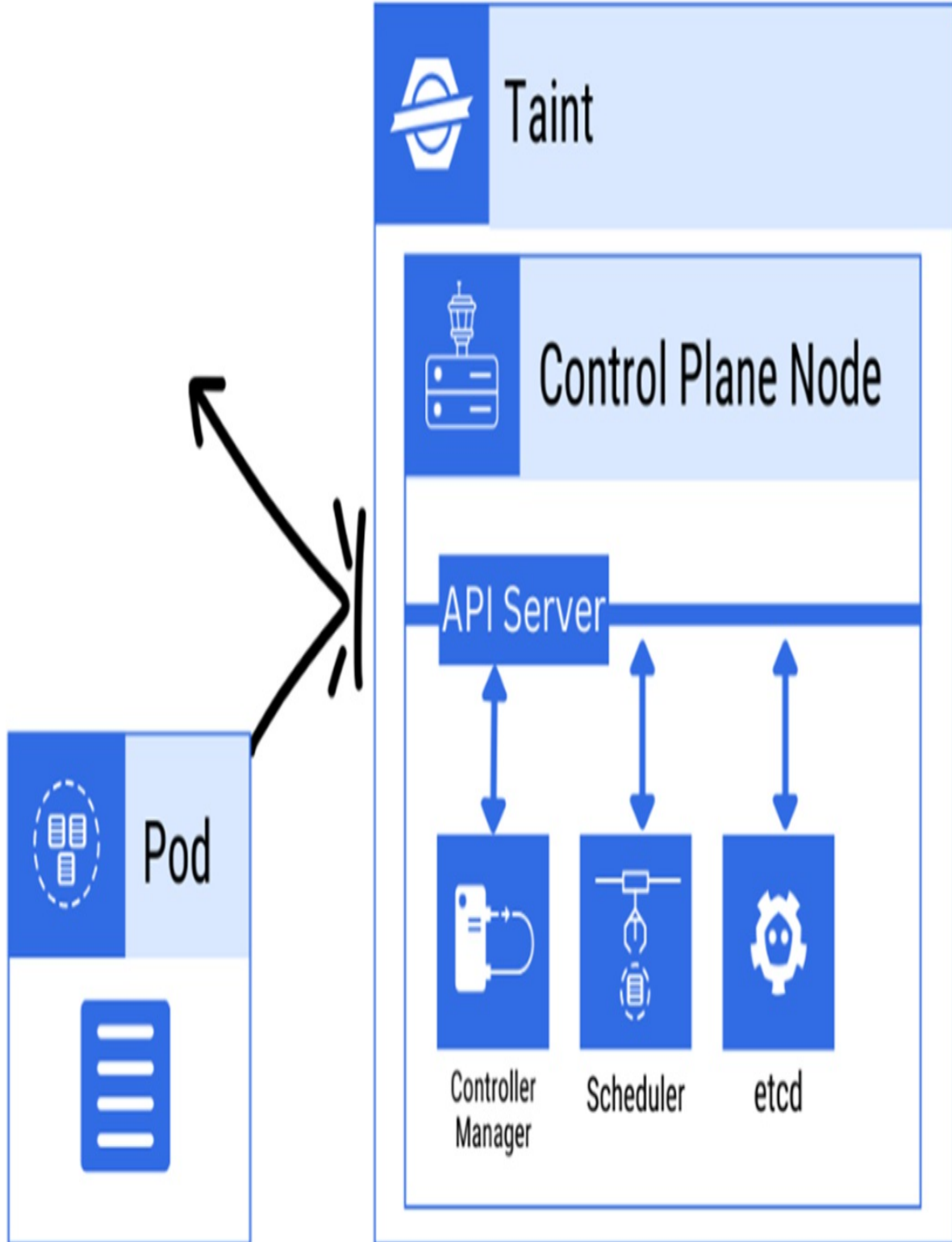
```
root@kind-control-plane:/# kubectl describe node kind-control-plane
Taints:          node-role.kubernetes.io/control-plane:NoSchedule
Taints:          <none>
Taints:          <none>
```

NOTE

The taint will not be shown on a single node cluster created with kind.

As we can see, a taint has been applied to our control plane node as a part of the cluster creation process and the taint is `node-role.kubernetes.io/control-plane:NoSchedule`. The taint has three parts to it: a key, an effect, and a value. A taint doesn't need to have a value, but must have a key and effect. So, in this case, the key is `node-role.kubernetes.io` and the effect is `NoSchedule`. This means that unless there is a toleration for that taint, the pod will not get scheduled to that node with that taint. In figure 2.6 you'll see a demonstration of this, as the pod doesn't have a toleration for the taint, so it's not scheduled to that node.

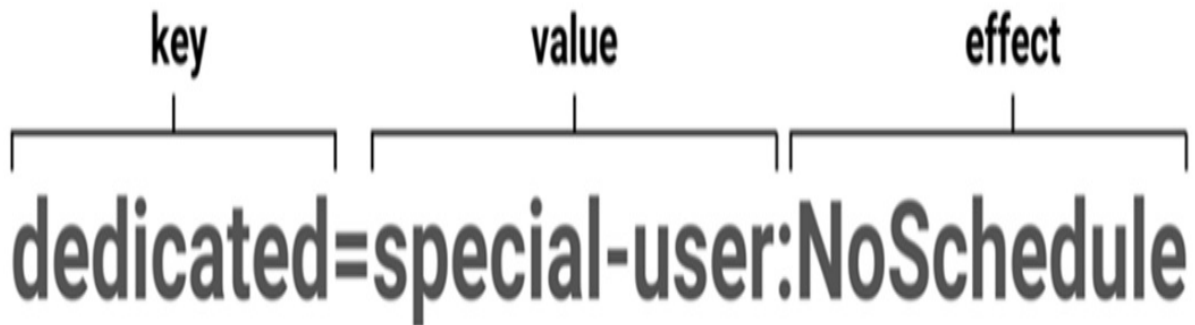
Figure 2.6 A taint is applied to the control plane node, and there is no toleration, therefore the pod is not scheduled



Let's apply a taint to one of our worker nodes, and see what a taint with a

key, effect and value looks like with the command `kubectl taint no kind-worker dedicated=special-user:NoSchedule`, as deconstructed in figure 2.7 to show which item is the key versus which item is the value and effect.

Figure 2.7 Breakdown of the parts of a taint into key, value, and effect



So, a full taint (key, value, and effect) will be appropriate when you want to be more specific about the qualification a pod must have to “get past” a taint. Yes, there’s a way to schedule a pod to a node even though that node may have a taint applied. This is called a toleration. This is important to remember because tolerations do not mean that you are selecting that particular node, it just means that the scheduler may choose to schedule the pod there if the conditions are right, amongst all the other nodes. This is how pods like the dns pod are running on the control plane. These pods have toleration for the taint. To view the tolerations within the pod YAML, run the command `kubectl get po coredns-668d6bf9c-6mvbt -o yaml -n kube-system | grep tolerations -A14`. This command will get the yaml output of the pod named ‘core-dns-668d6bf9c-6mvbt’ in the kube-system namespace and then use the grep tool to filter the results and give us 14 lines after the result. The output will be similar to figure 2.8, and you can compare that to the taint that’s placed on the control plane node.

Figure 2.8 Comparison between how the taint is set and the toleration for it in the coredns pod

```
root@kind-control-plane:/# k -n kube-system get po coredns-668d6bf9bc-6mvbt -o yaml | grep tolerations -A14
tolerations:
- key: CriticalAddonsOnly
  operator: Exists
- effect: NoSchedule
  key: node-role.kubernetes.io/control-plane
- effect: NoExecute
  key: node.kubernetes.io/not-ready
  operator: Exists
  tolerationSeconds: 300
- effect: NoExecute
  key: node.kubernetes.io/unreachable
  operator: Exists
  tolerationSeconds: 300
volumes:
- configMap:
```

This effect and key must match exactly with the taint of the node

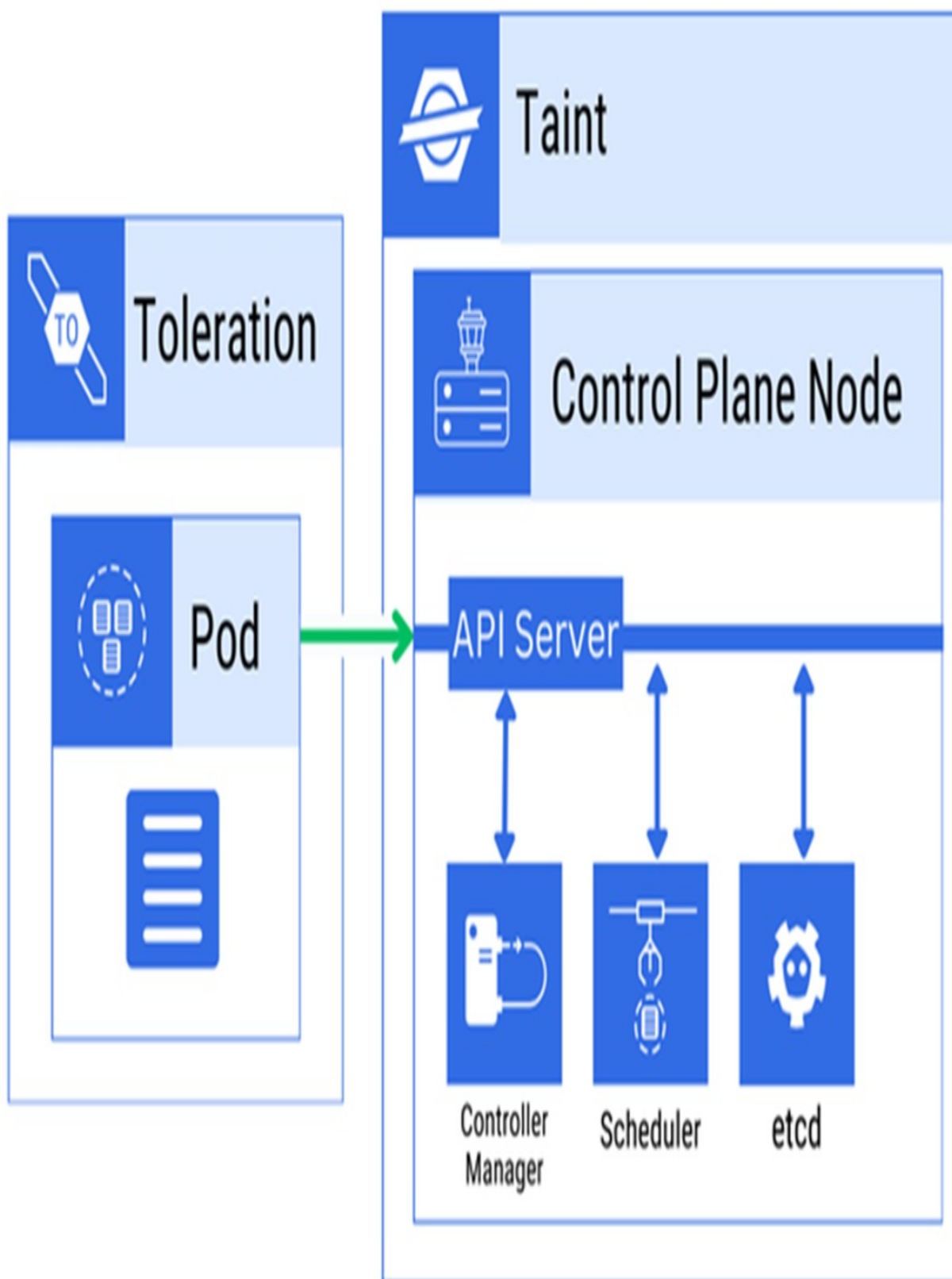
```
root@kind-control-plane:/# k describe no kind-control-plane | grep Taints
Taints:          node-role.kubernetes.io/control-plane:NoSchedule
```

As we see in the YAML spec for this coredns pod, within the list of tolerations, a key “node-role.kubernetes.io/control-plane” and the effect “NoSchedule” which match our taint. This is how the coredns pod was able to get scheduled to the control plane node. Remember that your core DNS pod will have a different name, so you’ll want to replace “core-dns-64897985d-4th9h” with the name of your core-dns pod. If you don’t know the name of that pod, you can get the name by typing `kubectl get po -A`

As we see in this case, the toleration must match the key and the effect exactly. If it’s off by even a little, then the pod will not get scheduled to the intended node. So, as you can imagine, having more loosely defined rules for tolerations makes it easier to define a toleration when you’re writing the YAML. In figure 2.8, you may have also noticed other fields like “operator” and “tolerationSeconds”. Operator, if not defined explicitly, like in the toleration for the node-role of control-plane taint, will default to ‘Equal’ of the two possible values of ‘Exists’ or ‘Equal’. Since the operator was not defined for the node-role taint toleration, it defaulted to Equal. A toleration matches a taint if they keys and effects are the same and the operator is equal. If there is a value, then the toleration matches a taint if the keys, effects, and values all match exactly and the operator is ‘Equal’. If the operator is ‘Exists’, no value should be specified but the key and the effect must match. So, referring to the command output in figure 2.8, the effect “NoExecute”

and key “node.kubernetes.io/not-ready” must match exactly since the operator is ‘Exists’. The “tolerationSeconds” is an optional parameter that allows you to specify how long the pod will stay bound to the node after the taint is added, in this case 300 seconds. There is an example of this In figure 2.9, where the toleration matches the taint of the node, therefore, it's successfully scheduled to the node.

Figure 2.9 The toleration applied to the pod ensures that it can be scheduled to the tainted node



Let's now create a pod that has a toleration for the taint that we had applied to kind-worker. This taint had an effect of "NoSchedule", a value of "special-user" and a key named "dedicated." So, this means that we must match all of those while using the default operator "Equal." Let's start with building out a pod template with the command `k run pod-tolerate --image=nginx --dry-run=client -o yaml > pod-tolerate.yaml`

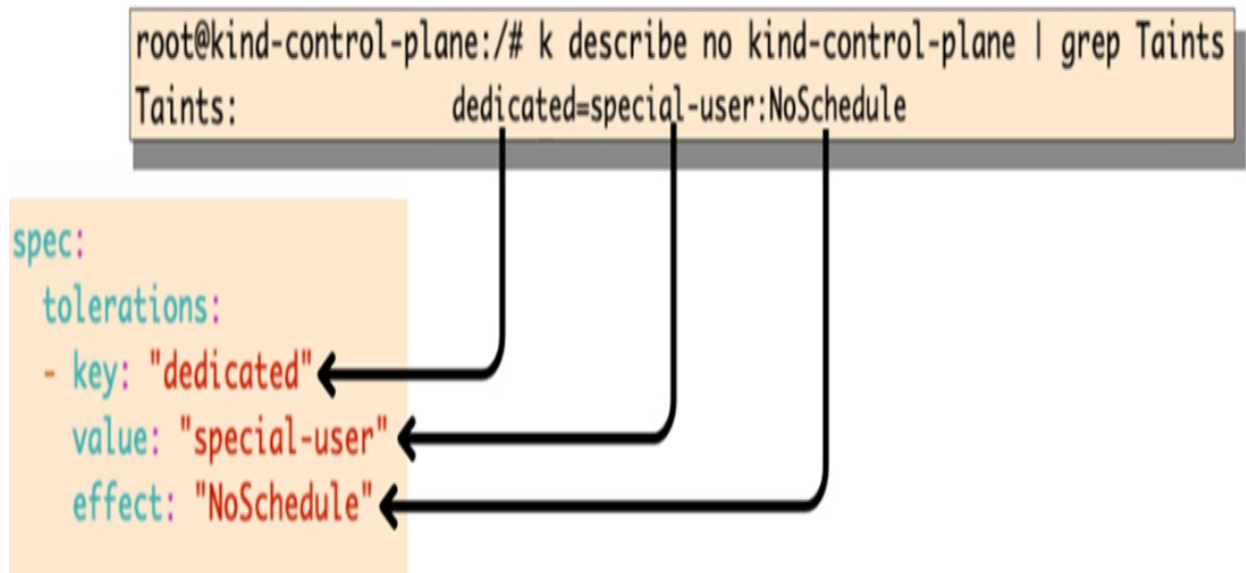
```
root@kind-control-plane:/# k run pod-tolerate --image=nginx --dry-run=client -o yaml > pod-tolerate.yaml
root@kind-control-plane:/# cat pod-tolerate.yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: pod-tolerate
  name: pod-tolerate
spec:
  containers:
  - image: nginx
    name: pod-tolerate
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

NOTE

We built this pod template with the dry-run flag and sent the output of the command to a file, which is much easier than typing out the file from scratch.

Now let's open the file "pod-tolerate.yaml" using the vim text editor with the command `vim pod-tolerate.yaml`. We can add the tolerations we need to match the taint that's applied to our worker node. We see in figure 2.10 by the output of the command `kubectl describe no kind-worker | grep Taints`, the key is "dedicated", the value is "special-user" and the effect is "NoSchedule". With the file still open, in line with the word "containers", insert "tolerations:". Just below that, insert the line "- key: "dedicated"", followed by "value: "special-user"" and "effect: "NoSchedule"". You can see exactly what it should look like in figure 2.10.

Figure 2.10 How tolerations for a taint are applied in order to effectively schedule a pod



Once you've added the toleration to the pod YAML manifest, you can save and close the file. Then you can submit the YAML to the API server with the command `kubectl create -f pod-tolerate.yaml` and the pod will be created in the default namespace. You can see which node the pod is running on with the command `kubectl get po -o wide`

```
root@kind-control-plane:/# kubectl create -f pod-tolerate.yaml
pod/pod-tolerate created
root@kind-control-plane:/# k get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NO
pod-tolerate	1/1	Running	0	6s	10.244.2.6	ki

You'll see from the wide output that the pod is running on the kind-worker node, but it could have just as easily been scheduled to the kind-worker2 because tolerations don't explicitly select which node the scheduler should schedule the pod to. If you're still having trouble creating the pod, you can run this command, which already has the YAML intact, then you can compare and see where you went wrong. Just type the command `k apply -f https://raw.githubusercontent.com/chadmcrowell/k8s/main/manifests/tolerate.yaml`

Next, we'll talk about node selectors and node names that you can add to your pod YAML which will make such explicit statements to the scheduler. *Node selectors*, as the name implies, allow you to select a node that a pod

gets scheduled to, based on that node(s) label. There are a lot of labels that are applied to a node by default. You can see a list of labels by performing the command `k get no --show-labels`. Labels don't change the operation of the node, but can be used to query the nodes with a particular label or to schedule pods to that node using its label (which we'll get to in the coming sections of this chapter). Labels can also be applied to pods. You can see pod labels on pods in the `kube-system` namespace with the command `k get po -n kube-system --show-labels`. The output will be similar to figure 2.11.

Figure 2.11 Show the default labels applied to nodes and pods when a Kubernetes cluster is created

```
root@kind-control-plane:/# k get no --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
kind-control-plane	Ready	control-plane	40m	v1.32.0	beta.kubernetes.io/arch=arm64,beta.kubernetes.io/os=linux,plane,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node.kubernetes.io/exclude-from-external-load-balancing
kind-worker	Ready	<none>	39m	v1.32.0	beta.kubernetes.io/arch=arm64,beta.kubernetes.io/os=linux,kubernetes.io/os=linux

```
root@kind-control-plane:/# k get po -n kube-system --show-labels
```

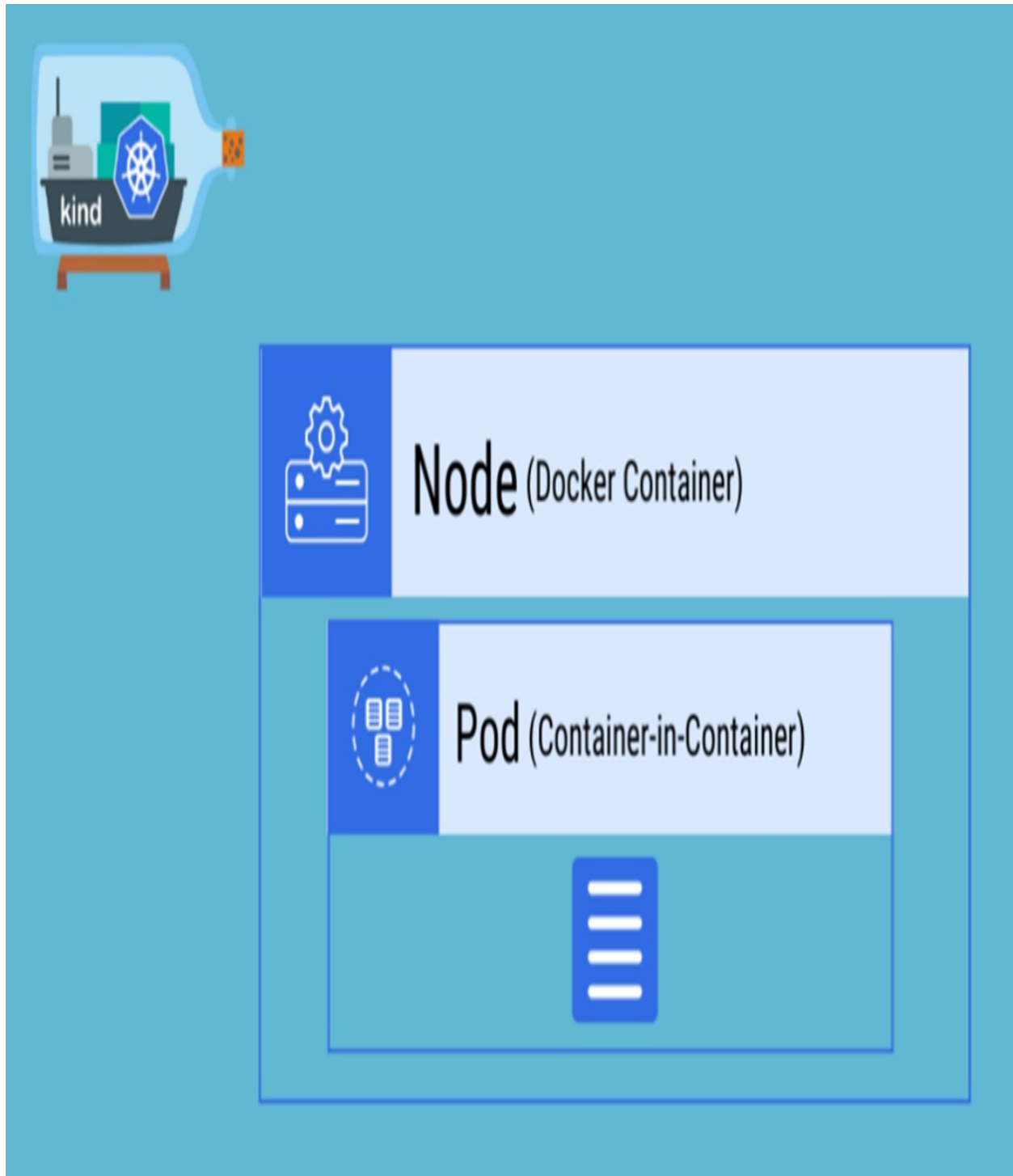
NAME	READY	STATUS	RESTARTS	AGE	LABELS
coredns-668d6bf9bc-69gc7	1/1	Running	0	40m	k8s-app=kube-dns,pod-template-hash=668d6bf9bc
coredns-668d6bf9bc-m9ld5	1/1	Running	0	40m	k8s-app=kube-dns,pod-template-hash=668d6bf9bc
etcd-kind-control-plane	1/1	Running	0	40m	component=etcd,tier=control-plane
kindnet-c8cq8	1/1	Running	0	40m	app=kindnet,controller-revision-hash=kindnet-c8cq8
kindnet-nxj2m	1/1	Running	0	40m	app=kindnet,controller-revision-hash=kindnet-nxj2m
kube-apiserver-kind-control-plane	1/1	Running	0	40m	component=kube-apiserver,tier=control-plane
kube-controller-manager-kind-control-plane	1/1	Running	0	40m	component=kube-controller-manager,tier=control-plane
kube-proxy-6tcvf	1/1	Running	0	40m	component=kube-proxy,tier=control-plane
kube-proxy-864gs	1/1	Running	0	40m	component=kube-proxy,tier=control-plane
kube-scheduler-kind-control-plane	1/1	Running	0	40m	component=kube-scheduler,tier=control-plane

Node name allows you to select a single node by its hostname. Let's keep the taint applied to the node so we can work through this together and better visualize how these node selectors are applied in Kubernetes.

2.1.4 Nodes in kind

Kind stands for Kubernetes in Docker. Kind creates a cluster's Kubernetes node inside of a Docker container. Depicted in figure 2.12, the container exists inside the pod, which is inside the node, which in this case (with kind), it's also a container comprised of a Kubernetes cluster in Docker. Kind does this by taking advantage of cgroups and namespaces. The node container is isolated by the host's Docker engine. Inside that container, there is another namespace/cgroup setup for each pod's containers. They're all still using the same host kernel underneath; the isolation is layered through the Linux kernel's features. This approach is more efficient and aligns with Kubernetes' modular approach to container runtimes (via the Container Runtime Interface or CRI). We will talk about CRI later in this chapter.

Figure 2.12 Kind cluster architecture, with nodes running as containers on our local system



Let's start by viewing the Docker container that kind created for us. You can view the Docker container with the command `docker container ls` and your output will look exactly like this, but with a different container ID:

```
$ docker container ls
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
c23f3ab00ba3	kindest/node:v1.31.0	"/usr/lo..."	4 days ago	Up

You will see the container name is “kind-control-plane” which you may have guessed consists of the control plane components. What about the worker node? Well, since we are using this cluster for testing (and for simplicity’s sake), for now we have combined the control plane and worker components in the same node, but for all production scenarios, these would be two or more separate nodes. To prevent getting sidetracked and to keep things simple, let’s continue. We’ll build some multi-node clusters later in this chapter since you will see this on the CKA exam. You can also see the nodes in a Kubernetes cluster using the `kubectl` tool with the command `kubectl get nodes` like so:

```
$ kubectl get no
```

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane	4d23h	v1.21.1

As you see in table 2.1, we can use shorthand for addressing certain resources in Kubernetes, which will come in handy for the exam as it saves a lot of keystrokes. Also in table 2.1 you can see other abbreviations or shorthand for addressing different Kubernetes resources.

Table 2.1 Kubernetes resource abbreviations and examples of use with `kubectl`

RESOURCE	ABBREVIATION	EXAMPLE
Namespace	ns	<code>k get ns</code>
Pod	po	<code>k get po</code>
Deployment	deploy	<code>k get deploy</code>
ReplicaSet	rs	<code>k get rs</code>
Service	svc	<code>k get svc</code>
Service account	sa	<code>k get sa</code>
Configmap	cm	<code>k get cm</code>
Daemonset	ds	<code>k get ds</code>
Persistent volume	pv	<code>k get pvc</code>
Persistent volume claim	pvc	<code>k get pvc</code>
Storage class	sc	<code>k get sc</code>
Network policy	netpol	<code>k get netpol</code>

Ingress	ing	k get ing
Endpoints	ep	k get ep

You can also see by typing `kubectl get no -o wide` that `containerd` is in fact used in our kind Kubernetes cluster. The `-o wide` is a way to retrieve a verbose output from the `get` command. We'll explore this more throughout the rest of this book.

```
$ kubectl get no -o wide
NAME                STATUS    ROLES          AGE   VERSION   INT
kind-control-plane  Ready    control-plane  35s   v1.31.0   172
```

Getting back to our single node cluster, let's now get a shell inside of the container in order to look inside. We can do this by typing `docker exec -it kind-control-plane bash` into our terminal, and we'll see this exact output, which means that you are operating as root inside the `kind-control-plane` container:

```
$ docker exec -it kind-control-plane bash
root@kind-control-plane:/#
```

From this point forward, we'll perform commands inside the container (after `root@kind-control-plane:/#`), as `kubectl` is already pre-installed and we can view additional information about our cluster. For example, we can use the `crictl` tool to list the control plane components as containers. The `crictl` tool is another command-line utility used to inspect and debug containers that are CRI (Container Runtime Interface) compatible. CRI is a standard for how container runtimes are built, of which, provides `kubelet` with a consistent interface for interacting with the container runtime, whether `docker`, `containerd` or any other container runtime that is CRI compliant. With the command `crictl ps`, you can list the containers running inside of our `kind-control-plane` container.

```
root@kind-control-plane:/# crictl ps
CONTAINER          IMAGE                                CREATED           STATE
1fec85d809fc4      e422121c9c5f9                      26 hours ago     Runni
1b8372af65cf2      296a6d5035e2d                      26 hours ago     Runni
85c7a1f4305f0      6de166512aa22                      26 hours ago     Runni
fe14d75b82341      296a6d5035e2d                      26 hours ago     Runni
76ab2ca16da9d      0e124fb3c695b                      26 hours ago     Runni
4198ed634fc67      1248d2d503d37                      26 hours ago     Runni
```


5aac336ec4338	0369cf4303ffd	26 hours ago	Runni
80a672211961d	96a295389d472	26 hours ago	Runni
a67abd31ab1ae	94ffe308aef9	26 hours ago	Runni

The `local-path-provisioner` is used for persistent storage in our cluster, `coredns` is used for resolving names to IP addresses in our cluster (DNS), and the `kindnet-cni` is used for pod to pod communication in our cluster. Pods are the smallest deployable unit in Kubernetes and can contain one or many containers. We'll see in future chapters that pods can run independently or as a part of a deployment, replicaSet, and statefulSet.

Let's now talk about the components that exist on a worker node. For the examples below, we can run these commands on a cluster that has one control-plane and two worker nodes. You can see in Appendix A how to set up a cluster like this. These components will be the same for each node in the cluster, whether you have just one worker node or if you have thousands of worker nodes. They all contain the kubelet, the kube-proxy and the container runtime. Let's look at the kube-proxy pods that run on the worker nodes in our cluster with the command `kubectl get po -o wide -A | grep worker`

```
root@kind-control-plane:/# kubectl get po -o wide -A | grep worke
kube-system          kindnet-fs6jt
kube-system          kube-proxy-szr7n
```

The kube-proxy creates our iptables rules and makes sure when services are created we can get to the pods associated with that service, as we mentioned before when we talked about the kube-proxy on the control plane node. The kindnet pod that we see there alongside the kube-proxy pod is for CNI, but again, considered a plugin, not part of core Kubernetes.

Next, let's take a look at the container runtime that's installed in each node. For this, we'll run the command `kubectl get no -o wide`. The following is an abbreviated output, to increase the readability inside of this book:

```
root@kind-control-plane:/# kubectl get no -o wide | awk '{printf
NAME                STATUS    ROLES    CONTAINER-RUNTIME
kind-control-plane   Ready    control-plane   containerd://1.7.
kind-worker          Ready    <none>         containerd://1.7.
```

As we can see in the last column of that output, containerd is the container

runtime that's used for our nodes. Containerd is a lightweight daemon for Linux and utilizes cgroups and namespaces to run the containers that are inside of our pods running in Kubernetes.

Finally, we'll take a look at the kubelet. Since kubelet is a systemd service running on the node, and since our nodes are really containers in kind, we'll have to get a shell into the container that is our worker node. First type `exit` to exit from the control plane shell. You can now get a shell to the worker node with the command `docker exec -it kind-worker bash`. Now, get the status of the service by running the command `systemctl status kubelet` like so:

```
$ docker exec -it kind-worker bash
root@kind-worker:/# systemctl status kubelet
• kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/etc/systemd/system/kubelet.service;
   ➤ enabled; preset: enabled)
   Drop-In: /etc/systemd/system/kubelet.service.d
            └─10-kubeadm.conf, 11-kind.conf
   Active: active (running) since Wed 2025-01-01
   ➤ 23:49:37 UTC; 1min 6s ago
     Docs: http://kubernetes.io/docs/
   Process: 219 ExecStartPre=/bin/sh -euc if [
   ➤ -f /sys/fs/cgroup/cgroup.controllers ]; then
   ➤ /kind/bin/create-kubelet-cgroup-v2.sh; fi (
   ➤ code=exited, status=0/SUCCESS)
     Process: 227 ExecStartPre=/bin/sh -euc if [ ! -f
   ➤ /sys/fs/cgroup/cgroup.controllers ] && [ ! -d
   ➤ /sys/fs/cgroup/systemd/kubelet ]; then mkdir -p
   ➤ /sys/fs/cgroup/systemd/kubelet; fi (code=exited,
   ➤ status=0/SUCCESS)
   Main PID: 228 (kubelet)
     Tasks: 15 (limit: 9526)
    Memory: 31.1M
       CPU: 2.282s
    CGroup: /kubelet.slice/kubelet.service
            └─228 /usr/bin/kubelet --bootstrap-
   ➤ kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf -
   ➤ kubeconfig=/etc/kubernetes/kubelet.conf -
   ➤ config=/var/lib/kubelet/config.yaml --container-runtime-
   ➤ endpoint=unix:///run/containerd/containerd.sock
   ➤ --node-ip=172.18.0.3 --node-labels=
   ➤ --pod-infra-container-image=registry.k8s.io/
   ➤ pause:3.10 --provider-id=kind://docker/kind/
   ➤ kind-worker --runtime-cgroups=/system.slice/
```

➡containerd.service

As you can see, the systemd service is active and running, which is a good thing. There may be a question on the exam where you would need to remember how to check if the kubelet service is running. Remember, if the kubelet is not running, your node status will show as “NotReady”. You can even test this by running the command `systemctl stop kubelet`, exiting out of the container shell, and then the command `kubectl get no`. Go ahead and try it. If you don’t have a shell to the docker container, you may need to use `sudo` with this command.

NOTE

it may take a few seconds for the message to get back to the API server that the kubelet service is not running, so if the node doesn’t show “NotReady” right away, don’t worry.

```
root@kind-worker:~# systemctl stop kubelet
root@kind-worker:~# exit
$ kubectl get no
```

NAME	STATUS	ROLES	AGE	VE
kind-control-plane	Ready	control-plane	5h27m	v1
kind-worker	NotReady	<none>	5h27m	v1
kind-worker2	Ready	<none>	5h27m	v1

To start the kubelet service again, simply get a shell inside the worker node again, then run the command `systemctl start kubelet`.

```
$ docker exec -it kind-worker bash
root@kind-worker:/# systemctl start kubelet
root@kind-worker:/#
```

We’ll talk more about troubleshooting services in chapter 8, but I just couldn’t wait to show you this valuable tool for telling if the kubelet service is not started in your Kubernetes cluster. For this next section, we’ll exit out of the “kind-worker” shell with the `exit` command.

Practice Exercises

Increase your efficiency with running `kubectl` commands by shortening

kubectl and creating a shell alias to “k”.

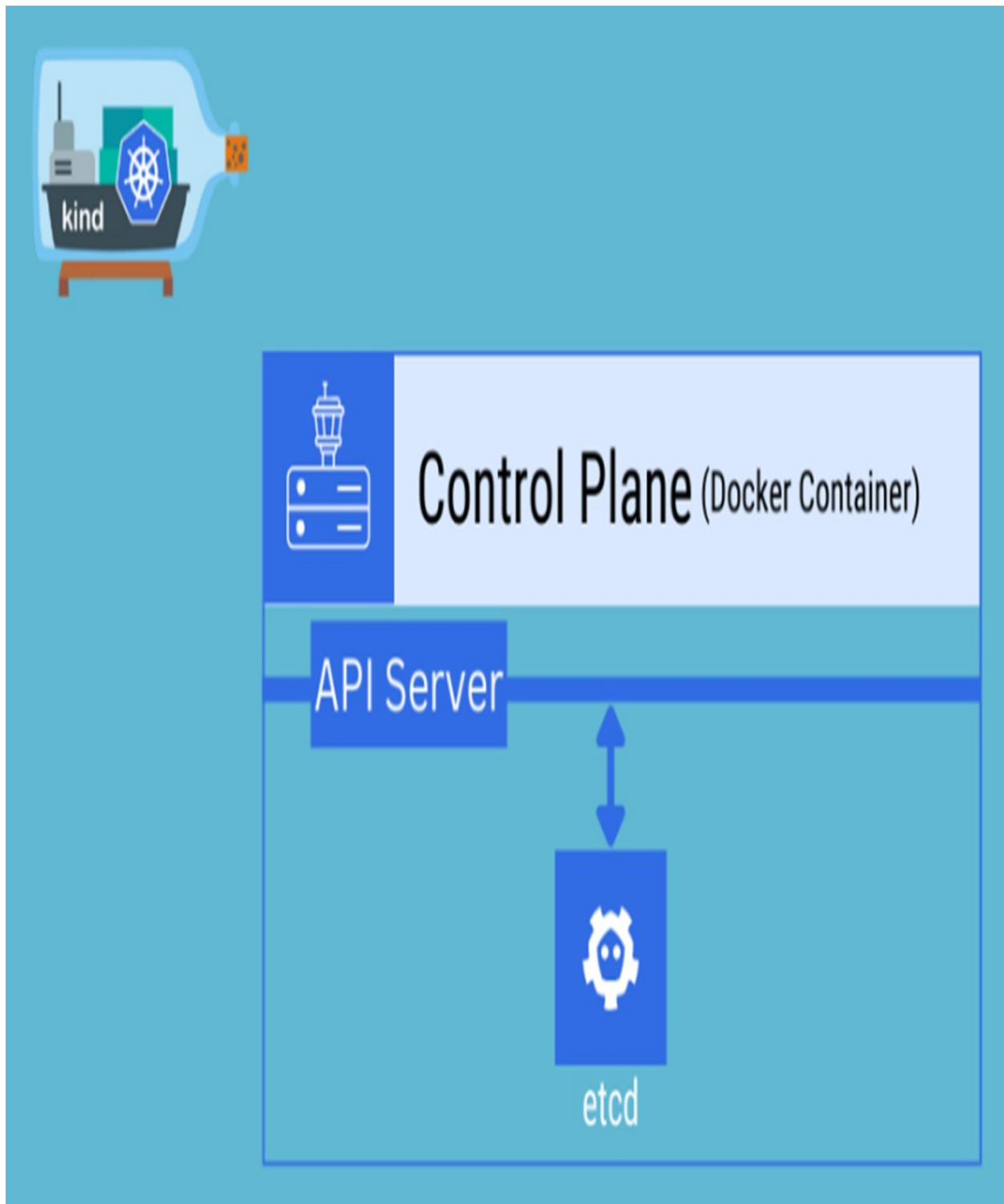
Using the kubectl CLI tool, get the output of the pods running in the kube-system namespace and show the pod IP addresses. Save the output of the command to a file named “pod-ip-output.txt”.

Upgrade the control plane components using kubeadm. When completed, check that everything, including kubelet and kubectl is upgraded to version 1.32.1.

2.2 Cluster state

In Kubernetes, etcd contains all of our cluster configuration data, which means information about how many pods or deployments are running in our cluster (including historical information), as well as which ports are exposed on which services. If you didn’t have this data, you’d be lost as to re-creating all Kubernetes resources from scratch. Just to be clear, we’re not talking about the data associated with your application. We’re just talking about the configuration of the Kubernetes cluster itself. This is depicted in figure 2.13, where the etcd datastore runs inside of a pod in the Kubernetes cluster, containing cluster configuration data.

Figure 2.13 Etcd exists as a pod in the kube-system namespace



Etcd requires its own server certificate to ensure secure communication, authentication, and integrity of data between the Kubernetes control plane and etcd. In the Public Key Infrastructure (PKI), mentioned in the previous chapter, the API server acts as a client that queries etcd, and etcd must verify

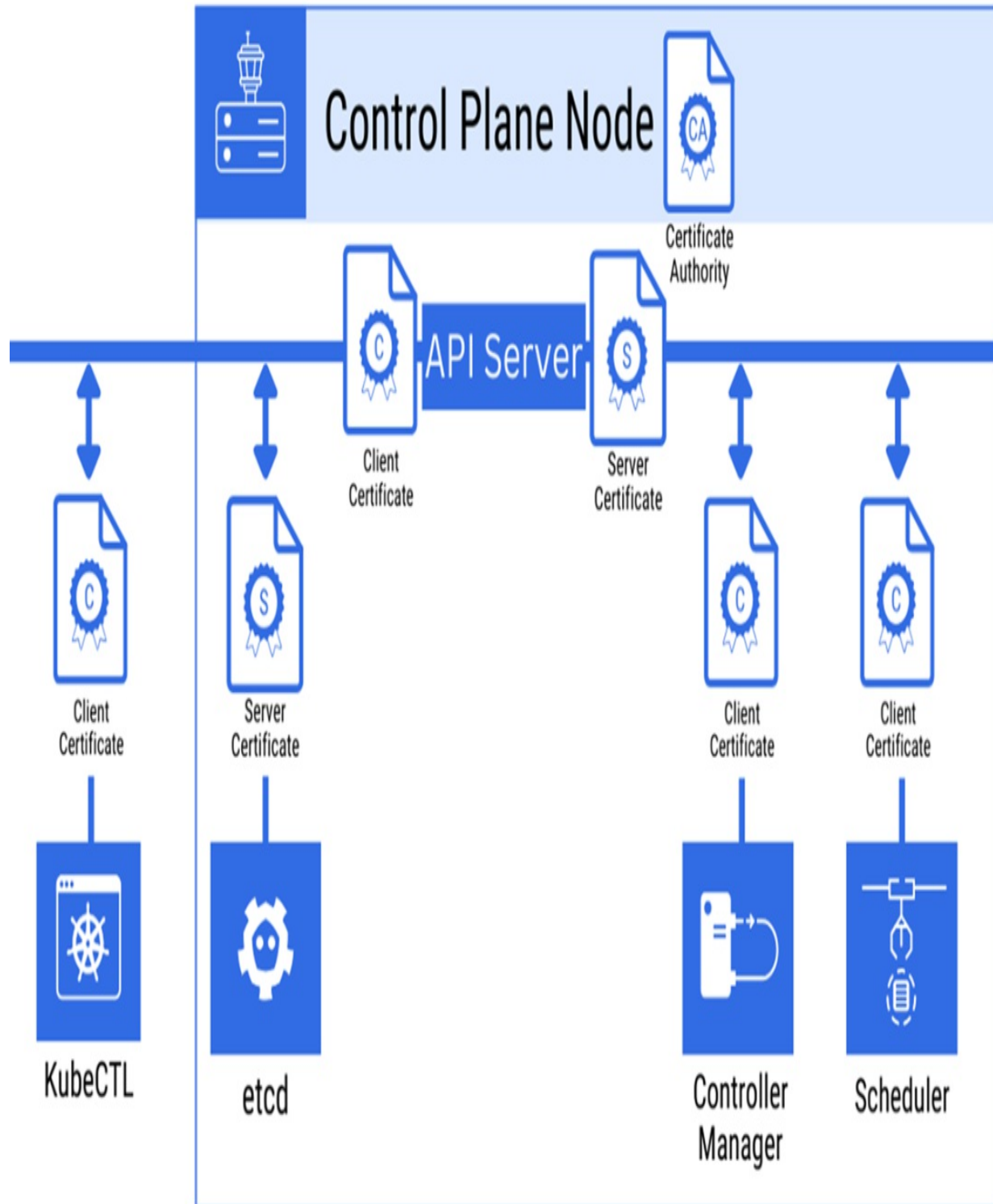
the API server's identity before responding. This secure communication via TLS encryption ensures that the communication is protected from any potential man-in-the-middle attacks, keeping data such as secrets, configmaps, and persistent volume claims protected. On the control plane, in the kube-apiserver.yaml file in the /etc/Kubernetes/manifests directory, you can see the certificate used by the API server to authenticate to the etcd instance. The following is a snippet from the kube-apiserver.yaml file, which contains the etcd-cafile (certificate authority file), etcd-certfile (client certificate file), and the etcd-keyfile (client key file):

```
spec:
  containers:
  - command:
    - kube-apiserver
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
    - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.c
    - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.ke
    - --etcd-servers=https://127.0.0.1:2379
```

2.2.1 Client and server certificates

In the previous section, we talked about etcd needing authentication to the cluster. We mentioned that this method of authenticating between client and server is called PKI (Public Key Infrastructure). In addition to PKI being the client-server model that Kubernetes imposes, it authenticates any requests that come into the API using a certificate file (as with kubectl). You won't have to create or manage these certificates, but it's good to know where these certificates reside on the control plane for the CKA exam. In figure 2.14, you'll see a visual representation of every client or server certification used for every component within a Kubernetes cluster. Let's run the command `ls /etc/kubernetes` to list the contents of that directory and see the client certificates used for kubelet, scheduler and controller manager.

Figure 2.14 Client and Server certificate placement in Kubernetes



NOTE

You'll still need a shell to the kind-control-plane. In case you forgot, here's the command: `docker exec -it kind-control-plane bash`

```
root@kind-control-plane:/# ls /etc/kubernetes
admin.conf  controller-manager.conf  kubelet.conf  manifests  pki
```

You'll see "controller-manager.conf", "kubelet.conf" and "scheduler.conf", which are representative of the kubeconfig used to authenticate to the Kubernetes API. If you view the contents of kubelet.conf, for example, you'll see a similar config to your kubectl kubeconfig (in ~/.kube/config). Type the command `cat /etc/kubernetes/kubelet.conf`

```
root@kind-control-plane:/# cat /etc/kubernetes/kubelet.conf
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tL
    server: https://kind-control-plane:6443
  name: kind
contexts:
- context:
    cluster: kind
    user: system:node:kind-control-plane
  name: system:node:kind-control-plane@kind
current-context: system:node:kind-control-plane@kind
kind: Config
preferences: {}
users:
- name: system:node:kind-control-plane
  user:
    client-certificate: /var/lib/kubelet/pki/kubelet-client-curre
    client-key: /var/lib/kubelet/pki/kubelet-client-current.pem
```

Notice the user is the system user and the client certificate is located on the node in /var/lib/kubelet/pki/kubelet-client-current.pem. This will be the case for all nodes in Kubernetes, as all nodes run the kubelet service.

You may have noticed a "pki" directory in /etc/kubernetes, and if you list the contents of that directory, you'll find more client and server certificates. Let's take a look with the command `ls /etc/kubernetes/pki`

```
root@kind-control-plane:/# ls /etc/kubernetes/pki
apiserver-etcd-client.crt  apiserver-kubelet-client.crt  apiserve
```


`apiserver-etcd-client.key` `apiserver-kubelet-client.key` `apiserver`

In this `pki` directory, you'll see another kubelet certificate (`kubelet.crt`). This one's a server certificate. Yes, you can have a client and server certificate for the same service in Kubernetes. This also is apparent from seeing the two files named "`apiserver.crt`" and "`apiserver-etcd-client.crt`" being a server and client certificate respectively. And since the Kubelet has a server certificate, it must have a client certificate, which is the file named "`apiserver-kubelet-client.crt`". The `apiserver.crt` is the server certificate to the client certificates that we looked at in the `/etc/kubernetes` directory just a moment ago. Finally, there's `etcd`. In the `/etc/kubernetes/pki/etcd` directory, you'll also see a "`server.crt`" file which pertains to the server certificate for `etcd`. The API server is constantly trying to authenticate to the `etcd` datastore and get the cluster config data. This way, not just anyone can view or modify the cluster state.

Practice Exercises

Use the CLI tool that will allow you to view the client certificate that the kubelet uses to authenticate to the Kubernetes API. Output the results to a file named "`kubelet-config.txt`"

2.3 Extensions interfaces

Kubernetes provides extension interfaces that allow administrators to customize and extend its functionality without modifying the core codebase. This means third-party developers can introduce new capabilities, enhancing Kubernetes features. Also, it allows Kubernetes to remain highly modular and customizable, making it easier for organizations to adapt it to their specific needs. For example, a company can solve its multi-cloud networking challenges and build secure, scalable, and high-performance solutions leveraging a key extensions interface of Kubernetes. There are four key interfaces that you must know about for the CKA exam, the Container Runtime Interface (CRI), the Container Network Interface (CNI), the Container Storage Interface (CSI), and Custom Resource Definitions (CRDs). We'll talk more about CRDs in Chapter 5, as they are a part of running new workloads, used alongside operators in Kubernetes. Let's dive

into the other three in-depth, to explore how each of them adds to the features of Kubernetes while maintaining that modularity.

2.3.1 Container runtime interface

The container runtime interface (CRI) defines how the kubelet talks to the container runtime (e.g. starting and stopping containers, pulling images, etc.). Once upon a time, Kubernetes relied solely on Docker for container management. Over time this created an unnecessary dependency on one tool, and thus CRI was born. The idea is to simplify the logic and make it available as a standard that every vendor could use, not just Docker. Essentially, it provides a standardized way for Kubernetes to launch, manage, and stop containers without being tied to a single runtime implementation. To interface with CRI-compatible runtimes, we use the `crictl` command-line tool. For the CKA exam, you will need to know how to use `crictl` because Kubernetes no longer uses Docker as a default container runtime. We should be familiar with using the `crictl` tool for stopping, starting, inspecting, verifying and checking the logs of containers. You can use the `crictl` tool during the exam to view the stopped and started containers with the command `crictl ps -a` (take off the `-a` for only showing the running containers). We can run this in our kind cluster, and get something similar to the following (I'll just output the first column for brevity sake):

```
root@kind-control-plane:/# crictl ps | awk '{print $1}'
CONTAINER
8d54e1f2032c4
61778f0404b76
28249f397ab90
54a063921d5e2
9036538d12795
77e60c9d53636
8b8e63d110a3d
6f34996ef4045
ea0a702ee0847
```

You can inspect the container to get the state of the container with the command `crictl inspect`. This is useful for troubleshooting a pod stuck in a `CrashLoopBackOff` state for example. We will run through some more troubleshooting scenarios in Chapter 8. If a system component (e.g. kube-apiserver, kube-controller-manager, or kube-scheduler) is failing, and

`kubectl logs` is not available, you can check the logs directly with `crictl logs` followed by the container ID (e.g. `crictl logs ea0a702ee0847`). The container ID is the output that we just showed from the previous `crictl ps` command (the first column).

NOTE

`crictl` is used directly on the node in which you are troubleshooting, so you won't see the containers running on other nodes in the cluster.

2.3.2 Container network interface

The container network interface (CNI) is a specification and library for configuring network interfaces in Linux containers. In a Kubernetes context, it defines how pod networking is set up and managed. Much like the CRI standardized the runtime, the CNI lets Kubernetes work with many different network providers without being tied to any single one. Kubernetes needed a flexible way to handle cluster networking across various environments—bare metal, cloud, or hybrid. Different projects (e.g., Calico, Flannel, Weave Net) implement different networking models. The two projects that we're concerned about for the CKA exam are Flannel and Calico. To determine which CNI plugin is running, you can check the status of the networking components by running `kubectl get pods -n kube-system`. The output will look similar to this:

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS
calico-kube-controllers-fdf5f5495-7lzb1	1/1	Running	2 (43)
canal-vmgsc	2/2	Running	2 (43)
canal-xntjt	2/2	Running	2 (43)
coredns-7695687499-27bvx	1/1	Running	1 (43)
coredns-7695687499-wfcnf	1/1	Running	1 (43)
etcd-controlplane	1/1	Running	3 (43)
kube-apiserver-controlplane	1/1	Running	2 (43)
kube-controller-manager-controlplane	1/1	Running	2 (43)
kube-proxy-nhwwx	1/1	Running	1 (43)
kube-proxy-nlw5b	1/1	Running	2 (43)
kube-scheduler-controlplane	1/1	Running	2 (43)

We'll talk more about the CNI in chapter 6, but for the CKA exam, you

should know how to install the CNI, as well as replace, and troubleshoot CNI related issues. The CNI controls how pods communicate with each other as they get scheduled to different nodes, so this is a fairly critical extension to Kubernetes.

Whenever Kubernetes schedules a new pod, the container runtime calls the CNI plugin(s). The plugin configures the network interface, sets up IP addresses, routes, and handles any additional tasks such as implementing network policies. When a Pod is removed, the plugin tears down those resources. You'll see that the CNI in use in our Kubernetes cluster, by default, is "kindnet" by running the command `k -n kube-system get ds kindnet`.

```
root@kind-control-plane:/etc/cni/net.d# k -n kube-system get ds k
NAME          DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NO
kindnet       1         1         1       1             1           ku
```

The configuration for this CNI can be viewed from the `/etc/cni/net.d` directory in a file named `10-kindnet.conflist`.

```
{
  "cniVersion": "0.3.1",
  "name": "kindnet",
  "plugins": [
    {
      "type": "ptp",
      "ipMasq": false,
      "ipam": {
        "type": "host-local",
        "dataDir": "/run/cni-ipam-state",
        "routes": [
          { "dst": "0.0.0.0/0" }
        ],
        "ranges": [
          [ { "subnet": "10.244.0.0/24" }
        ]
      }
    },
    {
      "type": "portmap",
      "capabilities": {
```

```
        "portMappings": true
    }
}
]
```

Under the hood, Kindnet uses the PTP (point-to-point) plugin from the CNI project (along with port mapping) to configure container networking. It's a simple, minimal solution that is sufficient for local development environments. In appendix C, you have instructions for installing flannel CNI. Once you do this, you can investigate the same directory and find out how the CNI is configured with Flannel instead of kindnet, since you will be using Flannel and Calico on the CKA exam.

2.3.3 Container storage interface

The container storage interface (CSI) standardizes how storage providers integrate with container orchestrators like Kubernetes. In earlier Kubernetes releases, storage plugins were “in-tree”—meaning each had to be compiled into Kubernetes itself. This was cumbersome and slowed innovation (new storage systems required changes to the Kubernetes codebase). CSI fixes this by creating a plugin model that's out-of-tree. A CSI driver runs as an external component that handles volume operations (attach, mount, format, snapshot, etc.) for a particular storage system (e.g. Amazon EBS, Ceph, VMware vSphere, etc.). Because the CSI specification is vendor-neutral, storage solutions from different providers (e.g., AWS, GCP, Azure, or on-prem systems) can all plug into Kubernetes the same way.

We'll dive into storage in chapter 7, but for the CKA exam, you may not need to install a CSI driver from scratch, but you must know how to work with StorageClasses, PersistentVolumes, and PersistentVolumeClaims. The command `kubectl get csidrivers` will list the installed CSI drivers, and most likely this will be the driver that you will use for the task at hand. Also, `kubectl get storageclasses` will show you the available storage provisioning mechanisms that you can create your PersistentVolumeClaim from.

Practice Exercises

View the CRI implementation using the `crictl` command-line tool. Determine the container runtime in use.

Use the `crictl` command-line tool to list the running containers. Stop and remove the `kube-apiserver` container by its container ID and see what happens.

Use the `kubectrl` command-line tool to determine what CNI is installed with the cluster. Also determine what version the CNI is currently running.

Use the `kubectrl` command-line tool to list the CSI drivers in your Kubernetes cluster as well as the CSI storageClasses.

2.4 Summary

- The bootstrapping process hides the complexity of creating a Kubernetes cluster, such as the PKI and `etcd` creation, generating `kubeconfigs` and more.
- The `Kubeconfig` is created for us by `kubeadm` in the bootstrap process and is how we authenticate with the API using `kubectl`. This file is located in `~/.kube/config` directory.
- Services are Kubernetes objects, providing load balancing to one or more pods based on the pod labels. Services have their own unique IP address and DNS name, through which pods can be reached.
- The client and server certificates in the directory `/etc/kubernetes/pki` allow the control plane and worker nodes to authenticate to the API. Multiple Kubernetes components can have both client and server certificates.
- `Etcd` is a key value datastore that contains the cluster configuration, including how many objects exist.
- The directory `/etc/kubernetes/manifests` contains YAML files for the API server, controller manager, scheduler and `etcd`. These files are not detected by the scheduler, therefore, whatever is in this directory `kubelet` will pick up and create automatically.
- In a multi-node cluster, the worker node runs `kubelet`, `kube-proxy`, and the container runtime, whereas the control plane also runs the `kubelet`, controller manager, scheduler and `etcd`. To prevent application pods

from running on the control plane, a taint applied during the bootstrap process.

- In order to standardize on pluggable extensions in Kubernetes, the CSI, CNI, and CSI were created.
- The CSI in Kubernetes enables seamless storage integration with third-party storage solutions. This way, no matter the storage backend, it works the same in Kubernetes and doesn't change the core functionality of Kubernetes itself.
- The main benefit of the CNI is a standard framework for configuring network connectivity between containers, enabling flexible, pluggable networking solutions without modifying Kubernetes core.
- The main advantage of CRI in Kubernetes is that it allows seamless integration of different container runtimes, enabling flexibility and standardization without modifying Kubernetes as a whole.

3 Identity and access management

This chapter covers

- The basics of RBAC
- Creating roles and role bindings
- Using certificates to create users and groups
- Creating service accounts with roles
- Mounting service accounts to pods

In this chapter, we'll focus on role-based access control, which is within the Cluster Architecture, Installation & Configuration section of the exam curriculum. Now that you know how the Kubernetes API works, it's essential to understand how to authenticate and authorize a user and/or a service account for the exam.

The cluster architecture, installation & configuration domain

This chapter covers part of the cluster architecture, installation and configuration domain of the CKA curriculum. This domain includes what consists of a Kubernetes cluster, and the ways in which we configure different aspects of the cluster. It encompasses the following competencies.

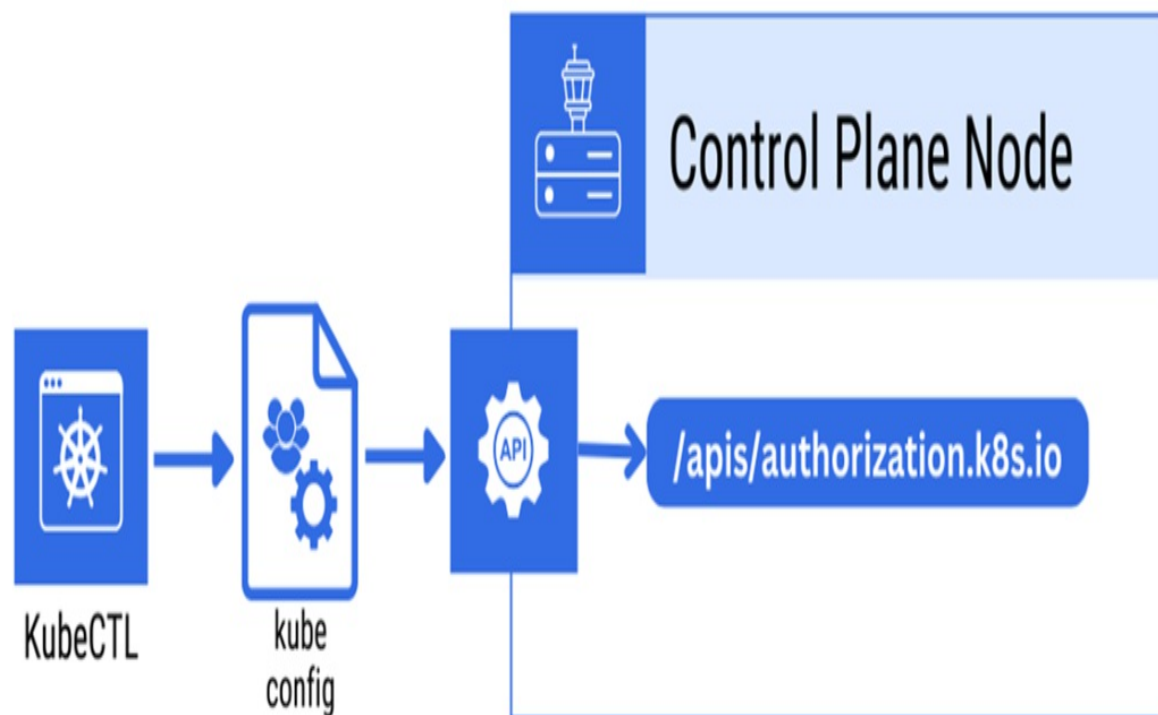
Competency	Chapter Section
Manage role-based access control (RBAC)	3.1, 3.2
Implement and configure a highly available control plane	3.3

3.1 Role-based Access Control (RBAC)

In order to access the resources within a Kubernetes cluster, you must first be authenticated. If you remember from chapter 2, we used the `kubectl` tool to interface with the Kubernetes API. We were allowed to do this because we

had a client certificate which carried a token to authenticate with, depicted in figure 3.1. This is called *role-based access control* because based on our role in Kubernetes, we were able to list the running pods in the kube-system namespace. There are different ways to control these actions and block them if we wish. The authentication and authorization system built into Kubernetes allows us the ability to control a given user or machine's access to a Kubernetes cluster. We'll talk more about this in the following sections.

Figure 3.1 Using a certificate and token to access the Kubernetes API



Kubernetes supports multiple authentication methods. The built-in plugin that we use with kubectl is called *certificate-based authentication* using an X.509 certificate. This is a form of authentication where the API server verifies the identity of a user based on a signed client certificate, stored in a kubeconfig file. The certificate is signed by Kubernetes' built-in Certificate Authority (CA) and presented during API requests to prove identity. There are other authentication methods that are sources for authentication, including bearer tokens, and service accounts. For the CKA Exam, it's essential to understand X.509 certificate authentication (for user authentication), and service account

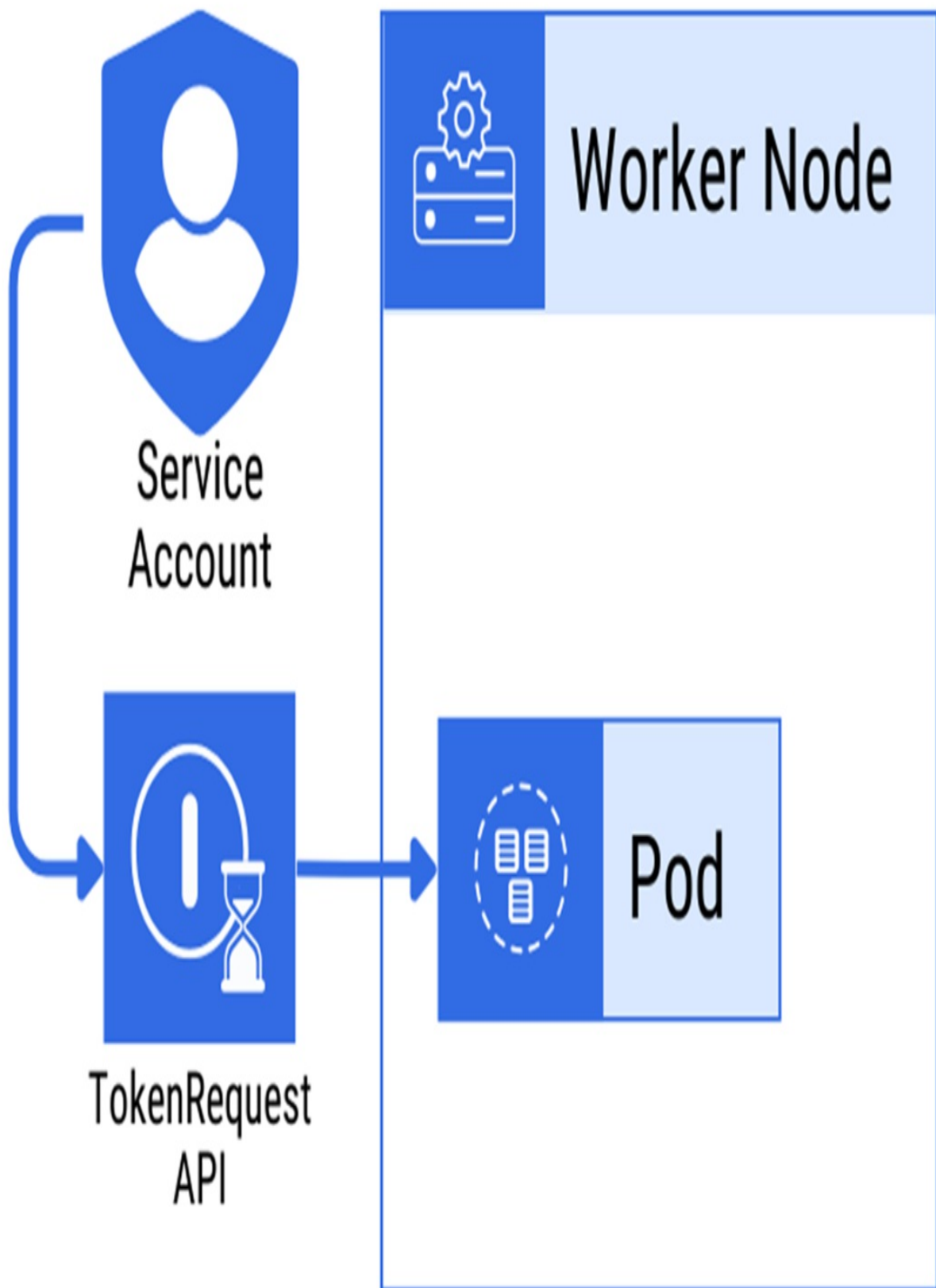
tokens (for Pod authentication with the API).

When you make a request to the Kubernetes API, the API server checks the configured authentication method(s) and validates the request using the first matching mechanism. The first method able to translate the meaning of the request handles the authentication. This involves determining if the request is coming from a human or a machine. If the request is coming from a human - like when we are using `kubectl` - then the user is authenticated as a specific username, and that name is used for subsequent steps. If the request originates from a machine, it must have a service account, with a token to provide the Kubernetes API for authentication. Service accounts are Kubernetes resources, whereas users are not. This allows other identity and access management (IAM) providers to easily integrate with Kubernetes via SSO through LDAP (Lightweight Directory Access Protocol) or AD (Active Directory) for Authentication. A service account is a Kubernetes resource, therefore, Kubernetes manages this along with other resources like pods, deployments and persistent volumes. There is a default service account created automatically in every namespace with the name “default”. You can view this default service account with the command `kubectl get sa`.

```
$ kubectl get sa
NAME          SECRETS    AGE
default       1          3d9h
```

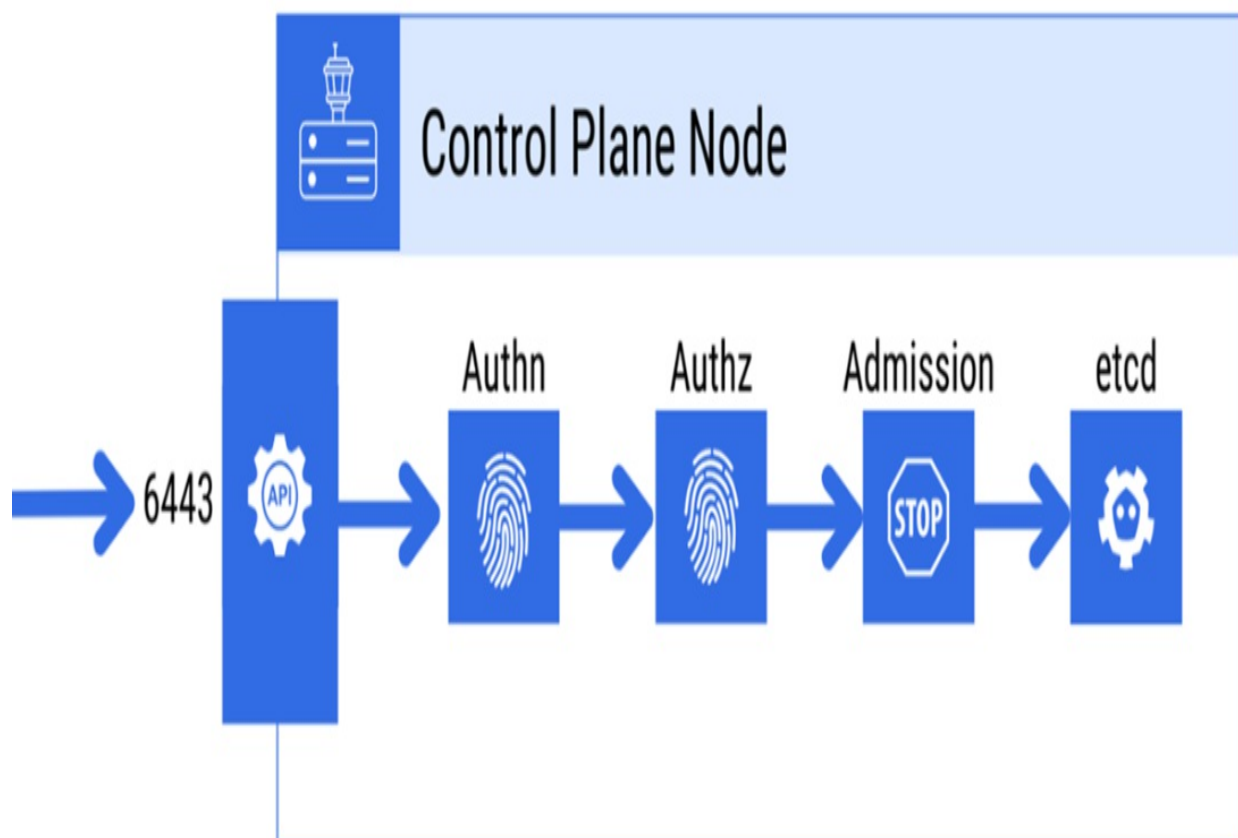
Service accounts are often associated with pods running in the cluster, which allow applications running inside the pod (as containers) to access the Kubernetes API and make requests. A token is mounted to each pod via service account, as depicted in figure 3.3. You can disable this feature, as most of the time you don't need your pods to talk to the API. Later in this chapter we will go through an example of mounting a service account in a pod and preventing service accounts from mounting to a pod..

Figure 3.2 The TokenRequest API can generate a short-lived token to authenticate to the Kubernetes API. The token expires when the pod is terminated.



After the authentication phase, the request is passed on to the authorization mechanism. RBAC is considered one of the authentication mechanisms, but there are others like the Node authorization plugin and the WebHook authorization plugin. For the purposes of the CKA exam, we are going to focus solely on RBAC, which you see depicted in figure 3.4. RBAC prevents unauthorized users from viewing or modifying the cluster state through the principle of least privilege. The principle of least privilege states that administrators can allow explicit access to a single subject in Kubernetes but provide no additional privileges to any other subjects (e.g. permissions to view pods, but not delete them).

Figure 3.3 The RBAC mechanism uses roles and role bindings to prevent unauthorized requests from accessing the Kubernetes API and persisting in etcd



The authorization mode in our kind cluster was set to RBAC as well as Node authorization on the Kubernetes API server during the bootstrap process. We can view the authorization modes inside of the `/etc/kubernetes/manifests`

directory in our kind-control-plane container. You will see it under spec, in the list of container commands as --authorization-mode=Node,RBAC. Let's get the contents of that file and pipe it to "more" with the command `cat /etc/kubernetes/manifests/kube-apiserver.yaml | more`

```
root@kind-control-plane:/# cat /etc/kubernetes/manifests/kube-apiserver.yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubeadm.kubernetes.io/kube-apiserver.advertise-address.endpoi
  creationTimestamp: null
  labels:
    component: kube-apiserver
    tier: control-plane
  name: kube-apiserver
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-apiserver
    - --advertise-address=172.18.0.4
    - --allow-privileged=true
    - --authorization-mode=Node,RBAC
    - --client-ca-file=/etc/kubernetes/pki/ca.crt
    - --enable-admission-plugins=NodeRestriction
    - --enable-bootstrap-token-auth=true
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
    - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.c
    - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.ke
    - --etcd-servers=https://127.0.0.1:2379
    - --insecure-port=0
    - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-
    - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-
    - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hos
    - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-cl
    - --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-cli
    - --requestheader-allowed-names=front-proxy-client
    - --requestheader-client-ca-file=/etc/kubernetes/pki/front-pr
    - --requestheader-extra-headers-prefix=X-Remote-Extra-
    - --requestheader-group-headers=X-Remote-Group
    - --requestheader-username-headers=X-Remote-User
    - --runtime-config=
  --More--
```

We could have opened this file in a text editor just as easily, but we used the

cat command instead. This YAML manifest is important to remember, as it contains the cluster configuration for many components in the Kubernetes cluster, such as the advertise address, privileged modes, and all the certificates and keys for etcd and kubelet.

EXAM TIP

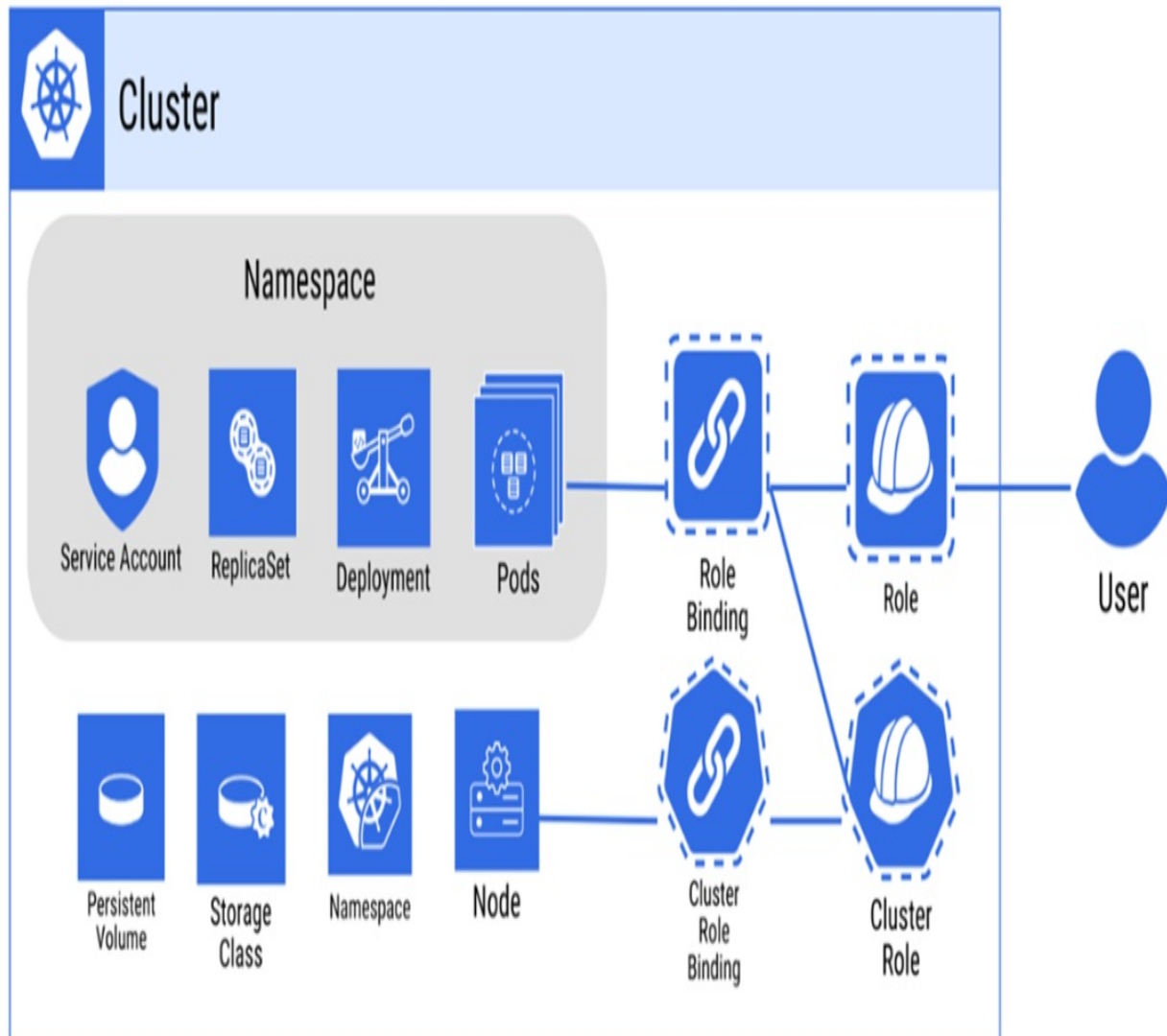
Remember the manifests for control plane components are in the `/etc/kubernetes/manifests` directory. This directory will be located on the control plane server, which you will have to SSH into in order to view or modify. On the exam, the SSH keys are shared, therefore, you can simply type `ssh control-plane-node` to get a shell to the control plane node.

When the RBAC authorization mechanism receives the request, it determines what action is allowed to be performed by the user or group based on its role and roleBinding (or clusterRole & clusterRoleBinding). Yes, groups are factored into RBAC as well, which simply means a group of users that are all assigned the same level of privilege in Kubernetes. We'll go through creating users and groups later in this chapter.

3.1.1 Roles and role bindings

So, how does RBAC know what actions are permitted? This is determined by four different types of objects in Kubernetes, depicted in figure 3.5: the role, the cluster role, the role binding and the cluster role binding. Earlier we talked about Kubernetes resources that are namespace scoped. Permissions for resources that are inside a namespace need to be scoped to that namespace as well. Same goes for resources that are not namespace-scoped such as persistent volumes, nodes or namespaces themselves. These resources need a properly defined clusterRole (cluster-wide) or Roles (namespaced).

Figure 3.4 Roles and role bindings define privileges in a namespace, where cluster roles and cluster role bindings are defining cluster wide privileges



NOTE

It helps to remember which type of user will be accessing namespaced resources vs. cluster resources. Applications developers will commonly be assigned roles because they develop applications that exist inside a namespace. Cluster administrators will commonly be assigned cluster roles because they are updating nodes, creating volumes and other operations outside of the namespace (cluster-wide).

Roles are not only scoped to a namespace, but also to a particular resource in Kubernetes (i.e. pods), and will list the specific actions allowed within that role. The actions are referred to as “request verbs”. They are like API

requests, but since a resource can have multiple endpoints, a request verb targets a specific endpoint and uses the lower-cased HTTP method of the request as the verb. So, “get”, “list”, “create”, “update”, “patch”, “watch”, “delete”, and “deletecollection” are all API request verbs, which allow you to perform an action on a resource, as depicted in figure 3.5.

Figure 3.5 Request verbs correspond roughly with HTTP verbs in the following way

HTTP verb	request verb
POST	create
GET	get, list, watch
PUT	update
PATCH	patch
DELETE	delete

get a resource

list a collection of resources

watch for streaming updates to a resource

Let’s look at an example by typing the command `kubectl create role pod-reader --verb=get,list,watch --resource=pods` and you’ll see output similar to figure 3.6.

Figure 3.6 With this imperative command, create a role and assign request verbs for a resource

This role allows the request verbs "get",
"list", and "watch" on pods



```
root@kind-control-plane:/# kubectl create role pod-reader --verb=get,list,watch --resource=pods
role.rbac.authorization.k8s.io/pod-reader created
```

Now, let's take a look at the YAML output from that role that we just created with the command `kubectl get role pod-reader -o yaml` and we'll see the output similar to figure 3.7.

Figure 3.7 The `kubectl get role` command, we can see which resources this role applies to and which actions the role allows

```
root@kind-control-plane:/# kubectl get role pod-reader -o yaml
```

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: Role
```

```
metadata:
```

```
  creationTimestamp: "2022-04-07T14:40:03Z"
```

```
  name: pod-reader
```

```
  namespace: default
```

```
  resourceVersion: "1783"
```

```
  uid: 54afc1e9-b4ac-4ed3-9e21-c3e20e19ce9c
```

```
rules:
```

```
- apiGroups:
```

```
  - ""
```

```
  resources:
```

```
    - pods
```

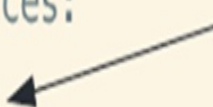
```
  verbs:
```

```
    - get
```

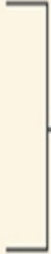
```
    - list
```

```
    - watch
```

**The resource that
this role applies to**



The request verbs



Instead of creating the role right away, we could have done a “dry run” and saved the contents to a YAML file. This comes in handy if you like to take a more declarative approach and keep your YAML files in a version control system. Let’s run the same command to create a role, but this time we’ll perform a dry run and save the output of the command to a file named “role.yaml.” Run the command `kubectl create role pod-reader --verb=get,list,watch --resource=pods --dry-run=client -o yaml > role.yaml` and we’ll see the file has been created, as in figure 3.8.

Figure 3.8 Instead of creating a role, we perform a dry run and then save the contents of the YAML to a file

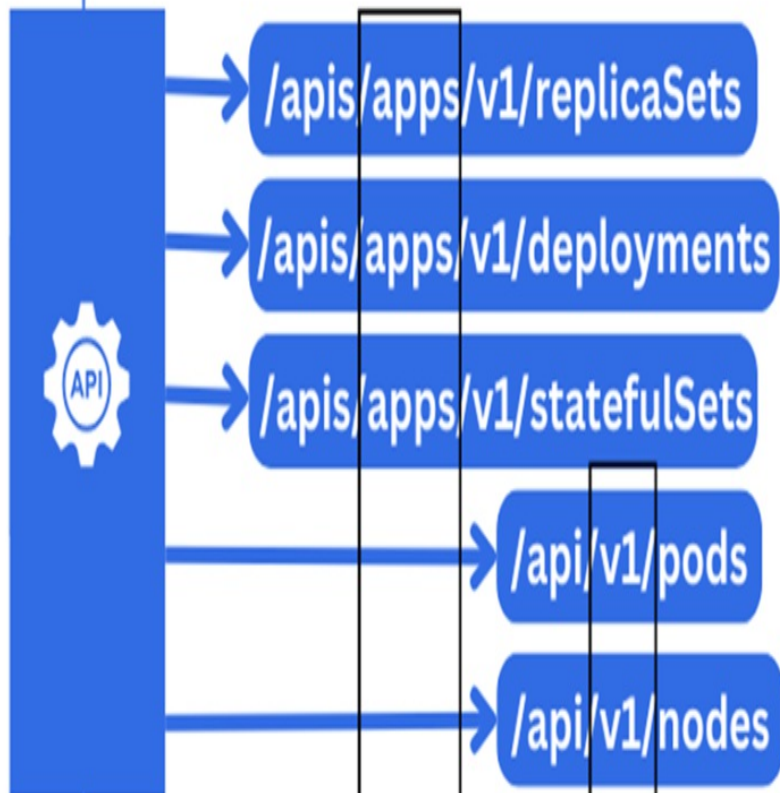


Remember, from the first chapter, how we looked at all the different Kubernetes API resources? Each resource had a path and the end of that path led you to the object. In the case of pods, the path is `/api/v1/pods`, so the API group is within “`/api/v1/`”. Deployments are in the API group `/apis/apps/v1`, which would be listed separately in this role definition, depicted here in figure 3.9.

Figure 3.9 The API group is used for specifying the resource in roles and cluster roles



Control Plane Node



**Apps API
Group**

**Core API
Group**

Notice that this role doesn't indicate permissions for a user at all. That's where role bindings come in. *Role bindings* will attach (or bind) a role to a user, group or service account and will provide that user, group or service account with a role. Let's create a role binding now with the command `kubectl create rolebinding pod-reader-binding --role=pod-reader -user=carol`

Let's take a look at the YAML output of our pod-reader-binding and notice the role reference ("roleRef") and the user ("subjects"). This role binding is how the user "carol" will be authorized to access pods in our Kubernetes cluster.

```
$ kubectl create rolebinding pod-reader-binding --role=pod-reader
rolebinding.rbac.authorization.k8s.io/admin-binding created
$ kubectl get rolebinding pod-reader-binding -oyaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  creationTimestamp: "2022-03-29T04:22:35Z"
  name: pod-reader-binding
  namespace: default
  resourceVersion: "566785"
  uid: 9f7d1036-9123-4288-94cf-ef527aa090d0
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-reader
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: carol
```

There's a really useful tool built-in to kubectl to verify if users can perform certain actions in the Kubernetes cluster without explicitly assuming that users identity. This command is `kubectl auth can-i`. If you get the help pages for this command, you'll see a plethora of examples you can use to test authorization as a user, group or service account.

```
$ kubectl auth can-i -h
Check whether an action is allowed.
```

VERB is a logical Kubernetes API verb like 'get', 'list', 'watch'. Shortcuts and groups will be resolved. NONRESOURCEURL is a partial

particular Kubernetes resource.

Examples:

```
# Check to see if I can create pods in any namespace
kubectl auth can-i create pods --all-namespaces

# Check to see if I can list deployments in my current namespace
kubectl auth can-i list deployments.apps

# Check to see if I can do everything in my current namespace (
kubectl auth can-i '*' '*'

# Check to see if I can get the job named "bar" in namespace "f
kubectl auth can-i list jobs.batch/bar -n foo

# Check to see if I can read pod logs
kubectl auth can-i get pods --subresource=log

# Check to see if I can access the URL /logs/
kubectl auth can-i get /logs/

# List all allowed actions in namespace "foo"
kubectl auth can-i --list --namespace=foo
```

Let's check if the user "carol" can list or delete pods in the default namespace in our Kubernetes cluster using the command `kubectl auth can-i`. Notice the output just says "yes" or "no". What an easy way to verify authorization in Kubernetes!

```
$ kubectl auth can-i get pods -n default --as carol
yes
$ kubectl auth can-i delete pods -n default --as carol
no
```

EXAM TIP

I encourage you to use `kubectl auth can-i` on the exam, as it will allow you verify that you've completed the task successfully and save on precious time

The way in which you create a cluster role and cluster role binding are exactly the same, so we won't go over this in detail. The `kubectl` commands for creating cluster roles and cluster role bindings are `kubectl create clusterrole` and `kubectl create clusterrolebinding` respectively.

EXAM TIP

Use the `-h` flag to get a list of examples for running your command (e.g. `kubectl create clusterrole -h`). This will save you time on the exam, as you can copy the examples without having to type out or remember the entire command.

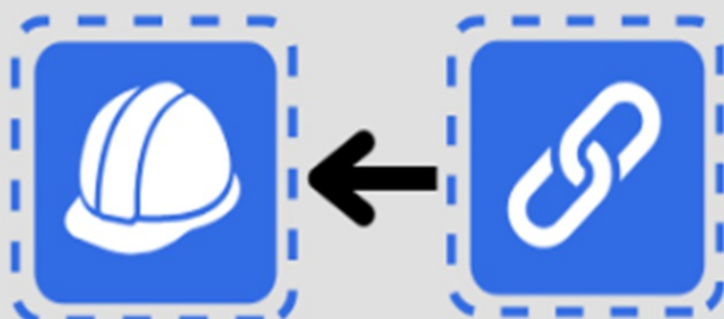
You can also use cluster roles with role bindings, when you require a resource that is not scoped to a namespace (e.g. nodes) to be accessible from a user, group or service account that is applied from within a namespace. You *cannot* attach a cluster role binding to a role because permissions from within a namespace (role permissions) can't be applied to a subject that is not namespace-scoped, as we see in figure 3.10. One thing you have to keep in mind is that permissions via cluster roles will affect all namespaces, current and future. This is why you must follow the principal of least privilege, and be cautious not to assign a role or cluster role that gives unfettered access to your Kubernetes cluster.

Figure 3.10 With the exception of cluster role bindings to roles, all other combinations are supported



Cluster

Namespace



Role

Role
Binding



Cluster
Role
Binding



Cluster
Role

3.1.2 System roles and groups

Now that we know what users, service accounts, roles, and bindings are used for, let's take a look at the built-in roles and groups, and what user and group we've been using for our own cluster thus far.

The API server creates a set of default cluster roles and cluster role bindings. They are directly managed by the control plane. You will see they are "cluster-admin", "admin", "edit" and "view" when you run the command `kubectl get clusterrole | grep -v system`

```
root@kind-control-plane:/# kubectl get clusterrole | grep -v syst
NAME                                CREATED AT
admin                              2022-03-25T17:44:03Z
cluster-admin                      2022-03-25T17:44:03Z
edit                              2022-03-25T17:44:03Z
kindnet                           2022-03-25T17:44:08Z
kubeadm:get-nodes                 2022-03-25T17:44:05Z
local-path-provisioner-role       2022-03-25T17:44:09Z
view                             2022-03-25T17:44:09Z
```

The cluster-admin cluster role is intended to be used cluster-wide, and allows read and write access for most objects including roles and role bindings, with the exception of resource quotas, endpoints, or the namespace itself. The edit cluster role will not allow you to modify roles or role bindings; however, it will allow you to access secrets and run pods with any service accounts inside the namespace. The view cluster role is just like it sounds, allowing you to view most namespaced objects except for secrets since the contents of a secret contains the credentials for API access.

EXAM TIP

Kubernetes allows assigning (binding) a cluster role to multiple users/groups/service accounts via role or cluster role binding. So, on the exam, it may save you time to create a role binding and assign it to an existing role (i.e. view)

Now that we know all the roles that are created by default, which one do we assume as the user of kubectl? For that, we'll have to talk about groups in a

little bit more detail. More specifically, the built-in groups that are managed by the Kubernetes API server. Like users, groups are not a managed resource in Kubernetes, so the group name is just an arbitrary name. However, these built-in groups are different. They are managed by the API server and cannot be modified. When the API server starts up, it reconciles with any missing permissions, and updates the default cluster role bindings with any missing subjects (i.e. groups). This allows the cluster to automatically repair any misconfigurations and keep the roles and bindings up to date with new releases of Kubernetes. These built-in default cluster role bindings are “system:authenticated”, “system:unauthenticated”, and “system:masters”.

- The **system:authenticated** group assigns privileges to users who are successfully authenticated.
- The **system:unauthenticated** group is used when none of the authentication plugins could validate the request.
- The **system:masters** group is used for super users and provides unfettered access to everything in Kubernetes.

The system:masters group is the group we have been assuming this entire time. Yikes! We better tread carefully, as to not misconfigure anything and deem our cluster incapable of running. All joking aside, you should never use this group for normal users, because if it were to fall into the wrong hands, it could be dangerous. Just to prove that we have unrestricted access to our cluster, we can use the command `kubectl auth can-i --list` to list the actions we can take on each and every object in the cluster.

```
root@kind-control-plane:/# kubectl auth can-i --list
Resources                                     Non-Resource URLs
* . *                                         []
[*]
selfsubjectaccessreviews.authorization.k8s.io []
selfsubjectrulesreviews.authorization.k8s.io  []
[/api/*]
[/api]
[/apis/*]
[/apis]
[/healthz]
[/healthz]
[/livez]
[/livez]
[/openapi/*]
```

```
[/openapi]
[/readyz]
[/readyz]
[/version/]
[/version/]
[/version]
[/version]
```

Furthermore, we can run the command `kubectl get clusterrolebinding cluster-admin -o yaml` to see the clusterrolebinding which binds the role “cluster-admin” to our user, who is a part of the system:masters group.

```
kubectl get clusterrolebinding cluster-admin -o yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  annotations:
    rbac.authorization.kubernetes.io/autoupdate: "true"
  creationTimestamp: "2022-03-25T17:44:03Z"
  labels:
    kubernetes.io/bootstrapping: rbac-defaults
  name: cluster-admin
  resourceVersion: "141"
  uid: d91f5e22-e179-47b3-b90e-2591f4617b3b
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: system:masters
```

Finally, we can see the role, which also shows the actions we can take on cluster resources. You’ll notice that “*” is a wildcard for everything. So, we have access to all resources in all API groups, and allowed to take any action (get, list, create, delete, update, patch, watch).

EXAM TIP

If you are ever stuck on how a cluster role is formatted, you can send the output of `kubectl` to a file and edit it inline. To do this, use the command `kubectl create clusterrole pod-reader --verb get,list,watch --resource pods --dry-run=client -o yaml > pod-reader.yaml`

Practice exercises

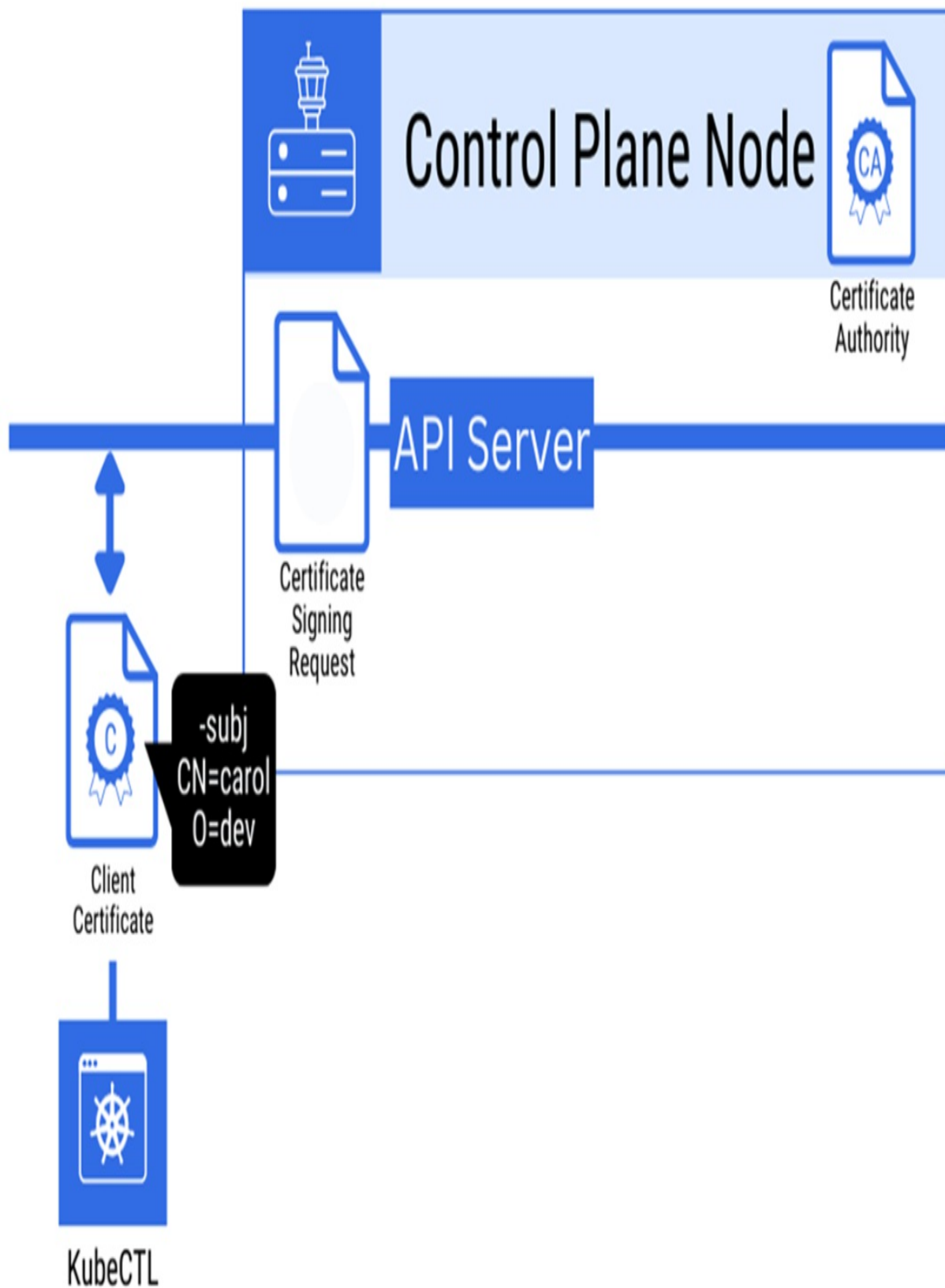
Create a new role named “sa-creator” that will allow creating service accounts.

Create a role binding that is associated with the previous “sa-creator” role, named “sa-creator-binding” that will bind to the user “Sandra”.

3.2 Users and groups

We’ve uncovered the greatest fear of every security engineer in the world: a back door to our Kubernetes cluster that allows full unrestricted access to do anything! We could even expose our cluster publicly to anonymous users if we wanted to, inviting anyone and everyone to get into our cluster and do as they please. No, don’t do that. Let’s let those security folks rest easy at night and create ourselves a new user that follows the principal of least privilege. Remember carol? We created a role binding that associates the “pod-reader” role to the “carol” user. Let’s actually create the kubeconfig that carol can use to access the cluster, as depicted in figure 3.12.

Figure 3.11 Creating a kubeconfig for a new user and generating a certificate using the certificate signing request



Since users and groups are arbitrary notations on a certificate and not managed by Kubernetes, let's start by creating a new certificate signing request. Remember we learned that the Kubernetes API is its own certificate authority? This means that it can sign certificates for us, rendering them valid authentication mechanisms for the Kubernetes API. How convenient!

First, let's generate a private key using 2048-bit encryption with the command `openssl genrsa -out carol.key 2048`

```
root@kind-control-plane:/# openssl genrsa -out carol.key 2048
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
```

Now let's create a certificate signing request file, using the private key we just created, that we'll eventually give to the Kubernetes API. It's important here that we specify our user and group in the common name of the certificate signing request with the command `openssl req -new -key carol.key -subj "/CN=carol/O=developers" -out carol.csr`

```
root@kind-control-plane:/# openssl req -new -key carol.key -subj
root@kind-control-plane:/# ls | grep carol
carol.csr
carol.key
```

Let's store the csr file in an environment variable, as we'll need it for later. To do this, use the command `export REQUEST=$(cat carol.csr | base64 -w 0)` to store the base64 encoded version of the CSR file in an environment variable named "REQUEST".

Now we have exactly what we need in order to create our certificate signing request resource in Kubernetes. We go about creating this resource just like we would with so many other Kubernetes resources and objects (e.g. deployments, pods, service accounts, etc.), via YAML file.

EXAMP TIP

Since you can have an additional tab open during the exam, utilize the search

function in Kubernetes docs (<https://kubernetes.io/docs>). This is allowed and encouraged. Before you click on the search result, hover over the link and ensure it's staying within the `kubernetes.io/docs` subdomain.

Let's go ahead and use the Kubernetes documentation to find a copy of the YAML structure we need for a certificate signing request resource. You'll need to become familiar with the Kubernetes documentation for the exam, as it is your lifeline and great time saver for copying examples (i.e. YAML) and pasting into your terminal. Let's go to <https://kubernetes.io/docs/reference/access-authn-authz/certificate-signing-requests/#create-certificatesigningrequest> and practice. Go ahead and copy the YAML for the certificate signing request and paste it into your terminal, changing three things about it, and that's the big long string to the right of the word "request". Let's replace this with our "REQUEST" environment variable. Also change the name and the groups to match the common name in the CSR file. It'll look like this:

```
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: carol
spec:
  groups:
  - developers
  request: $REQUEST
  signerName: kubernetes.io/kube-apiserver-client
  usages:
  - client auth
EOF
```

If you don't want to bother with copying and pasting, or maybe you don't have access to a browser right now, that's ok. Go ahead and run this command which will achieve the same result: `kubectl apply -f https://raw.githubusercontent.com/chadmcrowell/k8s/main/manifests/carol.yaml`

Congratulations, you've successfully submitted a signing request to the Kubernetes API! Since this is an object and Kubernetes manages, we can run the command `kubectl get csr` to see a list of requests. You'll notice that the one that we just submitted is named `carol` and it's condition is "Pending".

```
root@kind-control-plane:/# kubectl get csr
NAME      AGE      SIGNERNAME                                    REQUESTOR
carol      4s       kubernetes.io/kube-apiserver-client          kubernetes-ad
```

We can simply approve the request with the command `kubectl certificate approve carol` and we'll see the condition change from "pending" to "Approved,Issued"

```
root@kind-control-plane:/# kubectl certificate approve carol
certificatesigningrequest.certificates.k8s.io/carol approved
root@kind-control-plane:/# kubectl get csr
NAME      AGE      SIGNERNAME                                    REQUESTOR
carol      2m12s    kubernetes.io/kube-apiserver-client          kubernetes-
```

Now that it's been approved, we can extract the contents of the signed certificate, namely the client certificate, base64 decode it and store it in a file named "carol.crt" with the command `kubectl get csr carol -o jsonpath='{.status.certificate}' | base64 -d > carol.crt`

```
root@kind-control-plane:/# kubectl get csr carol -o jsonpath='{.s
root@kind-control-plane:/# ls | grep carol
carol.crt
carol.csr
carol.key
```

Now that we have the key (carol.key) and client certificate (carol.crt), we can add these credentials to our kubeconfig. We can simultaneously embed the certificate and key values into the file (similar to our current kubeconfig) with the command `kubectl config set-credentials carol --client-key=carol.key --client-certificate=carol.crt --embed-certs`

NOTE

If we didn't want to embed the certificate and key, we could leave off the `--embed-certs` option. This would list the path to the two files instead of the raw certificate values.

You will now see in the credentials for carol in the kubeconfig by using the command `kubectl config view`

```
root@kind-control-plane:/# kubectl config view
apiVersion: v1
```



```
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://kind-control-plane:6443
  name: kind
contexts:
- context:
    cluster: kind
    user: kubernetes-admin
  name: kubernetes-admin@kind
current-context: kubernetes-admin@kind
kind: Config
preferences: {}
users:
- name: carol
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
- name: kubernetes-admin
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
```

Now, in order to add the user carol to our context, we'll run the command `kubectl config set-context carol --user=carol --cluster=kind`. Once you run this command, you'll notice that the context has been added by running the command `kubectl config get-contexts`

```
root@kind-control-plane:/# kubectl config set-context carol --use
Context "carol" created.
root@kind-control-plane:/# kubectl config get-contexts
CURRENT   NAME                               CLUSTER   AUTHINFO   NA
*          carol                             kind      carol      NA
          kubernetes-admin@kind            kind      kubernetes-admin
```

Now, if you notice the star in the “current” column on the left indicates which context we are currently using. Before moving forward, let's run the command `kubectl run nginx --image nginx` to create a pod, so we can properly test if the user “carol” has the appropriate permissions according to our role named “pod-reader”. Then, to switch contexts and use the “carol” context, run the command `kubectl config use-context carol`. You'll notice the asterisk changed the current context to “carol”.

Finally, just like with the `kubectl auth can-i` command, we can verify that

carol can list pods, but cannot delete pods.

```
root@kind-control-plane:/# kubectl config use-context carol
Switched to context "carol".
root@kind-control-plane:/# kubectl get po
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           13s
root@kind-control-plane:/# kubectl delete po nginx
Error from server (Forbidden): pods "nginx" is forbidden: User "c
```

Let's take it one step further and bind the role (using a role binding) to a group versus a user (which is what we have it as right now). To switch back to the admin user, type the command `kubectl config use-context kubernetes-admin@kind`. Then, delete the current role binding with the command `kubectl delete rolebinding admin-binding` and let's create a new role binding with the command `kubectl create rolebinding pod-reader-bind --role=pod-reader --group=developers`. Let's switch back to the user carol with the command `kubectl config use-context carol` and test our authorization again (this time via groups) with the command `kubectl get po`.

```
root@kind-control-plane:/# kubectl config use-context kubernetes-
Switched to context "kubernetes-admin@kind".
root@kind-control-plane:/# kubectl delete rolebinding admin-bindi
rolebinding.rbac.authorization.k8s.io "admin-binding" deleted
root@kind-control-plane:/# kubectl create rolebinding pod-reader-
rolebinding.rbac.authorization.k8s.io/pod-reader-bind created
root@kind-control-plane:/# kubectl config use-context carol
Switched to context "carol".
root@kind-control-plane:/# kubectl get po
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           8m9s
```

Now that we are done testing with the “carol” user, let's switch back to the admin user again and continue on to the next topic in this chapter: service accounts.

Practice exercises

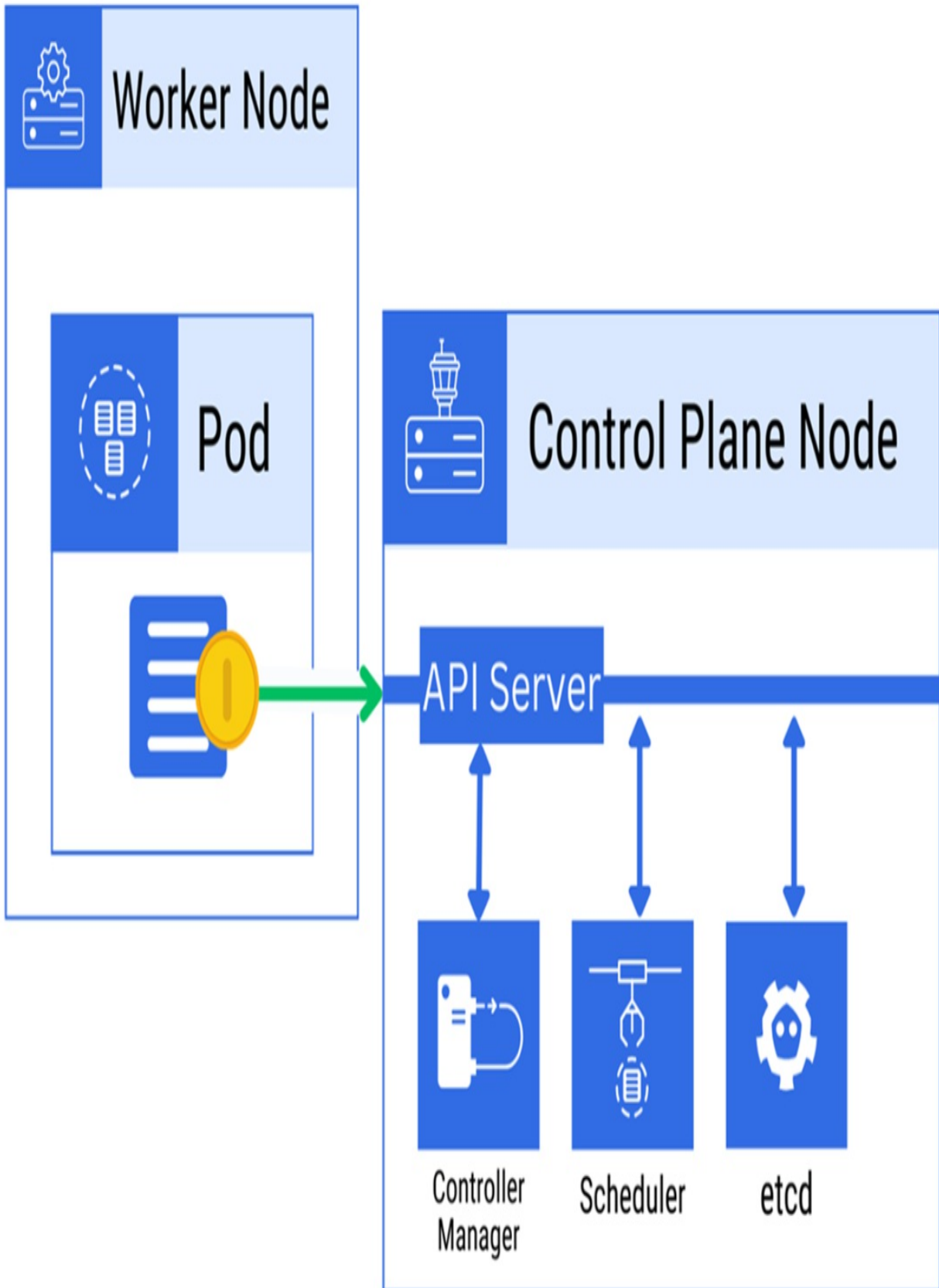
Create a new user named “Sandra”, first creating the private key, then the certificate signing request, then using the CSR resource in Kubernetes to generate the client certificate.

Add that new user “Sandra” to your local kubeconfig using the `kubectl config` command.

3.3 Service accounts

Users and groups are great and all, but what about machines? They need to access the Kubernetes API too, and sometimes we utilize tools like Helm or Jenkins or a Load Balancer Controller, running as pods in our cluster, that need to communicate with the Kubernetes API in order to carry out actions (read, write, delete, etc.) as a part of their primary function. We provide them with a similar authentication mechanism, namely a token, in order to carry out these actions, as depicted in figure 3.12. Then, we can limit their privileges using roles and role bindings, again, making sure that we are following the principle of least privilege. We don’t want a situation where a pod in our cluster has unrestricted access like our `systems:masters` group had. The good news is, this default group is not applied to service accounts in the same way it was to our admin user.

Figure 3.12 Service accounts mounted to pods can access the API in the same way users can



Service accounts are namespace-scoped resources. So, you'll find at least one in each namespace, and that one, called the "default" service account is created automatically each time you create a new namespace. Let's try it!

```
root@kind-control-plane:/# kubectl create ns web
namespace/web created
root@kind-control-plane:/# kubectl get sa -n web
NAME          SECRETS  AGE
default       1        10s
```

Yes, this default service account is created automatically, you don't have to do anything special to make this happen. This is the default behavior. If we run the command `kubectl describe secret -n web`, we can see the token that is also automatically generated with the service account as well.

```
root@kind-control-plane:/# kubectl describe sa -n web
Name:          default
Namespace:     web
Labels:        <none>
Annotations:   <none>
Image pull secrets: <none>
Mountable secrets: default-token-mv8xd
Tokens:        default-token-mv8xd
Events:        <none>
```

Furthermore, you can cat out the token and reveal its value with the command `kubectl describe secret default-token-mv8xd -n web`

```
root@kind-control-plane:/# kubectl describe secret
➡default-token-mv8xd -n web
Name:          default-token-mv8xd
Namespace:     web
Labels:        <none>
Annotations:   kubernetes.io/service-account.name: default
                kubernetes.io/service-account.uid: b0f4ff77-
➡80be-44ab-85b3-b6f662678fff
```

Type: kubernetes.io/service-account-token

Data

====

```
namespace: 3 bytes
token:      eyJhbGciOiJSUzI1NiIsImtpZCI6ImllyN3N
```

```

➡Cd1JPOHZMbkhFZ3BLQkVBbUloZGx0eU5JSEFkS3JuUFRxS
➡3dwWGsifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2
➡NvdW50Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC
➡9uYW1lc3BhY2UiOiJ3ZWl1LCJrdWJlcm5ldGVzLmlvL3NlcnZp
➡Y2VhY2NvdW50L3NlY3JldC5uYW1lIjoizGVmYXVsdC10b2t1bi1
➡tdjh4ZCIsImt1YmVybmV0ZXMuaW8vc2Vydm1jZWJjY291bnQvc2V
➡ydm1jZS1hY2NvdW50Lm5hbWUiOiJkZWZhdWx0Iiwia3ViZXJuZ
➡Rlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQud
➡wlkIjoiyjBmNGZmNzctODBiZS00NGFiLTg1YjMtYjZmNjYyNjc4
➡ZmZmIiwic3ViIjoic3lzdGVtOnNlcnZpY2VhY2NvdW50Ond1Yjpk
➡ZWZhdWx0In0.DoBREFRm9p0UnSfqaI4qemDGFKa5nkXBfXePPQsc
➡Am8C-S4an0X6JtcQShRp04WrDQzqGYCQ2nnClxdxCPd8BPwBV-x
➡BzsidL1Cwg-ixQTElvJLIx0N6CB9FGpiFBBHFY5GRTtX3LFBcWqo
➡UoqNmyDUqJJn1NEKqGzy5-4bjjMNOQ5JYywtjm50cxiGE2f10RB
➡ju7FBzZVWmvzo2XhtVR18LWUdomaZ1IwESXFU5HZNesK-1Mqcv
➡D4C5wWc9igVBBxFJBOMJ06Z_Afi9vitnCTCYarQHcg66Rv0VK-xy
➡rc00RtjQbFLI1rjy68F3zG4BrvU9ChQ0uMPAbCkG1ay4g

```

ca.crt: 1066 bytes

Technically, we could use this token (as a user) to authenticate with the Kubernetes API. This is why the default “view” role doesn’t allow viewing the service accounts and why we are hesitant to give read access of our secrets to any user. These tokens can be used outside of the cluster as well.

When we create a pod, this default service account is automatically mounted to the pod, so the pod can authenticate to the Kubernetes API. Let’s run the command `kubectl get po nginx -o yaml | grep volumeMounts -A14` to list the service account within the pod YAML. You’ll see that the default service account is in fact mounted to the pod, at the path `/var/run/secrets/kubernetes.io/serviceaccount`

```

root@kind-control-plane:/# kubectl get po nginx -o yaml |
➡ grep volumeMounts -A14
  volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: kube-api-access-nhxf2
      readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  nodeName: kind-worker2
  preemptionPolicy: PreemptLowerPriority
  priority: 0
  restartPolicy: Always
  schedulerName: default-scheduler

```

```
securityContext: {}
serviceAccount: default
serviceAccountName: default
terminationGracePeriodSeconds: 30
```

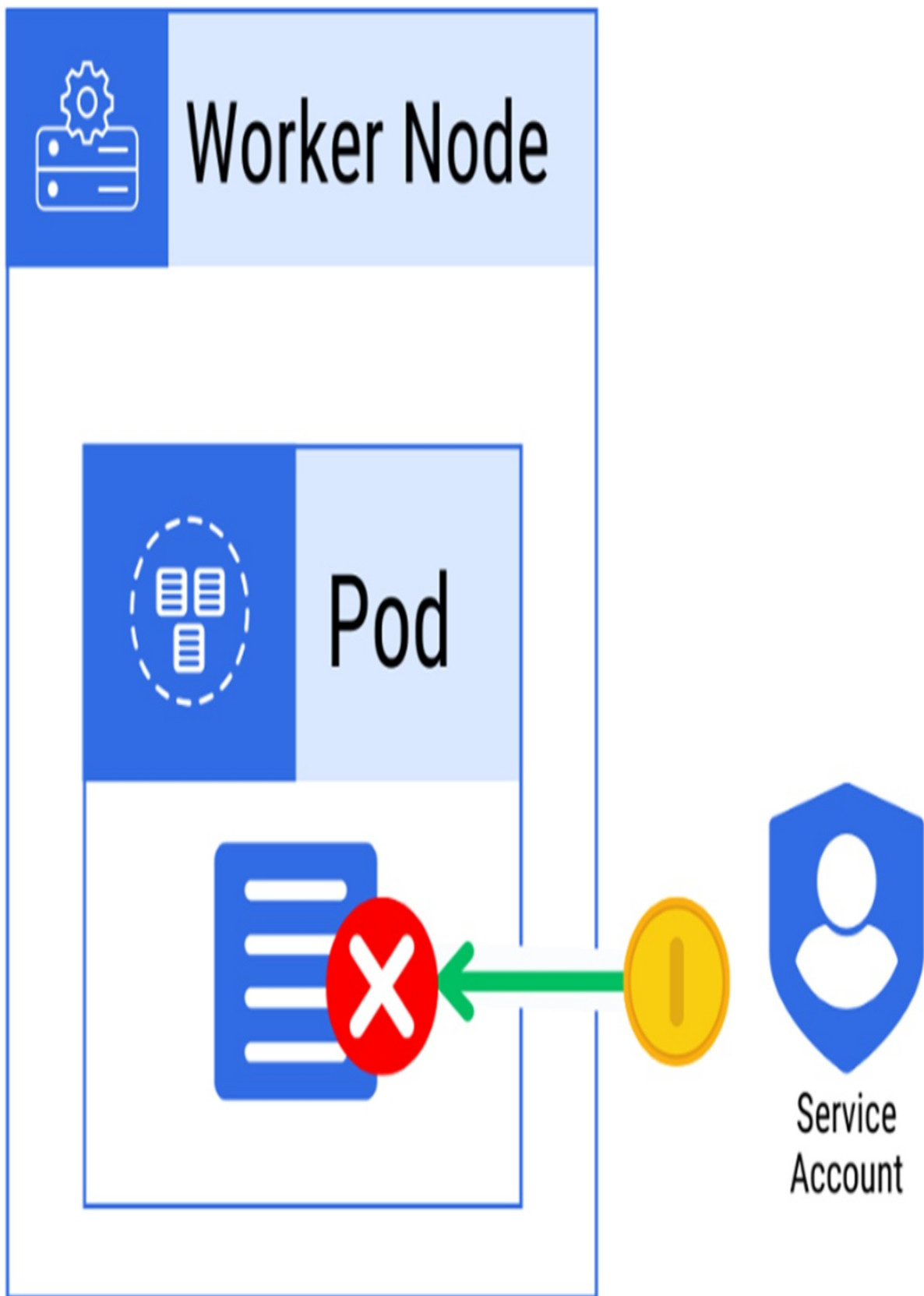
If we exec into the pod, we can view the token, in the same manner as we did before by describing the secret. To get a shell into the pod's container and cat out the token, we can run the command `kubectl exec -it nginx -- sh` followed by `cat`

`/var/run/secrets/kubernetes.io/serviceaccount/token` (the “#” indicates that you have successfully entered a shell inside the container)

```
root@kind-control-plane:/# kubectl exec -it nginx -- sh
# cat /var/run/secrets/kubernetes.io/serviceaccount/token
eyJhbGciOiJSUzI1NiIsImtpZCI6IlliN3NCd1JPOHZNbkhFZ3BLQkV
➤BbUloZGx0eU5JSEFkS3JuUFRxS3dwWGsfQ.eyJhdWQiOiIsiaHR0cH
➤M6Ly9rdWJlcm5ldGVzLmRlZmF1bHQuc3ZjLmNsdXN0ZXIubG9jYWwiX
➤SwiZXhwIjoxNjgwMTQ0NDMxLCJpYXQiOiJlE2NDg2MDg0MzEsImIzcyI6
➤Imh0dHBzOi8va3ViZXJlcy5kZWZhdWx0LnN2Yy5jbHVzdGVyLmx
➤vY2FsIiwia3ViZXJlcy5pbyI6eyJuYW1lc3BhY2UiOiJkZWZhdW
➤x0IiwicG9kIjp7Im5hbWUiOiJlZ2lueCIsInVpZCI6ImRmMzE3OTA3LT
➤gwNTAtNDJmMS05ZWZmLTNhNDY2NzU4M2E5NiJ9LCJzZXJ2aWNlYWVjb3
➤VudCI6eyJuYW1lIjoizGVmYXVsdCIsInVpZCI6ImRkMjlkZWZhdWVzLTkyMj
➤M0NGQyOC1iOGE1LWUwODIxNTU1NGRjZCJ9LCJ3YXJuYWZ0ZXIiOiJlE2NDg
➤2MTIwMzh9LCJlcy5pbyI6eyJuYW1lc3BhY2UiOiJkZWZhdWVzLTkyMj
➤aWNlYWVjb3VudDpkZWZhdWx0OmRlZmF1bHQifQ.ECeBokD1IbvCbBOIi6
➤yiWtV1VIPABtxwnXmtudiQcRdmglQX06MVBEdCVJ4PT_rbnS7oB5fNAYFU
➤2-8xU0EcDofeJH6IFEkALaI09TLCcha5YnQ65C6J1aKV-58BR2I9aI
➤ZDb9uCJ247nyrDc075e2nf5zuBcfX82X5Yl0qAUTuiWDG-HPIwLfVnW
➤Yj9AasiFOMAMVaM_Ydsgru1U2vHwGFi-7vo2CITU33X3NoTyDCSuNaNlAcB08_
➤aWJu0YpQFtlAzpHQJR0XqGveIqu7cCMBd4XuD-m9AQDpkUZY59G0sJ
➤INCzkKo0ch1_nWx_PU42-drEwHU8m1HEdGwsAhNRg#
```

Now, let's say we didn't want to automatically mount the token to every pod that we create. We can turn this feature off from the serviceAccount itself, or we can only dismount the token for certain pods in the pod YAML specification, as depicted in figure 3.13. We can even do both, just in case our team members accidentally forget to address this for the pods they create.

Figure 3.13 You can choose to disable mounting secrets from either the service account or the pod, or both.

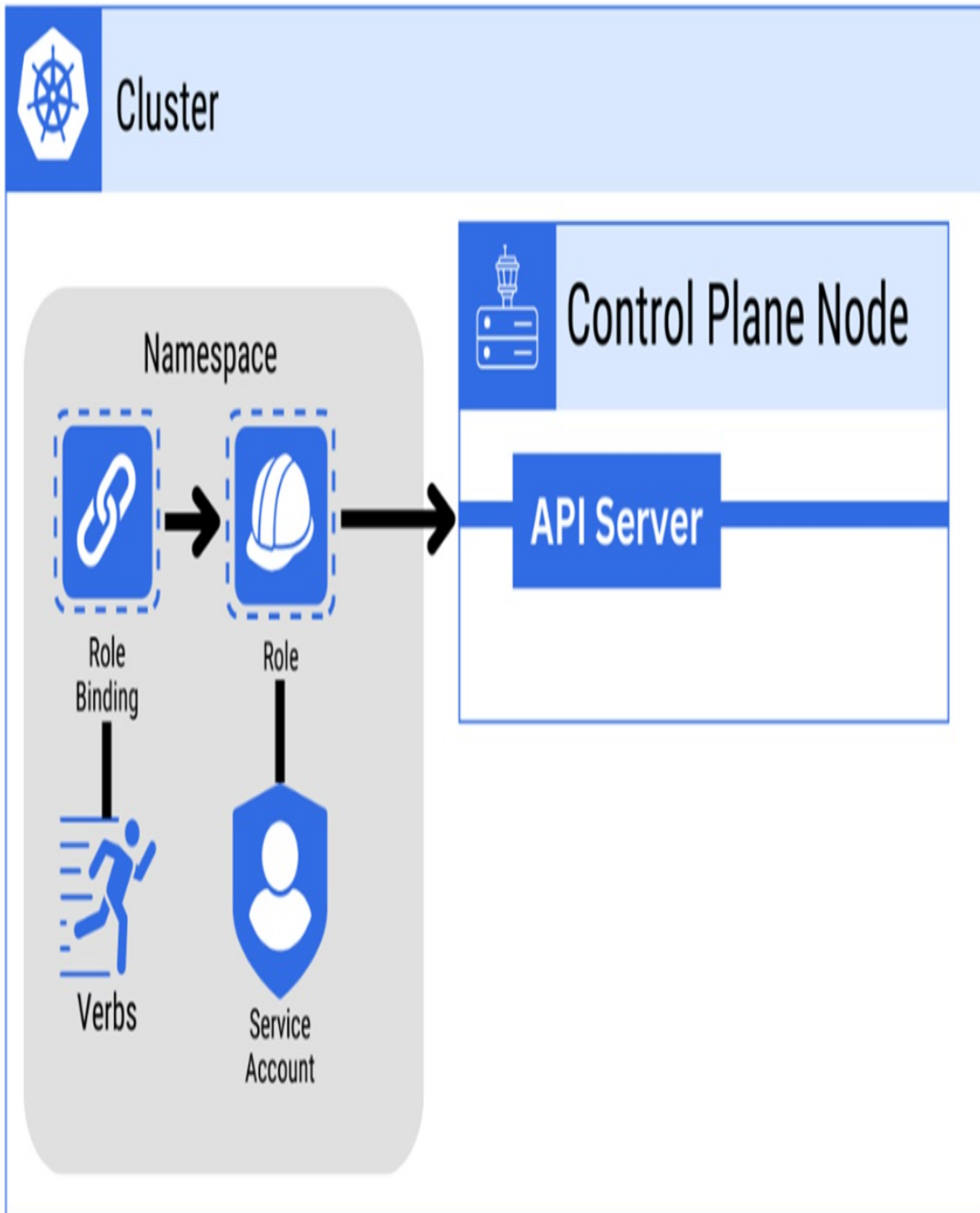


Let's create a new service account and tell the service account not to mount the token to the pod automatically. The quickest way to do this is to create the framework for the service account YAML and then modify it in place with the command `kubectl create sa nomount-sa --dry-run=client -o yaml > nomount-sa.yaml`. Let's add the `automountServiceAccountToken: false` to the end of the file `nomount-sa.yaml` with the following command `echo "automountServiceAccountToken: false" >> nomount-sa.yaml`

```
root@kind-control-plane:/# echo "automountServiceAccountToken: fa
root@kind-control-plane:/# cat nomount-sa.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  creationTimestamp: null
  name: nomount-sa
automountServiceAccountToken: false
```

Now that we have the YAML all set, we can create the service account object with the command `kubectl create -f nomount-sa.yaml`. If we create a new pod and use this service account, we won't see it mounted at all. Let's create a new pod named 'no-mount' and specify within the YAML to use this new service account, much like figure 3.14

Figure 3.14 A role subject can be a user, group or service account in authorizing against the Kubernetes



We'll use the same shortcut method of output a skeleton of the YAML manifest and modifying it afterwards with the command `kubectl run no-mount --image nginx --dry-run=client -o yaml > no-mount-pod.yaml`

NOTE

If you've created a new kind cluster since the last chapter, you'll need to install vim so we can edit this file. Use the command `apt update && apt install -y vim`

Now that we have that file locally, we can edit it using our vim text editor, so just type the command `vim no-mount-pod.yaml` to open it. Just under the spec, indented two spaces, you should add `serviceAccountName: nomount-sa`. It will look like this:

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: no-mount
  name: no-mount
spec:
  serviceAccountName: nomount-sa
  containers:
  - image: nginx
    name: no-mount
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

Now that we have explicitly told the pod to use this service account, let's create the pod and inspect it with the command `kubectl create -f no-mount-pod.yaml` and `kubectl get po no-mount -o yaml | grep volumeMounts -A14`. You will notice that the correct service account is used and you don't see any volume mounts. This is a good thing.

```
root@kind-control-plane:/# kubectl get po no-mount -o yaml | grep
  serviceAccount: nomount-sa
  serviceAccountName: nomount-sa
root@kind-control-plane:/# kubectl get po no-mount -o yaml | grep
root@kind-control-plane:/#
```

Now, let's create yet another pod, and we'll use the default token, which is set to automatically mount tokens to pods, but we'll explicitly tell it not to during the creation of our pod. Let's create a new pod YAML file with the

command `kubectl run default-no-mount --image nginx --dry-run=client -o yaml > default-no-mount-pod.yaml`. Again, let's open the file with vim and just below the spec, indented two spaces, we'll put `automountServiceAccountToken: false`. We don't have to tell the pod to use the default token, because if not specified, it will use the default service account automatically. This is the case for all pods by default. The pod yaml will look like this:

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: default-no-mount
  name: default-no-mount
spec:
  automountServiceAccountToken: false
  containers:
  - image: nginx
    name: default-no-mount
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

Let's create the pod with the command `kubectl create -f default-no-mount-pod.yaml`. Again, we'll see the default service account used with the pod, but no volume mounts, which is what we want.

```
root@kind-control-plane:/# kubectl get po default-no-mount -o yaml
serviceAccount: default
serviceAccountName: default
root@kind-control-plane:/# kubectl get po default-no-mount -o yaml
root@kind-control-plane:/#
```

Now that we've learned about creating service accounts and mounting them to pods (or explicitly not mounting them to pods), let's now go through assigning roles and rolebindings to service accounts. This happens in a similar fashion to creating any other role or role binding, but the subject is going to be a service account instead of a user or a group. Let's start by creating a role that will only allow the action "list" for the resource "pods". To do this we can run the command `kubectl create role pod-list --`

verb=list --resource=pods. Now, we'll create the role binding named "pod-list-bind" to bind the service account to the role "pod-list" for the service account "nomount-sa" `kubectl create rolebinding pod-list-bind --role=pod-list --serviceaccount=default:nomount-sa`

```
root@kind-control-plane:/# kubectl create role pod-list --verb=list --resource=pods --rbac.authorization.k8s.io/pod-list created
root@kind-control-plane:/# kubectl create rolebinding pod-list-bind --role=pod-list --serviceaccount=default:nomount-sa --rbac.authorization.k8s.io/pod-list-bind created
```

To test that our role and role binding worked for this service account, we can use the `kubectl auth can-i` command to see if we can list pods as this service account. To do this we'll run the command `kubectl auth can-i list pods --as system:serviceaccount:default:nomount-sa`.

```
root@kind-control-plane:/# kubectl auth can-i list pods --as system:serviceaccount:default:nomount-sa
yes
```

The service account is addressed this way (`system:serviceaccount:default:nomount-sa`) because this is the username of the service account according to the RBAC plugin. This is the perspective that we are looking at the service account from, so we need to address it with the "system:serviceaccount" prefix followed by the namespace (in our case it was "default"), then the service account API resource name. Congratulations! You have created a service account and assigned it to a role via a role binding in Kubernetes!

You can now use this service account for multiple pods at the same time, as long as those pods exist within the same namespace. You can never use a service account from a different namespace for a pod. Also, to use the "nomount-sa" service account for new pods, you'll just have to specify the service account to use in the pod YAML, for each pod, like we did in the previous example. You may have an idea to delete the default service account, but this is not possible. The service account resource has its own controller, which allows it to be automatically created when we create a new namespace.

Practice exercises

Create a new service account named 'secure-sa' and create a pod that uses the 'secure-sa' service account. Make sure the token is not exposed to the pod.

Create a new cluster role named 'acme-corp-role' that will allow the create action on deployments, replicates, and daemonsets. Bind that cluster role to the service account 'secure-sa' and make sure the service account can only create the assigned resources within the default namespace and nowhere else. Use `auth can-i` to verify that the 'secure-sa' service account cannot create deployments in the kube-system namespace and output the result of the command + the command itself to a file and share that file.

3.4 Summary

- Anything that tries to access Kubernetes must be authenticated via certificate carrying a token to authenticate with.
- The authentication plugin is used in Kubernetes to provide a mechanism to accept or deny user requests.
- Kubernetes has its own certificate signing request object, which can sign certificates to make them valid for a user or anything trying to access the Kubernetes API.
- A common plugin called Role-Based Access Control(RBAC) is used to give permissions to users in a Kubernetes cluster.
- Users and groups are arbitrary meanings in Kubernetes, so there is no user database or directory structure. The certificate is the only thing that's checked, so the user or group can be any name.
- You can assign RBAC permissions via roles, cluster roles, role bindings and cluster role bindings, each used for different use cases.
- All combinations of roles, cluster roles, role bindings and cluster role bindings can be used for the exam, except for one. Cluster role binding to roles.
- There are two main subjects in RBAC, the user and service accounts. Users are not a managed resource in the Kubernetes API, whereas service accounts are.
- When creating a user, the common name inside the certificate signing request is required. You can generate a client certificate from the Kubernetes API, which is its own certificate authority
- When creating a role for the exam, you can use the "dry run" option and

save yourself some time typing the YAML for a role where values need to be added.

- Built-in groups are different than regular groups in Kubernetes. Built-in groups are managed by the API server and cannot be modified.
- Service accounts are automatically created when new namespaces are created. The token inside of a service account is mounted to a pod, unless otherwise specified in the YAML manifest.
- A service account token is automatically mounted to a pod, unless specified in the pod YAML. You can also disable the mounting of a token when you first create a service account.
- A useful tool to use during the exam is called `kubect1 auth can-i` and it can test permissions for users and service accounts.

4 Deploying applications in Kubernetes

This chapter covers

- Scheduling pods to nodes
- Creating multiple containers within a pod
- Requesting and limiting the resources a pod can take
- Passing configuration data to pods

Now let's shift our focus to the workloads and scheduling part of the exam objectives and focus on the many different aspects of scheduling and their intricacies. Deploying applications to run on Kubernetes is commonly referred to as “scheduling”, which makes sense because we're placing a pod on a node to run. There are many ways to deploy applications (aka schedule pods) to Kubernetes, as we'll discover in this chapter. You will realize that one size doesn't fit all, which creates freedom to run the applications in the environment that's best for the containerized application. This includes the ability to reserve resources from the underlying infrastructure as well as decouple components such as configuration information and sensitive data within.

The workloads & scheduling domain

This chapter covers part of the workloads and scheduling domain of the CKA curriculum. This domain covers the ways in which we run applications on Kubernetes. It encompasses the following competencies.

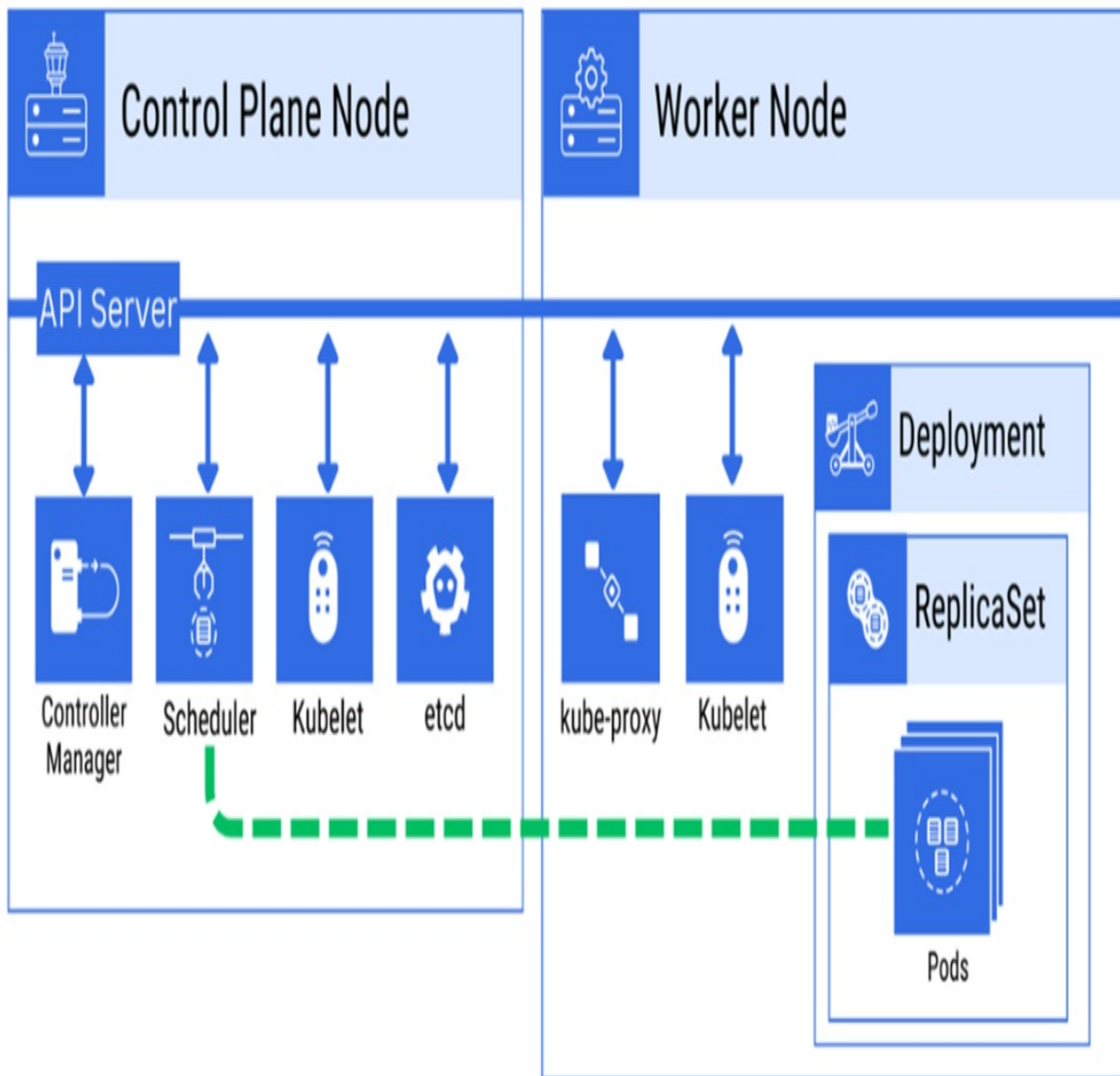
Competency	Chapter Section
Use ConfigMaps and Secrets to configure applications	4.3
Understand how resource requests and limits can affect Pod scheduling	4.3

Awareness of manifest management and common templating tools	4.2
Understanding the primitives used to create robust, self-healing, application deployments	4.1

4.1 Scheduling applications

As we've discovered in previous chapters, the Kubernetes component that controls the assignment of a pod to a node is called the scheduler. The scheduler will not only tell the pod to run on a specific node, but it will also ensure that the resources are available on the node so that the pod can run successfully, as depicted in figure 4.1.

Figure 4.1 The scheduler schedules the pods to run on the worker node after verifying resource availability



The CKA exam will test your knowledge of creating and updating application deployments. This includes scheduling to certain nodes, according to their labels. For example, an exam question might say “Apply the label ‘disk=ssd’ to the worker node named ‘kind-worker’ and schedule a pod named ‘ssd-pod’ to ‘kind-worker’ node by using node selector”. In order to better approach a task like this when we sit for the exam, let’s create a new pod from YAML with the command `kubectl run ssd-pod --image=nginx --dry-run=client -o yaml > ssd-pod.yaml`. This command will save the pod YAML in a file named `ssd-pod.yaml`. Before we start modifying the

YAML, we have to show the existing labels on each node in our cluster with the command `kubectl get no --show-labels`.

```
root@kind-control-plane:/# kubectl get no --show-labels
NAME                                STATUS    ROLES                                AGE    VERSION
kind-control-plane                 Ready     control-plane,master                13d    v1.23.4
  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,
  ➤kubernetes.io/arch=amd64,kubernetes.io/hostname=kind-control-plane,
  ➤kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,
  ➤node-role.kubernetes.io/master=,node.kubernetes.io/exclude-from-
  ➤-external-load-balancers=
kind-worker                        Ready     <none>                               13d    v1.23.4
  ➤beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,
  ➤kubernetes.io/arch=amd64,kubernetes.io/hostname=kind-worker,
  ➤kubernetes.io/os=linux
kind-worker2                       Ready     <none>                               13d
  ➤v1.23.4 beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,
  ➤kubernetes.io/arch=amd64,kubernetes.io/hostname=kind-worker2,
  ➤kubernetes.io/os=linux
```

EXAM TIP

On the exam, there may not be a label pre-applied to the node. You can check the labels on a node with the command `kubectl get no --show-labels`

Let's apply a new label to the node named `kind-worker` that will signify that this node has an SSD disk as opposed to HDD. This way, the pod can be scheduled to the `kind-worker` node by using a node selector. To apply a label to a node, we use the syntax "key=value" so we can select that label at a later time. To apply a label with the key "disktype" and value "ssd", we can use the command `kubectl label no kind-worker disktype=ssd`. Then we can run the show labels command again to see that the label was applied successfully.

```
root@kind-control-plane:/# kubectl label no kind-worker disktype=ssd
node/kind-worker labeled
root@kind-control-plane:/# kubectl get no --show-labels
NAME                                STATUS    ROLES                                AGE    VERSION
kind-control-plane                 Ready     control-plane,master                13d    v1.23.4
  ➤beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,
  ➤kubernetes.io/arch=amd64,kubernetes.io/hostname=kind-control-plane,
  ➤kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,
  ➤node-role.kubernetes.io/master=,node.kubernetes.io/exclude-from-
```

```

➡-external-load-balancers=
kind-worker          Ready    <none>                13d    v1.23.
➡beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,
➡disktype=ssd,kubernetes.io/arch=amd64,kubernetes.io/hostname=
➡kind-worker,kubernetes.io/os=linux
kind-worker2         Ready    <none>                13d    v1.23.
➡  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,
➡kubernetes.io/arch=amd64,kubernetes.io/hostname=kind-worker2,kul

```

Now that the worker node is labeled, we can add the node selector to our pod YAML, to ensure that the pod gets scheduled to this newly labeled node. Go ahead and open the file “ssd-pod.yaml” in a text editor like vim (any text editor will do). Once we have it open, we can add the line “nodeSelector:” in line with the word containers, and just below that add “disktype: ssd” indented two spaces. We indent two spaces to indicate that the specified parameters are grouped with the previous block above it (e.g. disktype is contained within the block) as you will see in figure 4.2.

Figure 4.2 Selecting the type of node to schedule a pod to, based on the node label disk=ssd

```
spec:
  containers:
  - image: nginx
    name: ssd-pod
  nodeSelector:
    disktype: ssd
  dnsPolicy: ClusterFirst
  restartPolicy: Always
```

Placed in line with containers.
diskType is indented two spaces

After you've added this, you can save the file. We can use the command `kubectl apply -f ssd-pod.yaml` to create the pod and schedule it to the node which we specified in the pod yaml by using the node selector. Let's now verify that the pod was scheduled to the intended node by running the command `kubectl get po -o wide`.

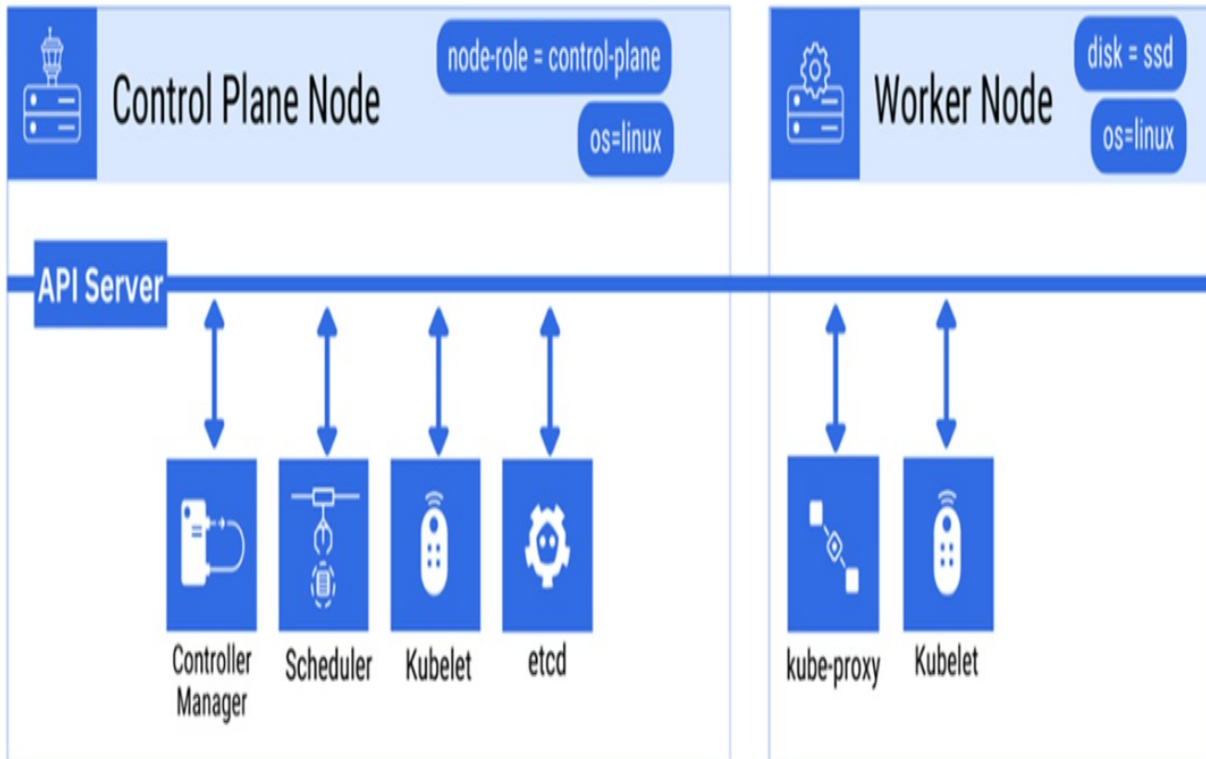
```
root@kind-control-plane:/# kubectl get po -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE
ssd-pod       1/1     Running   0           25s   10.244.1.2    kind-wo
```

Looks like the pod was scheduled to the correct node, and the rules for the node selector applied successfully. Great job!

There are other scenarios in which you would want to change how pods are

scheduled. In most cases, however, you want to limit your alterations to scheduling in Kubernetes. Keep in mind that changing the default rules for scheduling changes the primary functionality of Kubernetes, which is to promote high availability and fault-tolerance. You may have a specific need to schedule pods to certain nodes (as in the previous task), but normally the scheduler is going to distribute workloads properly and efficiently. In figure 4.3, you can see the default labels for both the control plane node and the worker nodes, with the addition of the `disktype` label we applied to `kind-worker` earlier in this chapter.

Figure 4.3 All nodes can have different labels which you can then select, and schedule pods to by selecting those labels



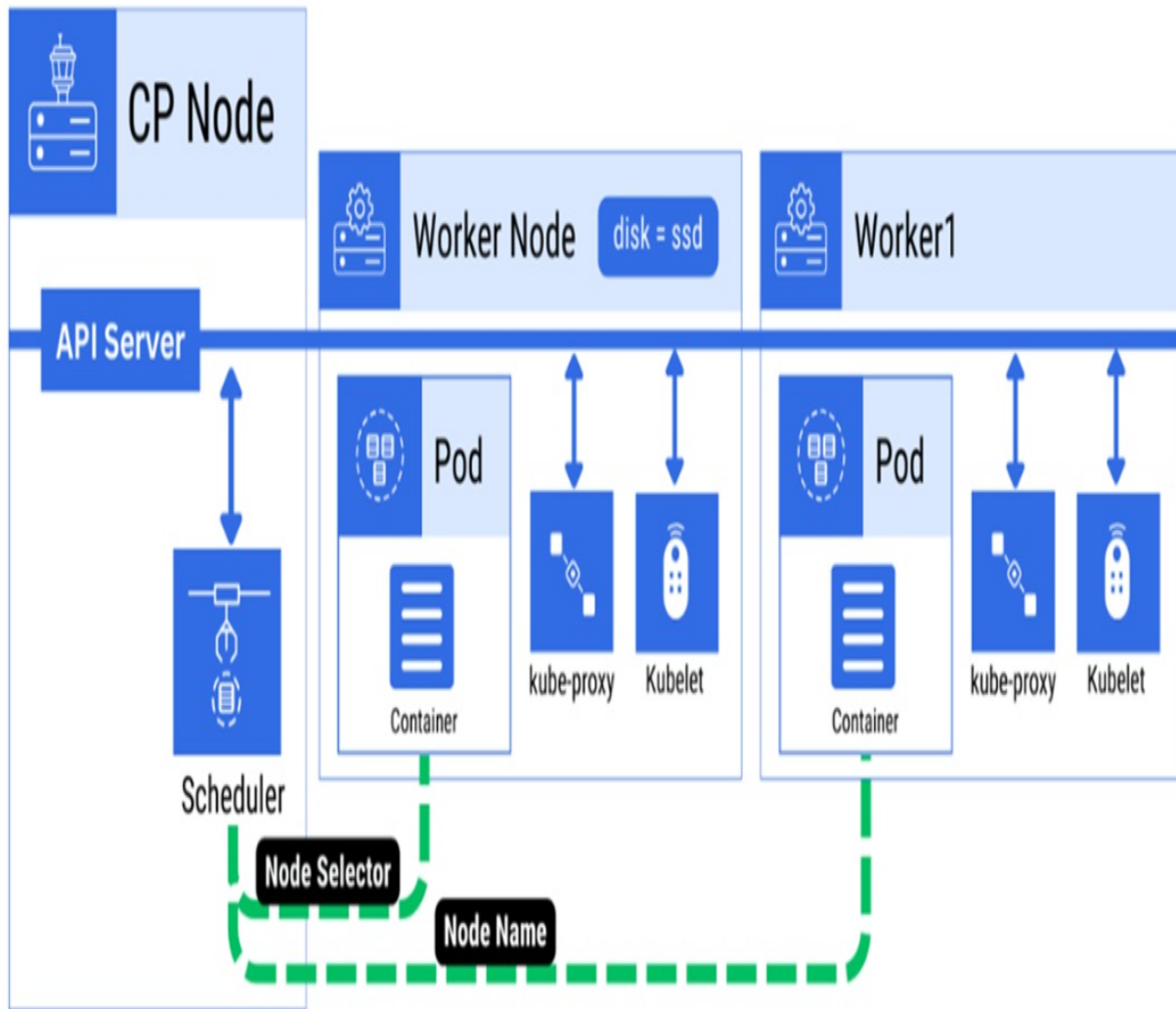
4.1.1 Node selectors

As we discussed in the previous section, there may come a time when you'd like to schedule a pod to a specific node. Perhaps that node has a certain type of CPU or memory that is optimized for your workload. There are many things we can use inside of our pod YAML (and deployment YAML), that

will help solve this. Let's talk about a few of them now, the node selector and the node name attribute inside of the manifest. We can add either one (not both) to the YAML manifest and be able to schedule the pod on a specific node. What's the difference? Well, the node selector is a little bit more flexible, as it identifies a node by its labels. This way, the labels can be applied to many nodes, not just one.

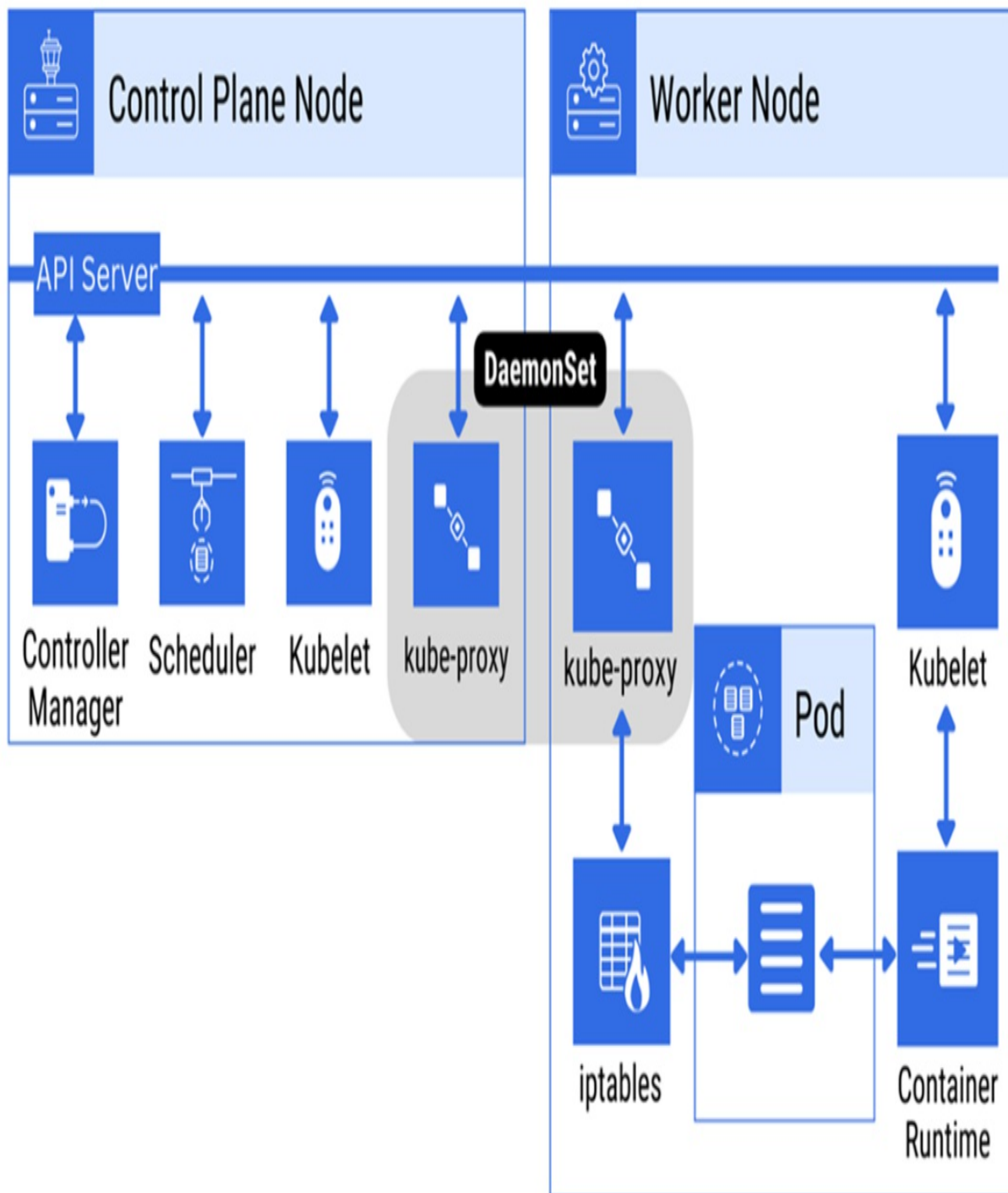
Adding a node name instead of a node selector is just the opposite. You can specify the specific node by name, but as we know, we can't have two nodes with the same name, so that limits our options a bit. What if we have node failure? What if the node runs out of resources? Creating a single point of failure by only scheduling a pod to a single node is not the best idea. As you'll see in figure 4.4, you can force a pod to be scheduled to a specific node by label or by name.

Figure 4.4 When scheduling a pod you can force the node it's scheduled to based on the node's name or label



A daemonSet is a type of Kubernetes object that ensures that one pod replica is always running on every single node in the cluster at all times. Even if you delete the pod, the daemonSet will respawn the pod on any node that's missing that daemonSet's pod. DaemonSets use the node name attribute for all pods within a daemonSet to schedule pods to specific nodes by their name. This ensures that one and only one pod is assigned to a node, and by using the node name, they ensure that there aren't accidentally two pods running on a single node, as depicted in figure 4.5 with the "kube-proxy" pod being an example of one daemonSet that runs in every Kubernetes cluster.

Figure 4.5 The kube-proxy pods are a part of a Daemonset and reside on each node in the cluster



Let's look at one of the daemonSet pods running in our cluster with the command `kubectl -n kube-system get po kindnet-2t1qh -o yaml | grep nodeName -a3` and you'll see output similar to figure 4.6.

Figure 4.6 Scheduling a pod by specifying the node name in the pod YAML

```
root@kind-control-plane:/# k -n kube-system get po kindnet-2tlqh -o yaml | grep nodeName -a3
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  hostNetwork: true
  nodeName: kind-control-plane
  preemptionPolicy: PreemptLowerPriority
  priority: 0
  restartPolicy: Always
```

 This pod will always be scheduled to the node named "kind-control-plane"

In this command, we are getting the YAML output of the kindnet daemonSet pod, which resides in the 'kube-system' namespace. Then, we can search through the YAML and find the instance of the node name with the grep tool. As you can see, the node name is added to the pod YAML for this daemonSet pod. Remember, the pod name will be different for your cluster, so replace the name kindnet-2tlqh with the pod name in your cluster. The command to retrieve the pod names is `kubectl get po -A`. Also, to look at the daemonSets running in your cluster, type the command `kubectl get ds`.

Hopefully, you can better understand where the node name is used and why it's used. In most cases, you will use a node selector, and you can probably see the benefits much more clearly. The node selector is used in many more scenarios, as it can apply to many nodes, selecting the node labels. For example, you can label nodes with "disk=ssd" to indicate that they have solid state drives, which may make a difference for applications that require high disk I/O.

Exam Tip

You may see node selector or node name on the exam, where the question might ask you to add the necessary values to an existing pod YAML. So, become familiar with the placement and proper syntax used here.

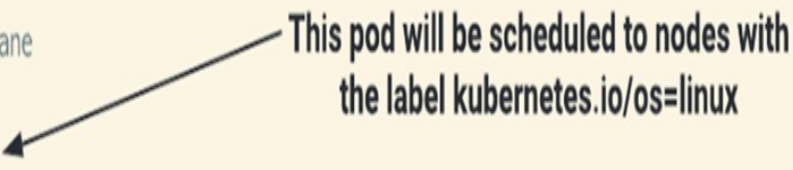
The same goes for CPU or memory intensive applications, as the node's resources have a direct impact on how the applications will run inside of pods running containers. To take a look at how node selectors are used in our cluster, look no further than the core DNS pods that are already running in the kube-system namespace. To see the pod YAML for a core-dns pod, we'll use the same method we did with the daemonSet pods with the command `kubectl -n kube-system get po coredns-64897985d-4th9h -o yaml | grep nodeSelector -a4` and you'll see output similar to figure 4.7.

NOTE

The name of your coredns pod will be different, so use the command `kubectl -n kube-system get po` to find your unique name

Figure 4.7 Selecting which node this pod will run on by it's label, specified in the pod YAML

```
root@kind-control-plane:/# k -n kube-system get po coredns-64897985d-4th9h -o yaml | grep nodeSelector -a4
    readOnly: true
    dnsPolicy: Default
    enableServiceLinks: true
    nodeName: kind-control-plane
    nodeSelector:
      kubernetes.io/os: linux
    preemptionPolicy: PreemptLowerPriority
    priority: 2000000000
    priorityClassName: system-cluster-critical
```



This pod will be scheduled to nodes with the label `kubernetes.io/os=linux`

In this command, we are getting the pod YAML output for the core DNS pod, which resides in the kube-system namespace, and searching for the word “nodeSelector”. Remember, the pod name will be different for your cluster, so replace the name “coredns-64897985d-4th9h” with the pod name in your cluster. The command to retrieve the pod names is `kubectl get po -A`.

Now that we've taken a look at node names and node selectors, as well as a

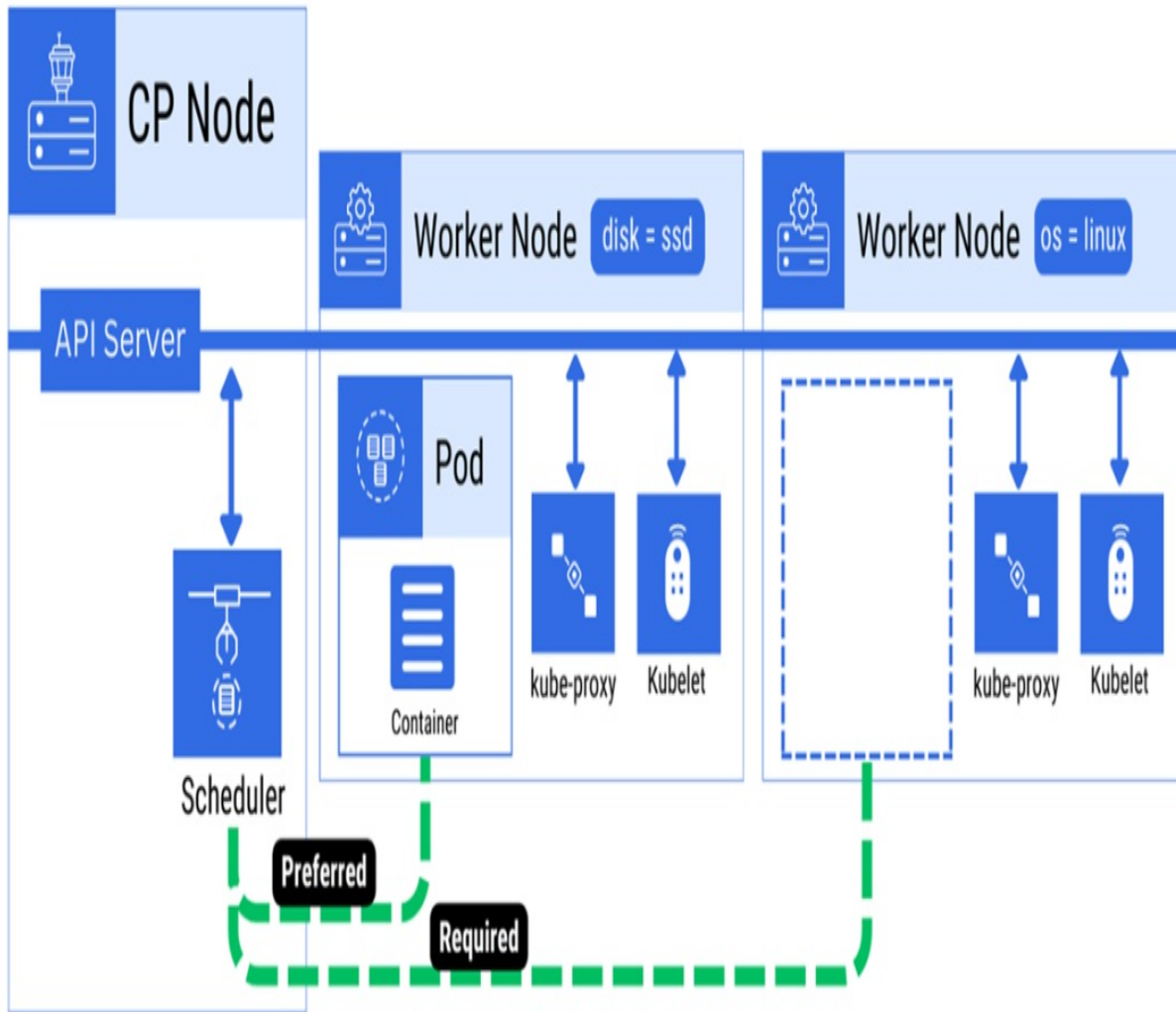
useful way to edit Kubernetes objects inline, let's move on to another scheduling technique-- called *affinity*-- to favor certain nodes over others.

4.1.2 Node and pod affinity

As we know, labels can be applied to both pods and nodes. We can use these labels to apply affinity. Affinity is a lot like a node selector, but its rules are more expressive. For example, a node selector rule will select a node with a label, whereas an affinity rule will prefer a node with a label, but will also accept scheduling to another node if that node label is not present.

In order to better understand affinity, we have to understand the problem it solves. There is an inherent problem in larger Kubernetes clusters where not only pods come and go, but nodes do as well. So, setting hard rules for the pod to be scheduled to a node based on its name or label don't suffice. Instead of alienating a pod if a node is not present, affinity provides a more flexible ruleset, so applications can be scheduled to a cluster environment that is constantly changing. As you see in figure 4.8, you can set rules that prefer one node, but if that node is unavailable it will still have the option to be scheduled to a second node.

Figure 4.8 Affinity prefers the pod be scheduled to a node with the label "SSD", but will schedule to node "LINUX" if none exist



Let's now go through an example of setting an affinity rule. We'll start with a brand new pod, and we'll use the dry run to create a template for our pod YAML, as we did previously with the command `kubectl run affinity --image nginx --dry-run=client -o yaml > affinity.yaml`. Once you have a file named "affinity.yaml" in your current directory, let's open it up in a text editor and add our node affinity rules. Just below the spec, in line with "containers", add "affinity:". Indent two spaces and add "nodeAffinity:", followed by "requiredDuringSchedulingIgnoredDuringExecution:". See Figure 4.9 for the remaining YAML syntax, as it is quite extensive.

Figure 4.9 Variations of affinity that prefer to schedule a pod to a node based on certain expressions

```
spec:
  containers:
  - image: nginx
    name: affinity
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: kubernetes.io/os
            operator: In
            values:
            - linux
        preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 1
          preference:
            matchExpressions:
            - key: disktype
              operator: In
              values:
              - ssd
```

Must schedule the pod to a node that has the label "kubernetes.io/os=linux" (required)

Also, schedule that pod to a node that has the label "disktype=ssd" (not required)

Below node affinity, where we have set the required attribute, indicates that the scheduler can't schedule the Pod unless this rule is met. This functions like nodeSelector, but with a more expressive syntax. The preferred attribute, however, is optional for the scheduler. If there isn't a node that matches this preference, then the pod will still be scheduled. Also, you can specify a

weight between 1 and 100 for each preferred rule, which the schedule evaluates based on the total weight based on the score of other priority functions for the node. Nodes with the highest total score will be prioritized.

Now that we've added our affinity rules, let's schedule our pod with the command `kubectl apply -f affinity.yaml`

```
root@kind-control-plane:/# kubectl apply -f affinity.yaml
pod/affinity created
root@kind-control-plane:/# kubectl get po -o wide
```

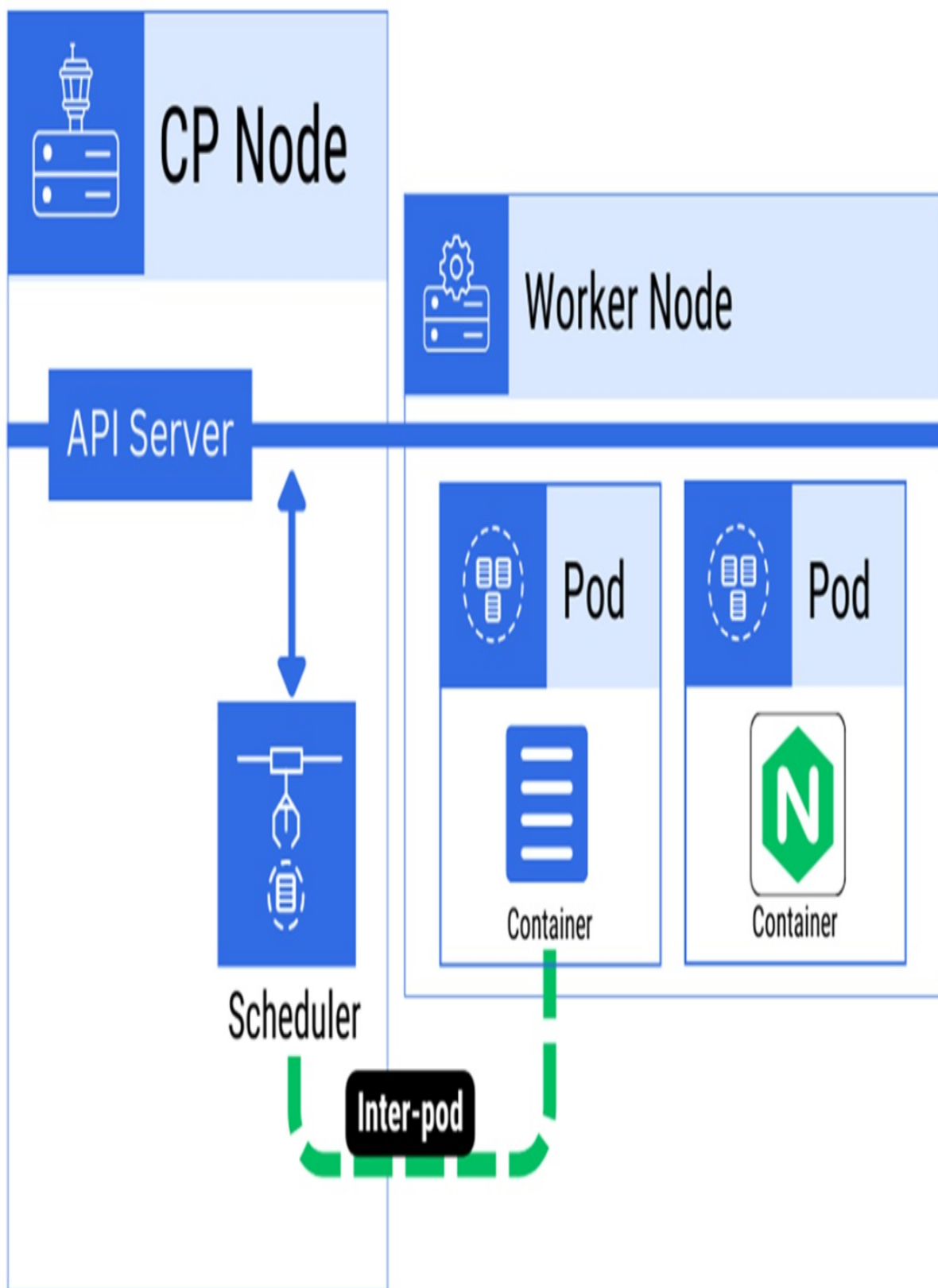
NAME	READY	STATUS	RESTARTS	AGE	IP	N
affinity	1/1	Running	0	14s	10.244.1.6	k
pod-tolerate	1/1	Running	0	24h	10.244.2.16	k

From the above output of the command `kubectl get po -o wide`, we see that the pod was scheduled to a node that did not have the `disktype=ssd` label (`kind-worker2`). That's because the weight that we set for this preferred rule was too low, and since there was another pod already running on `kind-worker`, the node `kind-worker2` had a higher total weight overall, being that it did not already have a pod running on it (higher weight equals greater probability that a pod will be scheduled).

If you didn't quite follow with the above, or if you'd like to check your work, apply this YAML to create your pod with the command `kubectl apply -f https://raw.githubusercontent.com/chadmcrowell/k8s/main/manifests/node-affinity.yaml`.

In the same vein as node affinity is inter-pod affinity, which is the preference to a node based on other pods that are already on that node. To put it in simpler terms, if a pod already exists on the node (e.g. `nginx`), then go ahead and schedule a new pod to that same node (the node with the `nginx` pod on it), as depicted in figure 4.10. The way Kubernetes checks if the pod exists is by its label. This works well with pods that need to run on the same node because of increased performance or latency requirements.

Figure 4.10 Pod will only be scheduled to a node that already has an NGINX pod running

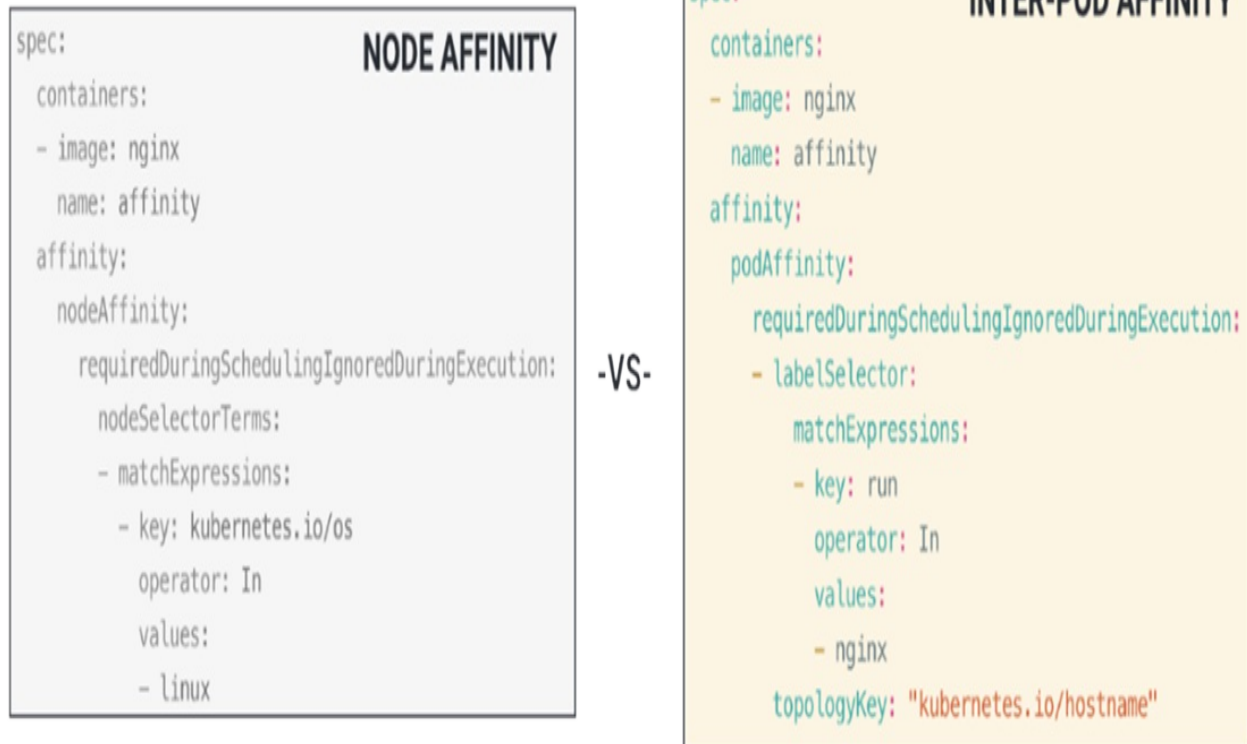


To go through a demonstration of inter-pod affinity, let's first create a pod that has the label `run=nginx` with the command `kubect1 run nginx --image=nginx`. Now, let's go ahead and use the same YAML file that we were using for node affinity, but change only a few lines, because, as you will see, setting the inter-pod affinity is very similar to how we set node affinity.

If you don't already have the previous YAML file, you can download it using `curl` and the `"-O"` flag to save the file with the same name using the command `curl -O https://raw.githubusercontent.com/chadmcrowell/k8s/main/manifests/node-affinity.yaml`

Go ahead and open the file in your text editor and change the name of the pod to `"pod-affinity"` to prevent a naming conflict if you still have the pod running from the previous demonstration. In addition, change `"nodeAffinity"` to `"podAffinity"`, change the line `"nodeSelectorTerms:"` to `"- labelSelector:"`, remove the hyphen from `"matchExpressions"` and change the key to `"run"` and value to `"nginx"`. Finally, add the `"topologyKey"` below the values, to be in line with the word `"labelSelector"`. Topology key is required for inter-pod affinity and simply indicates the node label key. In this case, all nodes in our cluster have the `"kubernetes.io/hostname"` key, so this will match all nodes. See figure 4.11 for a comparison between node affinity and inter-pod affinity. You can remove everything below the topology key, as in this example we're just focusing on the inter-pod affinity required rule.

Figure 4.11 How inter-pod affinity is different than node affinity in its structure within the pod YAML



Now that our nginx pod is running and we've typed out the YAML for our new pod with inter-pod affinity rules, we can go ahead and schedule the pod with the command `kubectl apply -f pod-node-affinity.yaml`. Just after you schedule the pod, you should see that the pod has been scheduled to the same node as the pod named "nginx" by running the command `kubectl get po -o wide`, and you'll see output similar to figure 4.12.

Figure 4.12 The pods have been scheduled to the same node based on their affinity rules

```

root@kind-control-plane:/# k run nginx --image=nginx
pod/nginx created
root@kind-control-plane:/# vim pod-node-affinity.yaml
root@kind-control-plane:/# k apply -f pod-node-affinity.yaml
pod/pod-affinity created
root@kind-control-plane:/# k get po -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
affinity	1/1	Running	0	5d22h	10.244.1.6	kind-worker2
nginx	1/1	Running	0	50s	10.244.1.7	kind-worker2
pod-affinity	1/1	Running	0	24s	10.244.1.8	kind-worker2
pod-tolerate	1/1	Running	0	6d22h	10.244.2.16	kind-worker

Scheduled to the same node

You can see how under normal circumstances, the pod would have been scheduled to the node named “kind-worker”, but because we told the scheduler to find all pods with the “run=nginx” label and schedule a pod named “pod-affinity” to that very same node, it overruled the default scheduler settings.

If you are getting different results, or maybe you can’t get your pod to be scheduled at all, please compare what you have to the YAML in this file, which you can download using the command `curl -O https://raw.githubusercontent.com/chadmcrowell/k8s/main/manifests/with-pod-affinty.yaml`

Practice exercises

Apply the label “disk=ssd” to a node. Create a pod named “fast” using the nginx image and make sure that it selects a node based on the label “disk=ssd”.

Edit the “fast” pod using `kubectl edit po fast` and change the node selector to “disk=slow”. Notice that the pod cannot be changed and the YAML was saved to a temporary location. Take the YAML in /tmp/ and

apply it by force to delete and recreate the pod using a single imperative command.

Create a new pod named “ssd-pod” using the nginx image, and use node affinity to select nodes based on a weight of 1, to nodes that have a label `disk=ssd`. If the selection criteria doesn’t match, it can also choose nodes that have a label `kubernetes.io/os=linux`

4.2 Pod metadata

After discussing scheduling in great length, let’s talk about some of the customizations you can make before scheduling your applications in Kubernetes. There are a number of options, which we’ll cover in this section, but as you can imagine, there’s no one size fits all. Not all Kubernetes objects (deployments, pods, secrets, config maps, etc.) are created in the same way. Sometimes you’ll need to consider the underlying hardware, not only in terms of selecting nodes to schedule pods to, as we discussed in sections 4.1.2 and 4.1.3, but also in terms of resource constraints, additional features, and unique characteristics that each application may need. Let’s start with resource requests and limits, which will help us with scheduling a pod to a node with adequate resources (CPU & RAM).

4.2.1 Resource requests and limits

Resource requests are much like labels, in that they provide information to the scheduler which determine the node in which that pod is scheduled to or placed on. The request comes in the form of a CPU and RAM minimum that the pod requires in order to run optimally. The limit also comes in the form of a value of CPU or RAM, but determines the maximum that the pod should consume, versus the minimum. These two parameters don’t have to be defined at the same time, meaning, you can apply requests without limits and vice versa. In the same vein, you can specify a request of only CPU without specifying a request of RAM. So, these fields are not dependent on each other and totally optional.

On the CKA Exam, you will be tested on your knowledge about resource limits and how it can affect how pods are scheduled. An example of an exam

question might be “Node worker-1 has 500MB of memory and .5 CPU available. Schedule a pod that would allow for this memory and CPU limitation, and still schedule to the node.” Knowing how to modify the pod YAML in order to change resource limits will enable you to successfully complete questions like this on exam day.

You can apply resource requests and limits for a pod by adding it into the container spec via YAML. Inline with the name and image of the container, you can add requests and limits as values below the resources. In order to best accomplish this (without having to know the exact placement within YAML), you can create a dry run of a pod again by performing the command `kubect1 run nginx2 --image=nginx --dry-run=client -o yaml > nginx2.yaml`

When you open the file in your favorite text editor, you’ll already see a resources section. As we know from the YAML language, we can simply indent two spaces below the word “resources” and begin defining our requests and limits. The word resources is used to insert resource limits and requests. Resource requests are CPU and memory minimums that you can specify for pods. Resource limits are the limits you can specify for the same. We’ll cover resource limits and requests later in this book. For now, let’s delete the curly braces after the word resources, as the curly braces indicate a blank entry, so we’re not specifying any resource limits or requests for our pods. So, deleting them will have the same effect. For this container, we’ll request 100 milicores of CPU and 128 mebibytes, and we’ll limit the container to using no more than 2 CPUs and 1 gigabyte of memory. You can see where to insert the requests and limits in the YAML spec in figure 4.13

Figure 4.13 Where to insert the resource requests for pods within the pod YAML

```
spec:
  containers:
  - image: nginx
    name: nginx2
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: 2
        memory: "1Gi"
    dnsPolicy: ClusterFirst
```

The container requires at least 100 milicores of CPU and 128 mebibytes in order to run, but not more than 2 CPUs and 1 gigabyte of memory

Now, you may be asking why the different measurements of CPU and memory. In the Kubernetes ecosystem, limits and requests for CPU are measured in CPU units. 1 CPU unit is equivalent to 1 physical CPU core, or 1 virtual core, depending on whether the node is a physical host or a virtual

machine running inside a physical machine. You can write the value in millicores, whereas the value “1000m” would be the same as “1 CPU”. Fractional requests are also allowed, so instead of 1 CPU, we can enter a value of “0.5 CPU” (equal to “500m”). Memory is measured in mebibytes, which is a measurement based on powers of two (A mebibyte equals 2^{20} or 1,048,576 bytes).

The reason we specify the requests and limits is because, if the node where a pod is running has enough of a resource available, it's possible (and allowed) for a container to use more resource than its request for that resource specifies. However, a container is not allowed to use more than its resource limit.

4.2.2 Multi-container pods

Sometimes in Kubernetes, it makes sense to have more than one container running inside of the same pod. This can be useful in some cases because each container inside of a pod shares the name network namespace and storage, so communication is faster and you don't have to establish additional services and network policies in order to communicate between pods. There are a couple of reasons for creating multiple containers inside of the same pod. One is for log collection, where all application logs would go to a separate container specifically for logging. Another reason is to run an initialization script before the application runs, all in the same pod. This is called an `initContainer`, and are containers that are designed to run to completion. They can do setup tasks like cloning a git repo, waiting for a DB to become ready, or validating secrets of config files. `initContainers` can also run in a sidecar pattern, when the `restartPolicy` is set to `Always`. The important distinction here is that in a sidecar pattern, the containers do not run to completion, but rather continue to run alongside the main application container.

Containers in the same pod also have the ability to share storage. So, you could have a volume attached to both pods, and stream that data to the volume simultaneously. This could be useful if one container were to output the logs to a file, then another container were to read them. In this case, the second container is sometimes referred to as a “sidecar container”. This is

because it's not acting as the main application, it's just assisting with the collection of logs.

Let's go ahead and see this in action by running the command to create a sidecar container `kubectl run sidecar --image=busybox --command "sh" "c" "while true; do cat /var/log/nginx/error.log; sleep 20; done" --dry-run=client -o yaml > sidecar.yaml`. Now, open the file "sidecar.yaml" in your favorite text editor. Change the "c" to a "-c" and add another container that will serve as the main container to the current sidecar container. In addition, to allow the sidecar container to read the log data, we'll go ahead and add a volume mount to an emptyDir volume type, as you can see in figure 4.14.

Figure 4.14 Adding an additional container to a pod, which will serve to read logs from the main application


```
spec:
  containers:
    - command:
      - sh
      - -c
      - while true; do cat /var/log/nginx/error.log; sleep 20; done
      image: busybox
      name: sidecar
      resources: {}
      volumeMounts:
        - name: log-vol
          mountPath: /var/log/nginx
    - name: nginx
      image: nginx
      volumeMounts:
        - name: log-vol
          mountPath: /var/log/nginx
  volumes:
    - name: log-vol
      emptyDir: {}
  dnsPolicy: ClusterFirst
```

change this to "-c"

volume mounted to the sidecar container at /var/log/nginx

volume mounted to the main container at /var/log/nginx

emptyDir volume is ephemeral (deleted along with pod)

An emptyDir volume is one of many types of volumes that live and die with the container. This is ephemeral storage, so as soon as the pod is deleted, the data is also deleted and gone forever.

Another use case for creating multiple containers in the same pod is to ensure the main application container is initialized. This is called an init container, being that its only job is to check on the main container or perform some task, and once it has done its job, it completed as opposed to continue running.

An init container can be added to any pod manifest, inline with the container's specification. To quickly create our starter YAML, let's run the command `kubectl run init --image=busybox:1.35 --command "sh" "c" "echo The app is running! && sleep 3600" --dry-run=client -o yaml > init.yaml`. Then, we'll enter “

initContainers:” inline with the word “container”. Everything else we'll mimic the existing container except for the command. We'll run the until loop until nslookup init-svc; do echo waiting for svc; sleep 2; done. The complete YAML should look similar to what you see in figure 4.15.

Figure 4.15 Init containers will not allow the main application to start before the service “init-svc” is created

```
spec:
  containers:
  - command:
    - sh
    - -c
    - echo The app is running! && sleep 3600
    image: busybox:1.35
    name: init
    resources: {}
  initContainers:
  - name: init-svc
    image: busybox:1.35
    command:
    - sh
    - -c
    - until nslookup init-svc; do echo waiting for svc; sleep 2; done
  dnsPolicy: ClusterFirst
```

change this to "-c"

wait for service named "init-svc" before initializing

You'll notice that specifying an init container is much like a regular container, but you'll notice the difference more clearly as you begin to create the pod. Upon creation, you see the pod go into a pending state. It will stay pending until you create the service named "init-svc". When you create the service with the command `kubectl create svc clusterip init-svc --tcp 80` the pod will change from "Init:0/1" to "running".

```
root@kind-control-plane:/# kubectl apply -f init.yaml
pod/init created
root@kind-control-plane:/# kubectl get po
NAME    READY    STATUS    RESTARTS    AGE
init    0/1      Init:0/1    0            7s
```

```

root@kind-control-plane:/# kubectl create svc clusterip init-svc
service/init-svc created
root@kind-control-plane:/# kubectl get po
NAME      READY   STATUS    RESTARTS   AGE
init      0/1     Init:0/1   0           79s
root@kind-control-plane:/# kubectl get po
NAME      READY   STATUS    RESTARTS   AGE
init      0/1     Init:0/1   0           4m17s
root@kind-control-plane:/# kubectl get svc
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP   PORT(S)    A
init-svc            ClusterIP     10.96.162.73   <none>        80/TCP     6
kubernetes          ClusterIP     10.96.0.1      <none>        443/TCP    1
root@kind-control-plane:/# kubectl get po
NAME      READY   STATUS    RESTARTS   AGE
init      1/1     Running   0           7m20s

```

Notice, the pod doesn't start running until approximately seven minutes later. So, if you're getting impatient, just wait a few more minutes for the pod to come back up.

4.2.3 ConfigMaps

So, now that we've talked about pods requesting CPU and memory, let's see what else pods can request by talking about a configMap in Kubernetes. A lot of times, you'll need to pass configuration data to your application, whether it be to change the log level or to change the background color of your web app. The most common way to do this is by using the configMap object in Kubernetes. Once you create the configMap object, you can inject it into a pod using various methods. You can inject it into a pod via an environment variable or via a volume mounted to the container. You can create a config map from literal (i.e. username and password), from a file (i.e. ca.crt), or from multiple files within a directory.

You can create a config map by using an imperative command or by creating YAML and checking it into your source code. Since we prefer the latter, let's create the YAML and save it to a file with the command `kubectl create configmap redis-config --from-literal=key1=config1 --from-literal=key2=config2 --dry-run=client -o yaml > redis-config.yaml`. This command will create a configmap named "redis-config", with literal key "key1" with value "config1" as well as literal key "key2" with value "config2", and we'll do a "dry run", which means to simulate

creating the resource, and finally output that to a file named “redis-config.yaml.” When we open this file in our text editor, we notice that the key and value pairs can be established line by line. Alternatively, we could perform multi-line arguments by specifying the key and then a pipe symbol (“|”), which we have done in previous chapters. Let’s replace the existing key/value pairs in our file in order to store the configMap data in a Redis cache for later use. To do this, delete the lines starting with “key1” and “key2”, and add a key named “redis-config” by adding it under “data” for a multi-line argument in our YAML file, similar to figure 4.16.

Figure 4.16 Creating a configMap in Kubernetes automatically base64 encodes the data within

```
apiVersion: v1
data:
  redis-config: |
    maxmemory 2mb
    maxmemory-policy allkeys-lru
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: redis-config
```

multi-line value for
the key "redis-config"

Now that we've created the YAML for our configMap, save and close the file. Create the configMap with the command `kubectl apply -f redis-config.yaml`.

Since the configMap is now available for our application to use, go ahead and create a YAML file for a redis pod with the command `kubectl run init --image=redis:7 --port 6379 --command 'redis-server' '/redis-master/redis.conf' --dry-run=client -o yaml > redis.yaml`. Let's add a volume mount to this pod for the Redis configuration data, as well as an emptyDir type volume for any ephemeral data. We can insert the volumes

and mount paths to the YAML spec, similar to figure 4.17.

Figure 4.17 Attaching configmap data to the container via configmap volume ensuring the volume name and the mountPath name are the same

```
spec:
  containers:
    - command:
      - redis-server
      - /redis-master/redis.conf
      image: redis:7
      name: redis
      ports:
        - containerPort: 6379
      volumeMounts:
        - mountPath: /redis-master-data
          name: data
        - mountPath: /redis-master
          name: config
      resources: {}
  volumes:
    - name: data
      emptyDir: {}
    - name: config
      configMap:
        name: redis-config
        items:
          - key: redis-config
            path: redis.conf
```

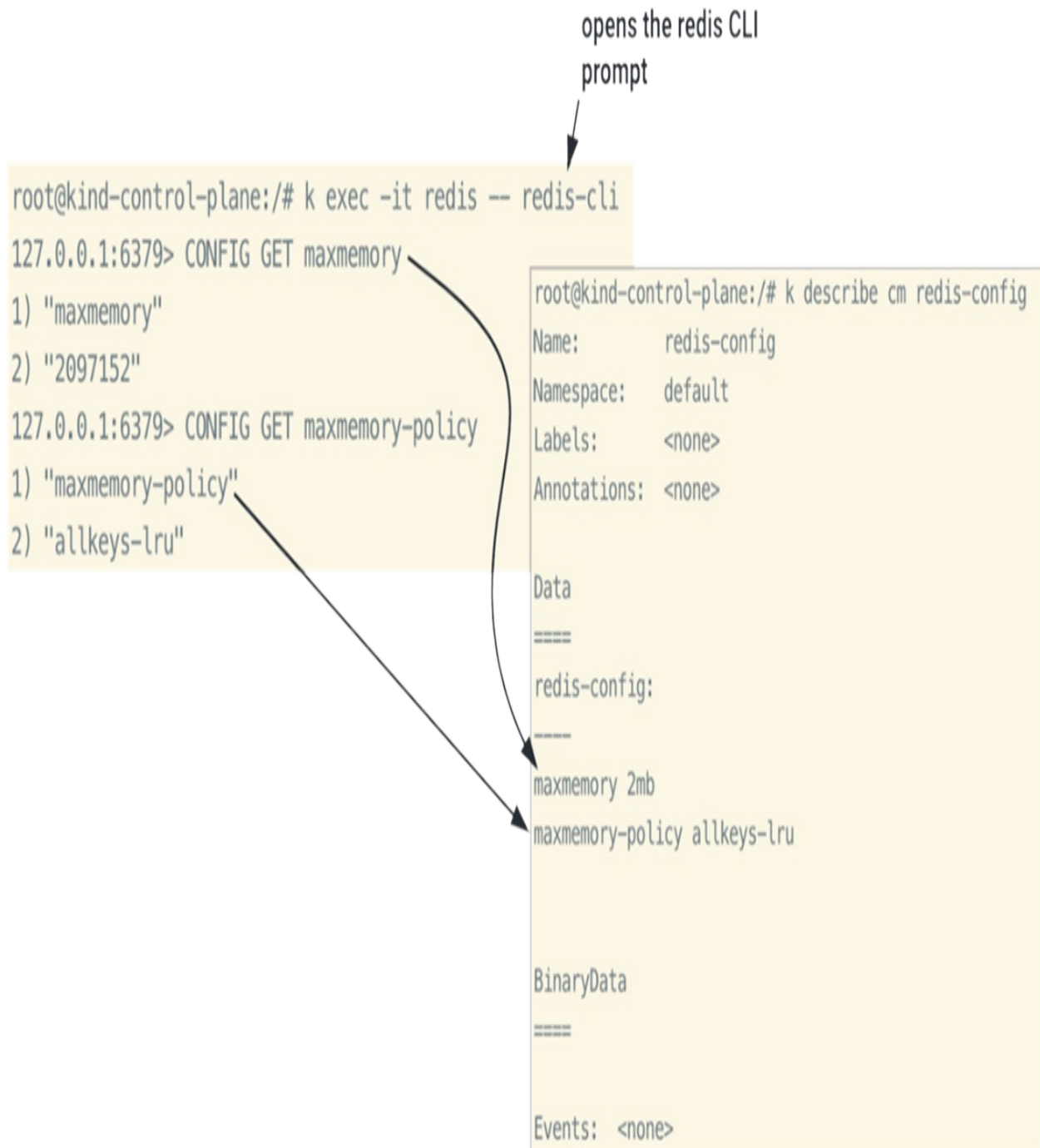
data in the container at /redis-master-data will reside in a emptyDir volume which gets deleted along with the pod

attach the config data from config map "redis-config" at key "redis-config" to path "/redis-master/redis.conf"

```
dnsPolicy: ClusterFirst
```


We'll go ahead and create that pod, and once it's running, we'll go ahead and get a shell inside of the container with the Redis CLI prompt with the command `kubectl exec -it redis -- redis-cli`. Once we've done that, we can issue the redis specific commands, which are `CONFIG GET maxmemory` and `CONFIG GET maxmemory-policy`, and you will get an output similar to figure 4.18.

Figure 4.18 We can view the configmap data via redis CLI, and relate that to the configmap original data



We can match these up directly with our config map data, and they match! This is a great way to insert configuration data into containers, and let that configuration data remain decoupled from the main application.

4.2.4 Secrets

Secrets are similar to config maps, but different in that they store secret data instead of application configuration data. Whether it be a database password or certificate data, you can store it in Kubernetes with a base64 encryption. However don't believe me if I ever said that it's secure, because it's not. Anyone who has access to your cluster can view and read the secret data, so never use this as your only method of security.

You can create a secret in the familiar way with the command `kubectl create secret generic dev-login --from-literal=username=dev --from-literal=password='S!B\*d$zDsb=' --dry-run=client -o yaml > dev-login.yaml`. Now open the file “dev-login.yaml” in your text editor and you'll notice that both the username and password are base64 encoded. Secrets values are encoded as base64 strings and are stored unencrypted by default, as show in figure 4.19.

Figure 4.19 All of the data inside of our secret named “dev-login” is automatically base64 encoded

```
apiVersion: v1
data:
  password: UyFCXCpkJHpEc2I9
  username: ZGV2
kind: Secret
metadata:
  creationTimestamp: null
  name: dev-login
```

The data inside of the secret has been base64 encoded

Now, go ahead and create the secret with the command `kubectl apply -f dev-login.yaml`. We can take this secret and make it available to a pod by mounting it to a directory inside of the container. Let's first create our pod YAML with the command `kubectl run secret-pod --image=busybox --command "sh" "c" "echo The app is running! && sleep 3600" --dry-run=client -o yaml > secret-pod.yaml`. Let's open the file "secret-pod.yaml" inside of a text editor and make some modifications. We'll add a volume mount to the container inside of the pod, and add a volume that's just a type of "secret", which Kubernetes knows what to do with. You can specify this in the pod YAML, as seen in figure 4.20.

Figure 4.20 Mount the secret data for the pod to access, ensuring the data can be reached from `/etc/secret-vol` inside the container

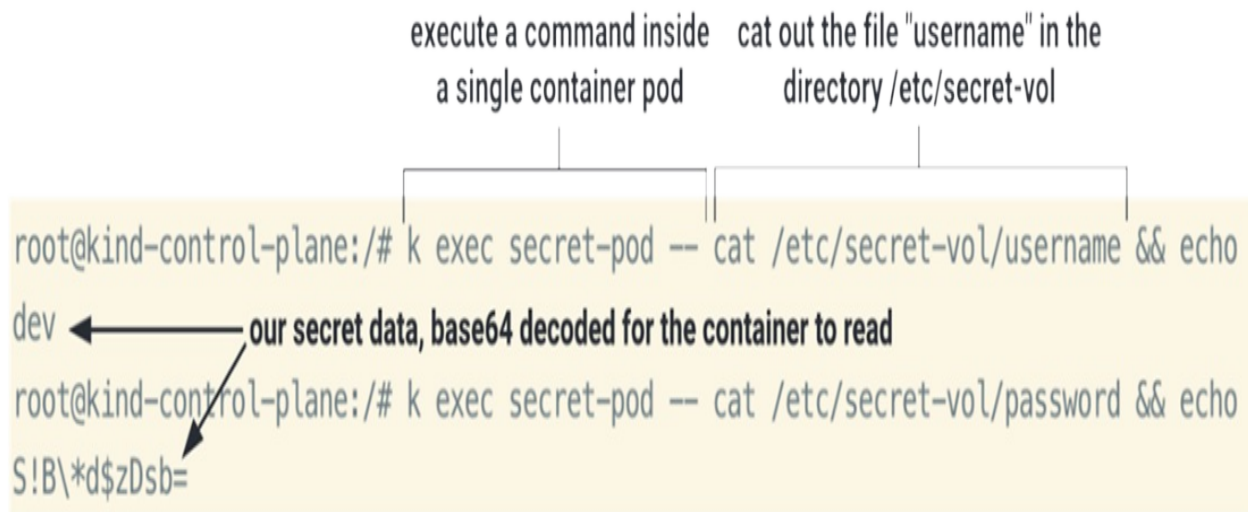
```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: secret-pod
  name: secret-pod
spec:
  containers:
  - command:
    - sh
    - -c
    - echo The app is running! && sleep 3600
    image: busybox
    name: secret-pod
    resources: {}
    volumeMounts:
    - name: secret-vol
      readOnly: true
      mountPath: "/etc/secret-vol"
  volumes:
  - name: secret-vol
    secret:
      secretName: dev-login
  dnsPolicy: ClusterFirst
```

**Mount the data to the container,
making it accessible from inside
the container**

**The volume type is "secret" in which
contains the "dev-login" secret we had
previously created**

At this point, we can view our secret data from within our pod by running the command `kubectl exec secret-pod -- cat /etc/secret-vol/username && echo` and `kubectl exec secret-pod -- cat /etc/secret-vol/password && echo` and you'll see output similar to figure 4.21.

Figure 4.21 Secret data can be obtained by running a command inside of the container



This is not the only way to give secret data to the container, we could also use the secret inside of the pod via environment variable. You can pass the “—env” flag after the “kubectl run” command with `kubectl run secret-env -image=busybox --command "sh" "c" "printenv DEV_USER DEV_PASS; sleep 8200" --env=DEV_PASS=password --dry-run=client -o yaml > secret-env.yaml`. When you open up the “secret-env.yaml” file, you can replace “value:” with “valueFrom:” and remove the value of “password”. Below “valueFrom:” you can indent two spaces and type “secretKeyRef:”. Below that you can indent two spaces and type “name:” followed by the name of the secret that we created earlier, which was “dev-login”. Then, in line with the name, you can type “key:” followed by “username”, as seen in figure 4.22. You can repeat this for however many secrets or secret values you have in Kubernetes. Also, multiple pods can reference the same secret, so we don’t have to delete the previous pod that we created (named “secret-pod”) in order to use the secret again inside of a new pod.

Figure 4.22 You can pass the secret data to a pod via environment variables. Set the command inside the container to print the environment variables (printenv). Don’t forget to change the “c”

to “-c”!!

spec:

containers:

- command:

**send the value of DEV_USER and
DEV_PASS to STDOUT**

- sh

- -C

- printenv DEV_USER DEV_PASS && sleep 8200

env:

- name: DEV_USER

valueFrom:

secretKeyRef:

name: dev-login

key: username

**Set DEV_USER environment variable
inside container to the value of the secret
"dev-login" at key "username"**

- name: DEV_PASS

valueFrom:

secretKeyRef:

name: dev-login

key: password

**Set DEV_USER environment variable
inside container to the value of the secret
"dev-login" at key "password"**

image: busybox

name: secret-env

Practice exercises

Create a pod named “limited” with the image “httpd” and set the resource requests to “1Gi” for CPU and “100Mi” for memory.

Create a deployment named “overloaded” running three replicas of pods. Inside the pod is an initContainer and a main container. For the initContainer, use the busybox image and keep it running by issuing the command “/bin/bash sleep 1d”. Set the CPU requests to 250 millicores and Memory requests to 600 mebibytes. For the main container, use the nginx image. Set the CPU requests to 250 millicores and Memory requests to 600 mebibytes. See if the pods within the deployment get to a running state. If not, adjust the requests to get all three replicas up and running.

Create a configmap named “ui-data” with the key and value pairs below. Apply a configmap to a pod named “frontend” with the image “busybox:1.28” and pass it to the pod via the following environment variables:

```
color.good=purple
color.bad=yellow
allow.textmode=true
how.nice.to.look=fairlyNice
```

4.3 Summary

- The term “Scheduling” refers to creating a pod and then assigning it to a node. You can change which node the pod is assigned to with a node selector, node name, or affinity rules. Make sure you know the correct YAML syntax for these scheduling changes.
- You can also apply labels to control pod scheduling. Like a node selector, you can use a label selector to schedule a pod to a specific node based on its label. For the exam, know how to apply labels to nodes and pods, to affect scheduling.
- Reserve CPU and memory for pods using resource requests and limits. This is important for the exam, as the YAML syntax for a pod or deployment is slightly harder than most.

- There are specific reasons to create more than one container inside of a pod. Sharing the same network namespace and storage allows for direct access and communication back and forth.
- Injecting configuration data and sensitive information into pods is simplified in Kubernetes using configmaps and secrets. Make sure you know how to use configmaps and secrets mounted as volumes to a pod as well as environment variables.

5 Running applications in Kubernetes

This chapter covers

- Scaling applications for high availability
- Rolling updates and rollbacks
- Exposing deployments to create services
- Performing maintenance tasks on a Kubernetes cluster
- Using Helm and Kustomize to install cluster components
- Installing Custom Resource Definitions (CRD) and operators

This chapter covers the operations side of Kubernetes, in that we will walk through how to maintain applications that are already running in Kubernetes. The previous chapter covered how to create pods, scheduling attributes, ConfigMaps and secrets. This chapter covers common approaches you will take on the exam to complete tasks such as providing additional resources for an application, scaling applications, providing a consistent endpoint for applications, and rolling out new versions of an application. For this chapter, we have a mix of two domains: Workloads & Scheduling and Cluster Architecture, Installation and Configuration. This chapter, in combination with the previous chapter, is focused on accomplishing high availability and self-healing for applications running in Kubernetes while customizing applications and extending the Kubernetes API.

The workloads and scheduling domain

This chapter covers part of the workloads and scheduling domain of the CKA curriculum. This domain covers the ways in which we run applications on Kubernetes. It encompasses the following competencies.

Competency	Chapter section
Configure workload autoscaling	5.1
Understand application deployments	5.1

and how to perform rolling updates and rollbacks	
Understand the primitives used to create robust, self-healing, application deployments	5.2

Cluster Architecture, Installation & Configuration

This chapter covers part of the cluster architecture, installation and configuration. This domain covers the ways in which we make Kubernetes highly available and a method to install cloud native applications. It encompasses the following competencies.

Competency	Chapter Section
Create and manage Kubernetes clusters using kubeadm	5.2
Use Helm and Kustomize to install cluster components	5.3
Understand CRDs, install and configure operators	5.4

5.1 Orchestrating applications

A deployment is a core object in Kubernetes, by which you achieve high-availability, robustness, and self-healing. Stateless applications - as the name implies – do not contain state (data), so the pods (and containers within) can be replaced and re-spawned without affecting the overall health of the application. Stateless applications in Kubernetes are commonly managed via Deployment. In contrast, the StatefulSet object in Kubernetes cannot be as easily replaced or re-spawn, as the data is required for the stateful application to run. Think of MySQL, where data is being written to a primary database table and replicated to additional read only database instances. You will not be tested on StatefulSets on the exam, so we will focus solely on Deployments.

5.1.1 Modifying running applications

So far in this book, we've paid special attention to pods, which is great, but a lot of times just one pod for your application is not sufficient. In Kubernetes, there's a way to create redundancy and fault tolerance with the creation of a Deployment. The CKA exam will test you on modifying existing deployments. This includes changing the image and changing the number of pod replicas. A task on the CKA exam may look something like this:

Exam Task

Create a deployment named "apache" that uses the image `httpd:2.4.54` and contains three pod replicas. After the deployment has been created, scale the deployment to five replicas and change the image to `httpd:alpine`.

If you don't already have access to an existing Kubernetes cluster, creating a Kubernetes cluster with `kind` is explained in appendix A. As soon as your `kind` cluster is built, use the `kubectl` tool pre-installed on the control plane node. You can get a bash shell to the control plane node by typing the command `docker exec -it kind-control-plane bash` and you can follow along.

In a stateless app in Kubernetes, creating replicas of the same application is not a problem. This is because all microservices data is decoupled from the container itself, therefore the requests get the same response no matter if they are going to the first replica of the application or the fifteenth. Let's go ahead and create a deployment named "apache" with the command `kubectl create deploy apache --image httpd:2.4.54 --replicas 3`. Now that we've created the deployment, we can check on the status with the command `kubectl get deploy` and we'll see that we have 3 replicas. Since we want to increase the number of replicas to 5, you would "scale the deployment" by running the command `kubectl scale deploy apache --replicas 5` which will scale the deployment from one pod to two pods.

```
root@kind-control-plane:/# kubectl create deploy apache --image httpd:2.4.54
deployment.apps/apache created
root@kind-control-plane:/# kubectl get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
apache        3/3     3            3           14s
root@kind-control-plane:/# kubectl scale deploy apache --replicas 5
deployment.apps/apache scaled
```

```
root@kind-control-plane:/# kubectl get po
```

NAME	READY	STATUS
apache-74f79bcc68-7lzqz	1/1	Running
apache-74f79bcc68-89zwg	1/1	Running
apache-74f79bcc68-v6drf	1/1	Running
apache-74f79bcc68-x7bfw	1/1	Running
apache-74f79bcc68-xdx4b	1/1	Running

You will notice that the pods are prefixed with the same name as the deployment followed by a suffix that is a hash value, the last five being unique since there cannot be duplicate pod names in a cluster.

Now that we have a new deployment running, let's change the image for the deployment from `httpd:2.4.54` to `httpd:alpine`. We can update the image by running the command `kubectl set image deploy apache httpd=httpd:alpine` and by performing the command `kubectl get po` you'll see that a whole new set of pods have been created for the deployment.

```
root@kind-control-plane:/# kubectl get po
```

NAME	READY	STATUS
apache-74f79bcc68-7lzqz	1/1	Running
apache-74f79bcc68-89zwg	1/1	Running
apache-74f79bcc68-v6drf	1/1	Running
apache-74f79bcc68-x7bfw	1/1	Running
apache-74f79bcc68-xdx4b	1/1	Running

```
root@kind-control-plane:/# kubectl set image deploy apache httpd=
deployment.apps/apache image updated
root@kind-control-plane:/# kubectl get po
```

NAME	READY	STATUS
apache-698b8cccbd-45nzs	1/1	Running
apache-698b8cccbd-bz4bm	1/1	Running
apache-698b8cccbd-dx4j2	1/1	Running
apache-698b8cccbd-jrz86	1/1	Running
apache-698b8cccbd-z8vnl	0/1	ContainerCreating
apache-74f79bcc68-89zwg	1/1	Terminating
apache-74f79bcc68-v6drf	1/1	Terminating
apache-74f79bcc68-x7bfw	1/1	Terminating
apache-74f79bcc68-xdx4b	1/1	Terminating

Modifying the deployment in this way means that there's no downtime for the application while the new version of the deployment (with the new image) gets rolled out. It also creates flexibility and increased agility to service your end users, accounting for spikes in load and being able to

incrementally change the application and add features with little downtime. A deployment resource is the most common in Kubernetes, as it takes full advantage of the native ecosystem, and we'll focus on it more in the coming sections of this chapter.

5.1.2 Application maintenance

Deployments are designed to run multiple instances of the same application, replicated to provide increased availability for the end users. As an abstraction of a deployment, which provides additional capabilities, is a ReplicaSet. As the name implies, a ReplicaSet's purpose is to maintain a stable set of replica pods running at any given time. This guarantees the availability of a specified number of identical pods. List the ReplicaSets in your cluster with the command `kubectl get rs`. You should get a similar output as the following:

```
root@kind-control-plane:/# kubectl get rs
```

```
NAME DESIRED CURRENT READY AGE
```

```
apache-74f79bcc68 1 1 1 28m
```

In figure 5.1, you will see the deployment and ReplicaSet depicted with three replicated pods running NGINX, as a part of both the deployment and ReplicaSet combination of Kubernetes objects.

Figure 5.1 A ReplicaSet inside of a Deployment with three replicas, indicating the naming scheme for Kubernetes objects



Deployment

name = nginx

ReplicaSet

name = nginx-85b98978db



Pod

name = nginx-85b98978db-wsr7p



Pod

name = nginx-85b98978db-krvnx



Pod

name = nginx-85b98978db-7kxrz

Speaking of the deployment configuration YAML, let's take look how the pod YAML is different from a deployment YAML. It's actually easier than you'd think. You basically take the pod YAML, and you put it in the section under "template:" under "spec:" in the deployment YAML. To prove this, let's perform two different commands, one to create a YAML file for a pod, and one to create a YAML file for a deployment. The command to create a pod YAML file is `kubectl run nginx --image nginx --dry-run=client -o yaml > pod.yaml` and the command to create a deployment is `kubectl create deploy nginx --image nginx --dry-run=client -o yaml > deploy.yaml`. You'll see the similarities between the pod.yaml file and the deploy.yaml file in figure 5.2.

Figure 5.2 A comparison between the YAML for a pod and the YAML for a deployment.

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
```

POD

```
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
```

```
  dnsPolicy: ClusterFirst
  restartPolicy: Always
```

SAME



```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: nginx
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: nginx
```

DEPLOYMENT

```
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
```

The deployment spec, created via YAML, is exactly like the pod spec, with the additional deployment spec as a parent resource above it. So, there's a spec for the deployment and within the spec for the deployment, there's the spec for the pod. Pretty easy, right?

We can use the deployment that we created in the exercise at the beginning of this chapter, or we can create the deployment from the file we just created with the command `kubectl create -f deploy.yaml`. We can view the deployments status with the command `kubectl get deploy`. We'll see the number of replicas in the "READY" column. We will see five replicas for the deployment named "apache" and since we didn't specify any replicas in the "kubectl create" command, we only have one for the deployment named "nginx". You can list the ReplicaSets again with the command `kubectl get rs`.

```
root@kind-control-plane:/# kubectl create deploy nginx --image ng
root@kind-control-plane:/# kubectl create -f deploy.yaml
deployment.apps/nginx created
root@kind-control-plane:/# kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
apache	5/5	5	5	97m
nginx	1/1	1	1	6s

```
root@kind-control-plane:/# kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE
apache-698b8cccbd	5	5	5	74m
apache-74f79bcc68	0	0	0	101m
nginx-76d6c9b8c	1	1	1	4m38s

You will notice from the output that there are two ReplicaSets that start with "apache". This is because earlier when we changed the image, the deployment controller created a new ReplicaSet, as it contains a different configuration. By default, all of the Deployment's rollout history is kept in the system so that you can roll back any time you want. We talk about rollouts and rollbacks later in this chapter. You could scale this deployment as we did at the beginning of this chapter by modifying the "deploy.yaml" file, or you could type the command `kubectl scale deploy nginx --replicas 2`. When modifying the YAML, you can open the file in your text editor and change the line that says "replicas: 1" to "replicas: 2", as you see

in a snapshot of the YAML file in figure 5.3. After you save the file, you can apply the same YAML file with the command `kubectl apply -f deploy.yaml`. You may get a warning that the resource is missing a “last-applied configuration”, but you can safely ignore the warning, as the deployment is still scaled to two replicas. The last applied configuration is added as an annotation to an object upon creation. The annotation contains the contents of the object configuration file that was used to create the object. The warning indicates that this annotation is missing, therefore, the new configuration file will be merged with the current.

We use `apply` here, as opposed to `create`, because a deployment already exists. The `apply` command overwrites the existing object, or if the resource doesn’t exist, it will create it. The `create` command will only create the object if one doesn’t already exist, but will return an error if the object exists.

Figure 5.3 Scale the deployment by changing the replica count in the deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: nginx
  name: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
```

**Scale the deployment
from 1 pod to 2 pods**



The deployment was scaled from one pod to two pods, and you can view the replica change with the command `kubectl get deploy nginx -o yaml`.

Behind the scenes, the ReplicaSet that's associated with that deployment was also modified. We can see this in action by looking at the events inside of the replica set by running the command `kubectl describe rs` and you'll see an output similar to figure 5.4.

Figure 5.4 Describe the ReplicaSet to get more info on the changes in it's configuration

```

root@kind-control-plane:/# k describe rs
Name:          nginx-85b98978db
Namespace:     default
Selector:      app=nginx,pod-template-hash=85b98978db
Labels:        app=nginx
               pod-template-hash=85b98978db
Annotations:   deployment.kubernetes.io/desired-replicas: 2
               deployment.kubernetes.io/max-replicas: 3
               deployment.kubernetes.io/revision: 1
Controlled By: Deployment/nginx
Replicas:      2 current / 2 desired
Pods Status:   2 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=nginx
          pod-template-hash=85b98978db
  Containers:
    nginx:
      Image:      nginx
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:      <none>
      Volumes:      <none>
Events:
  Type     Reason             Age   From                    Message
  ----     -
  Normal   SuccessfulCreate    22s   replicaset-controller   Created pod: nginx-85b98978db-mbqj6
  Normal   SuccessfulCreate    4s    replicaset-controller   Created pod: nginx-85b98978db-vpt52

```

Get more detail about the replica set object

The replica set was changed when scaling the deployment

The replica set gets created along with the deployment. If we delete the deployment, we can see that the replica set is deleted as well. Delete the deployment with the command `kubectl delete deploy nginx`

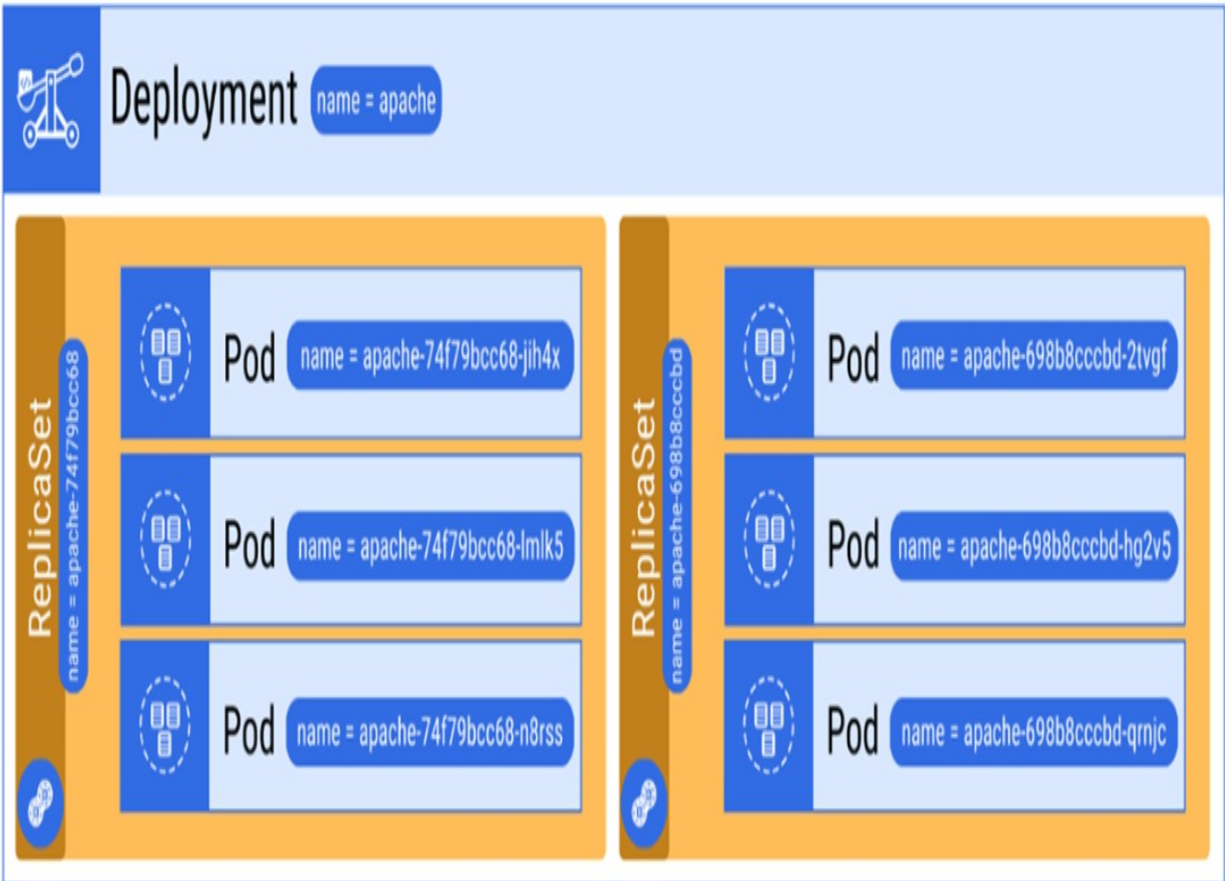
```
root@kind-control-plane:/# kubectl delete deploy nginx
deployment.apps "nginx" deleted
root@kind-control-plane:/# kubectl get deploy
No resources found in default namespace.
root@kind-control-plane:/# kubectl get rs
No resources found in default namespace.
```

EXAM TIP

If you want to describe everything (i.e. all replicaSets) all at once, instead of listing the names one-by-one in the command line, you can just type the name of the resource (e.g. `kubectl describe rs`) and the output will include all resources that currently exist.

So, we've seen what happens to a replica set if we modify the deployment replica count. If we modify the deployment image, it creates a brand new ReplicaSet, as we saw earlier. The ReplicaSet contained a new set of pods, with new names and a new configuration, as you see depicted in figure 5.5.

Figure 5.5 When the Deployment image is changed, a new ReplicaSet is created with a new set of pod replicas



If you're fast enough, you can see the old pods being terminated and the new pods being created. You can run the command `kubectl get po` no later than 3 seconds after running the `set image` command. After you do so, you'll see an output similar to figure 5.6. You could also run the command `kubectl get po -w` to watch the pods change status in real-time.

Figure 5.6 Changing the image for pods in the deployment results in a new ReplicaSet

change image for pods
running in deployment
named "apache"

change pod named
"httpd" to image
"httpd:alpine"

```
root@kind-control-plane:/# k set image deploy apache httpd=httpd:alpine
deployment.apps/apache image updated
```

```
root@kind-control-plane:/# k get po
```

NAME	READY	STATUS	RESTARTS	AGE
apache-698b8ccbd-qrnjc	0/1	ContainerCreating	0	1s
apache-698b8ccbd-rssvv	0/1	ContainerCreating	0	1s
apache-698b8ccbd-s4rwf	0/1	ContainerCreating	0	0s
apache-74f79bcc68-jlh4x	1/1	Terminating	0	
apache-74f79bcc68-lmlk5	1/1	Running	0	
apache-74f79bcc68-n9rss	1/1	Running	0	22s
apache-74f79bcc68-wrdvl	1/1	Running	0	22s
apache-74f79bcc68-wvbrn	1/1	Running	0	15s

new set of pods are created as a
part of the new replica set

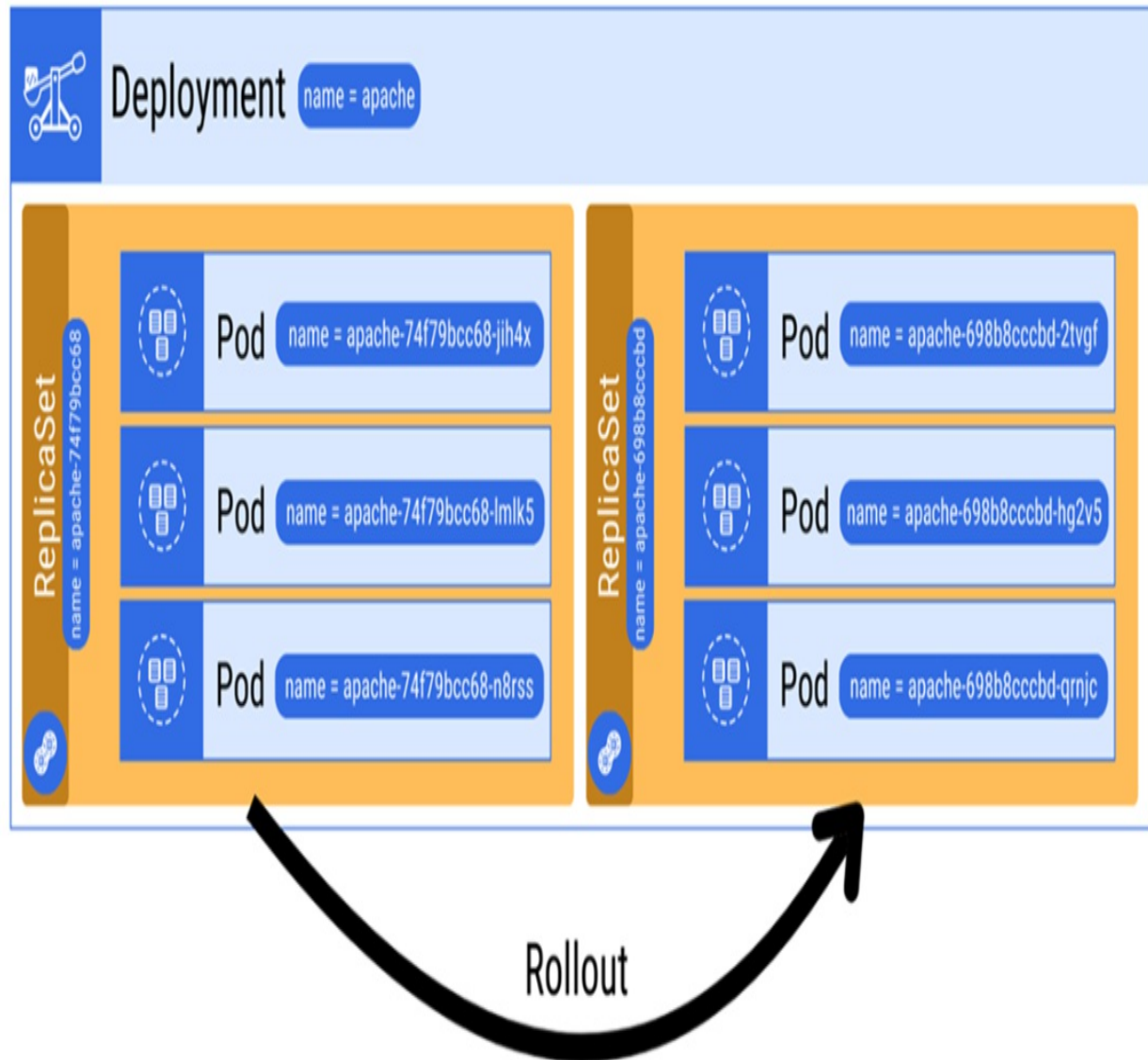
```
root@kind-control-plane:/# k get po
```

NAME	READY	STATUS	RESTARTS	AGE
apache-698b8ccbd-2tvvf	1/1	Running	0	2s
apache-698b8ccbd-hg2v5	0/1	ContainerCreating	0	2s
apache-698b8ccbd-qrnjc	1/1	Running		
apache-698b8ccbd-rssvv	1/1	Running		
apache-698b8ccbd-s4rwf	1/1	Running	0	3s
apache-74f79bcc68-jlh4x	1/1	Terminating	0	18s
apache-74f79bcc68-lmlk5	1/1	Terminating	0	25s
apache-74f79bcc68-n9rss	1/1	Terminating	0	25s
apache-74f79bcc68-wrdvl	1/1	Terminating	0	25s
apache-74f79bcc68-wvbrn	1/1	Terminating	0	18s

pods from the old replica set are terminated, but not
before the new pods are all up and running

You'll find that there's a lot of interesting information in the output of the `kubectl get po` command, including that the name of the pod actually comes from the name of the replica set. Also, the pods from the old replica set won't terminate until the new replica set pods are all up and running. This is called a *rollout strategy*, which ensures that the application can still operate, even in an upgrade scenario. This prevents downtime of the app, and ensures that no connections are severed if someone is trying to access the application from outside of the cluster, as you see depicted in figure 5.7.

Figure 5.7 Rollout strategy in Kubernetes that states only a certain number of replicated pods can be unavailable when rolling a new version out



This is the benefit of stateless apps that contain multiple replicas of the same application. A rollout strategy is defined in the deployment spec, which you can locate by running the command `kubectl edit deploy apache`. You'll see output similar to figure 5.8. If no strategy is defined when the deployment is created, the default will be applied, which is called a "rolling update". A rolling update is what we just described, where the pods from an old replicaSet don't terminate until the new replica set pods are ready. The other type of strategy is called a "recreate" strategy. A recreate strategy will terminate the old replica set pods before the new replicaSet pods are running. This strategy requires a period of downtime for the application.

Figure 5.8 The rollout strategy is set to “rollingUpdate” with a max surge of 25% and a max unavailable of 25%

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "2"
  creationTimestamp: "2023-05-31T16:54:32Z"
  generation: 3
  labels:
    app: apache
  name: apache
  namespace: default
  resourceVersion: "858"
  uid: e46c0aa7-5fc9-4326-b7ec-966c41217ed2
spec:
  progressDeadlineSeconds: 600
  replicas: 5
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: apache
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
```

RollingUpdate rollout strategy for new rollouts or rollbacks. At most 25% of pods will be added. At most 25% of pods can be unavailable

If you wanted to change the strategy to recreate, you would change the type from “RollingUpdate” to “Recreate” and delete the lines that start with “rollingUpdate”, “maxSurge” and “maxUnavailable”. A recreate strategy is good for many deployments, because it offers the fastest rollout strategy, but as we discussed, there will be downtime.

EXAM TIP

If you find yourself wondering what values go where in a YAML file, during the exam you can use `kubectl explain`. For example, the command `kubectl explain deploy.spec.strategy` will list values that are available for input in the spec field. Try it out!

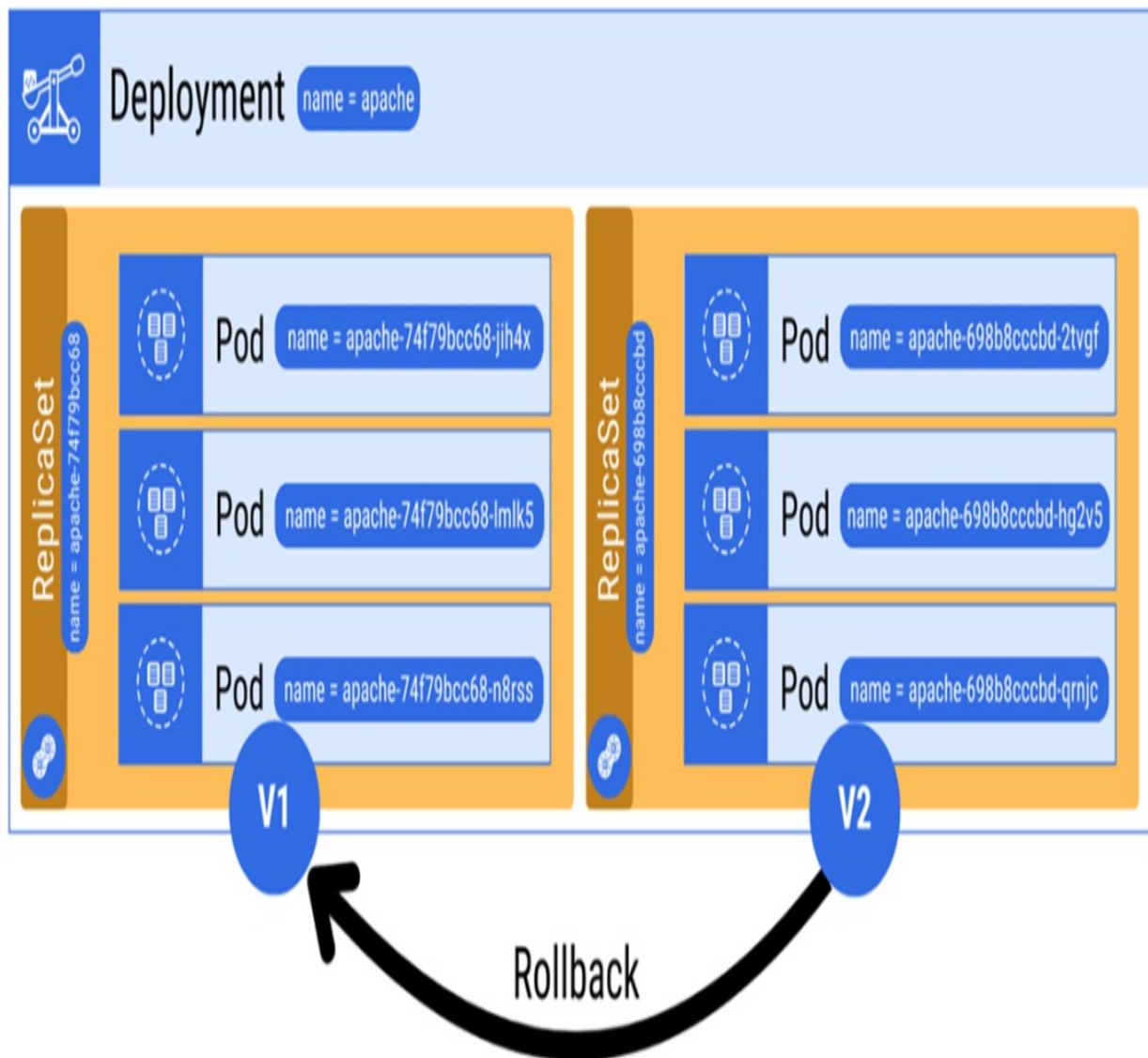
In a rolling update rollout strategy, the max surge and max unavailable fields are optional and can be a percentage or whole number. The max surge specifies the maximum number of pods that can be created over the amount set by the replicas in the deployment. For example, in figure 5.8, since the replica count is 3, and the “maxSurge” is 25%, the pods cannot surge above 4 pods (percentages round up for a surge). The max unavailable is just as it sounds, the number of pods that can be unavailable for the deployment. In our example, since “maxUnavailable” is set to 25%, no more than one pod can ever be unavailable at once.

Wait, there’s more! In addition to a replicaSet, there’s a tracking capability for deployments called rollouts.

5.1.3 Application rollouts

Application rollouts are a way for a Kubernetes administrator or developer to rollout to a new version of your application. Through the rollout process via `kubectl rollout`, you can also roll back to a previous version, being a recorded number of a revision that the deployment has, as you see depicted in figure 5.9.

Figure 5.9 There remains two rollout revisions, in the event you need to rollback to a previous version of the application



Every time pods are changed within a deployment, a new revision is created. We can see the rollout history by typing the command `kubectl rollout history deploy apache`

```
root@kind-control-plane:/# kubectl rollout history deploy apache
deployment.apps/apache
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

In this case, there are two revisions, one for the initial deployment (when we created the deployment with the `kubectl create deploy` command), then a

second when we changed the image tag from alpine to 2.

If you take a look at the “CHANGE-CAUSE” column, it says none. The change cause column is an opportunity to record a note along with your deployment change. So, let’s say you wanted to add a note in the change cause column that says “updated image tag from alpine to 2”. You can apply this to revision 2 by typing the command `kubectl annotate deploy apache kubernetes.io/change-cause="updated image tag from alpine to 2"`

```
root@kind-control-plane:/# kubectl annotate deploy apache kubernetes.io/change-cause="updated image tag from alpine to 2"
deployment.apps/apache annotated
root@kind-control-plane:/# kubectl rollout history deploy apache
deployment.apps/apache
REVISION  CHANGE-CAUSE
1          <none>
2          updated image tag from alpine to 2
```

As you can see, the change cause column for revision 2 has been changed. Now you can tell why the deployment was revised, and what changes occurred from revision 1 to revision 2.

Having a revision history like this allows you to revert to a previous revision. This is called a *rollback*. Let’s say that, when you deployed the new version of your application and changed the image tag for your deployment, you introduced a bug in your application. You must quickly revert to prevent users of your application from experiencing that bug. You can roll back to the previous revision with the command `kubectl rollout undo deploy apache`

```
root@kind-control-plane:/# kubectl rollout undo deploy apache
deployment.apps/apache rolled back
root@kind-control-plane:/# kubectl rollout history deploy apache
deployment.apps/apache
REVISION  CHANGE-CAUSE
2          updated image tag from alpine to 2
3          <none>
```

Now, you’ve reverted back to the previous revision, which in turn creates a new revision. The revision number will never go backwards, it will always go up. You can now enter a message into the change cause column for revision 3 with the command `kubectl annotate deploy apache kubernetes.io/change-cause="reverted back to the original"`

```
root@kind-control-plane:/# kubectl annotate deploy apache kubernetes.deployment.apps/apache annotated
root@kind-control-plane:/# kubectl rollout history deploy apache deployment.apps/apache
REVISION  CHANGE-CAUSE
2          updated image tag from apline to 2
3          reverted back to the original
```

Now both of the revisions are recorded with notes. The reason why you don't see revision 1 anymore is because reverting back to revision 1 would be the same as reverting back to revision 3, so to save duplication, revision 1 is removed.

Let's say that you wanted to check the status of the rollout. Maybe the rollout didn't go so well. This can happen if the image tag is not available. Maybe we typed it incorrectly or the image tag is no longer available from our image registry. We can check the status of our rollout based on the revision with the command `kubectl rollout status deploy apache --revision 3`

```
root@kind-control-plane:/# kubectl rollout status deploy apache -
deployment "apache" successfully rolled out
```

Our deployment was successfully rolled out when we reverted back to revision 1, so that's good. No misspellings or unavailable images for us! Phew!

The last thing I wanted to show you with rollouts is the ability to pause the rollout, so that only some of the pods in the deployment are on revision 3 and half of the pods are on revision 4. We can see this in action with the commands (NOTE: you have to perform these two commands within one second of each other) `kubectl set image deploy apache httpd=httpd:2.4` and `kubectl rollout pause deploy apache`. When you get the status of the rollout, you'll see a message that says "Waiting for deployment "apache" rollout to finish: 2 out of 3 new replicas have been updated..." Which means that the rollout is paused.

```
root@kind-control-plane:/# kubectl set image deploy apache httpd=
deployment.apps/apache image updated
root@kind-control-plane:/# kubectl rollout pause deploy apache deployment.apps/apache paused
root@kind-control-plane:/# kubectl rollout status deploy apache
```

Waiting for deployment "apache" rollout to finish: 2 out of 3 new

There are now 2 out of 3 new replicas running as a part of the new revision. Let's go ahead and resume (unpause) the deployment, so that the revision can finish with the command `kubectl rollout resume deploy apache`

```
root@kind-control-plane:/# kubectl rollout resume deploy apache
deployment.apps/apache resumed
root@kind-control-plane:/# kubectl rollout status deploy apache
deployment "apache" successfully rolled out
```

Now we have successfully rolled out to revision 4, where the new image for the pods in the deployment is set to "httpd:2.4".

EXAM TIP

If you're unsure of the order of the command or need examples of common ones, utilize the help menu in the terminal with the command `kubectl rollout --help` and `kubectl rollout status --help` and you can easily obtain the correct command syntax

5.1.4 Exposing deployments

The act of exposing deployments means that the deployment can be accessed from the end user, potentially from outside the cluster (depending on the type of service you create). We are going to go into depth about services in chapter 6, but I wanted to just mention that there's an easy way to create a service that is "connected" to a deployment. This means instead of creating a service from scratch, you can use the command `kubectl expose deploy nginx --name nginx-svc --port 80 --type ClusterIP --dry-run=client -o yaml > nginx-svc.yaml`. You'll see a snapshot of the YAML that this command creates in figure 5.10. Of course, since we like to do things declaratively in Kubernetes, we'll do a dry run and create a file named "nginx-svc.yaml".

Figure 5.10 Selecting a deployment via service in Kubernetes

```
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: nginx
    name: nginx-svc
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
  selector:
    app: nginx
  type: ClusterIP
status:
  loadBalancer: {}
```

Use the selector field to attach to deployments with the label "app=nginx"

The important part to notice in reference to the deployment is the selector. If you remember we talked about node selectors and label selectors in the last chapter, and here we're talking about *service selectors* to associate a service to the pods within a deployment. This particular service will associate itself with any deployment that has the label "app=nginx". This is useful information to know as we proceed through this book, and will come in handy as foundational knowledge for the next chapter where we talk about services in-depth.

Practice Exercises

Using `kubectl`, create a deployment named "apache" using the image "httpd:latest" with one replica. After the deployment is running, scale the replicas up to 5.

Update the image for the deployment "apache" from "httpd:latest" to "httpd:2.4.63". Do not create a new YAML file or edit the existing resource (only use `kubectl`).

Using `kubectl`, view the events of the ReplicaSet that was created as a result of the image change from the previous exercise.

Using `kubectl`, rollback to the previous version of the deployment named "apache"

For the existing deployment named "apache", change the rollout strategy for a deployment to "Recreate".

5.2 Node maintenance

In the lifecycle of your application running in Kubernetes, there may come a time when the underlying nodes require maintenance, or the application requires additional resources in order to optimize performance. This is the robustness of deployments, as they are able to move pods around the cluster from node to node, facing no downtime. With redundancy and high availability built in to the Kubernetes ecosystem, we'll learn how to safely

perform maintenance in order to create robust application deployments.

For the exam, you will need to know how to move a pod to a different node. An exam task will look something like this:

Exam Task

The node named “kind-worker” in the cluster is experiencing problems with leaking memory. You must take down the node for maintenance by first disabling the scheduling of new pods to the node “kind-worker”. Then, evict all pods that are currently running on “kind-worker”. Finally, once you’ve verified that there are no pods running on “kind-worker”, enable scheduling once again.

For this exam task, we’ll have to use a two-node kind cluster, and we’ll need to offload a pod to a secondary node, so we’ll need at least one running pod in the cluster as well. If you do not have access to a two-node cluster, I recommend you create one, as explained in Appendix A. Once you’ve opened a shell to the control plane node using the command `docker exec -it kind-control-plane bash`, we can run the command `kubectl create deploy nginx -image nginx` to get a pod running in the cluster. Now, we can begin the exam task by disabling scheduling for the kind-worker node.

To disable scheduling to a node you can use the command `kubectl cordon no kind-worker`. You will get the output similar to this:

```
root@kind-control-plane:/# kubectl cordon kind-worker
node/kind-worker cordoned
root@kind-control-plane:/# kubectl get no
```

NAME	STATUS	ROLES	A
kind-control-plane	Ready	control-plane	7
kind-worker	Ready,SchedulingDisabled	<none>	6

If you run the command `kubectl get no` right after, you’ll notice that the status changes from “Ready” to “Ready,SchedulingDisabled”. This means that if I try to create a second pod, the pod will remain in a pending state because the scheduler has marked the node as unavailable. We can see this in action with the command `kubectl create deploy nginx2 -image nginx` followed by the `kubectl get po` command to see the status. The output

should look like this:

```
root@kind-control-plane:/# kubectl create deploy nginx2 --image n
deployment.apps/nginx2 created
root@kind-control-plane:/# kubectl get po
NAME                                READY   STATUS    RESTARTS   AGE
nginx-76d6c9b8c-hr9z6              1/1     Running   0           111s
nginx2-b648d744f-n6xb9             0/1     Pending   0           3s
```

Now, we need to move all running pods on that node in order to take it down for maintenance. This process is called “draining the node”, as in evicting the pods and moving them to another node. In order to do this in our two-node kind cluster, we must first remove the taint applied to the control plane node that prevents pods from being scheduled to it in order for the pods to move from the worker node (when we drain it) to the control plan node. We talked about taints in chapter 2, so this should be a familiar concept. Perform the command `kubectl taint no kind-control-plane node-role.kubernetes.io/control-plane:NoSchedule-` to untaint the control plane node. The output will look like this:

```
root@kind-control-plane:/# kubectl taint no kind-control-plane no
node/kind-control-plane untainted
```

Now that the control plane node is untainted, we can drain the node “kind-worker” with the command `kubectl drain kind-worker --ignore-daemonsets`. The output will look similar to this:

```
root@kind-control-plane:/# kubectl drain kind-worker --ignore-dae
node/kind-worker already cordoned
Warning: ignoring DaemonSet-managed Pods: kube-system/kindnet-h76
evicting pod default/nginx-76d6c9b8c-hr9z6
pod/nginx-76d6c9b8c-hr9z6 evicted
node/kind-worker drained
root@kind-control-plane:/# kubectl get po -o wide
NAME                                READY   STATUS    RESTARTS   AGE    IP
nginx-76d6c9b8c-4494r              1/1     Running   0           52s    10.2
nginx2-b648d744f-n6xb9             1/1     Running   0           3m6s   10.2
```

You’ll see by running the command `kubectl get po -o wide` again, now the pods are running on the control plane node. To complete our exam task, let’s enable scheduling for the node once again by running the command `kubectl uncordon kind-worker`. The output will look like this:

```
root@kind-control-plane:/# kubectl uncordon kind-worker
node/kind-worker uncordoned
```

This completes the exam task. You should now be familiar with how to move pods from one node to another using the `cordon`, `drain` and `uncordon` options with `kubectl`.

5.2.1 Cordon and drain nodes

Cordon is an interesting word, isn't it? The official meaning is to form a barrier around, meaning a protective barrier. In the context of Kubernetes, the act of cordoning takes place on a node, and is marking the node as unschedulable. If you cordon a node, this puts the node out of service, and routinely ready for maintenance, as you see in figure 5.11. You may be wondering what kind of maintenance. Well, you may need to upgrade the RAM on a node, patch a security vulnerability, or to replace it entirely.

Figure 5.11 Cordon a node to disable scheduling

```
root@kind-control-plane:/# k cordon kind-worker
node/kind-worker cordoned
root@kind-control-plane:/# k get no
```

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane,master	31h	v1.23.4
kind-worker	Ready,SchedulingDisabled	<none>	31h	v1.23.4

mark the node named
"kind-worker" as unschedulable

node marked "SchedulingDisabled"
and no pods can be scheduled

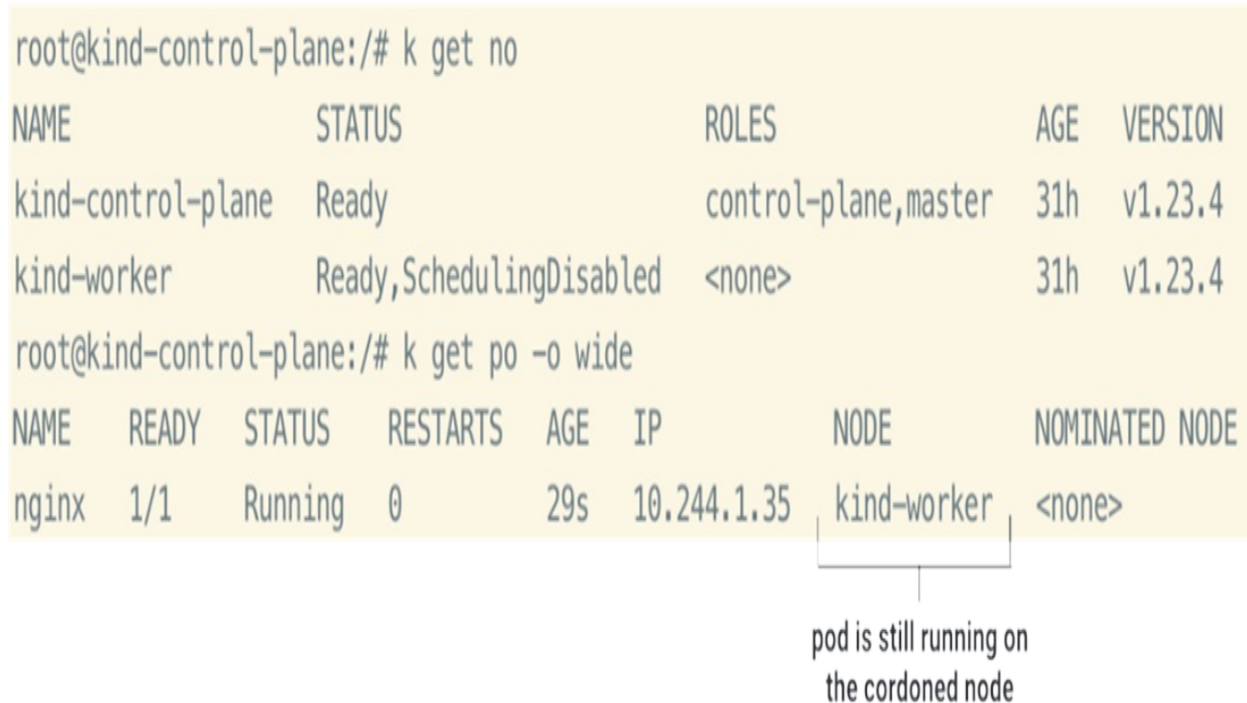
As you see, when you perform the command `kubectl cordon kind-worker`, you disable scheduling to the node, therefore, any pods that would normally tolerate this node for scheduling, can't until the cordon is released.

EXAM TIP

After you cordon a node, make sure you uncordon it. If the node has scheduling disabled, you may get points deducted from your final score.

So, that begs the question, what about existing pods running on that node? You have marked the node as unschedulable, but that just applies for scheduling from that point onward. It does not take into account pods that are currently running on that node, as you see in figure 5.12. You may have guessed it from the title of this section, but the act of moving the pods off of the node and scheduling them elsewhere is called *draining*.

Figure 5.12 Cordoning the node doesn't mean it will evict a pod from a node



```
root@kind-control-plane:/# k get no
```

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane,master	31h	v1.23.4
kind-worker	Ready,SchedulingDisabled	<none>	31h	v1.23.4


```
root@kind-control-plane:/# k get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
nginx	1/1	Running	0	29s	10.244.1.35	kind-worker	<none>

pod is still running on the cordoned node

The act of draining is just how it sounds, it is draining the node of all pods that are currently running on it. If the pod(s) are managed by a ReplicaSet, the same rules of scheduling still apply, and for that matter, the pod doesn't

move, it actually is deleted and recreated on another node, it just happens in a way that ensures the old pod is not removed before the new pod is running, much like a rollout. If you don't already have a deployment running, you can start one with the command `kubectl create deploy nginx --image nginx`, and you too can follow along in the fun. In the spirit of fun, let's see what happens to all the pods that are currently running on that node when its drained (pods managed by replicates and not). To drain the node and ignore the kube-proxy and kubenet daemonsets that are running on every node in the cluster, perform the command `kubectl drain kind-worker --ignore-daemonsets --force`. Force is necessary in this case, because one of the pods is not managed by a replica set. You will see an output similar to figure 5.13.

Figure 5.13 Draining the node removes all pods running on the specified node, where some are deleted permanently

```
root@kind-control-plane:/# k get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
nginx	1/1	Running	0	35s	10.244.1.2	kind-worker	<none>	<none>
nginx-85b98978db-kxxfc	1/1	Running	0	28s	10.244.1.3	kind-worker	<none>	<none>

```
root@kind-control-plane:/# k drain kind-worker --ignore-daemonsets --force
node/kind-worker cordoned drain node "kind-worker" ignore kube-system pods
WARNING: deleting Pods not managed by ReplicationController, ReplicaSet, Job, DaemonSet or StatefulSet: default/nginx; ignoring DaemonSet-managed Pods: kube-system/kindnet-dfxxp, kube-system/kube-proxy-nq72h
evicting pod default/nginx-85b98978db-kxxfc
evicting pod default/nginx
pod/nginx-85b98978db-kxxfc evicted
pod/nginx evicted ← pod was deleted, as it wasn't managed by a replicaSet
node/kind-worker drained
root@kind-control-plane:/# k get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
nginx-85b98978db-fnnb6	0/1	Pending	0	4s	<none>	<none>	<none>	<none>

As you can see, a couple of actions occur. The node is drained and the pods are moved off, but one of the pods is deleted and does not return. This is

because it wasn't managed by a replicaSet, so there's no mechanism to reschedule that pod. Secondly, the pod that was a part of a replicaSet is in a pending state. As we learned in the last chapter, the control plane nodes have a taint applied to them that requires a toleration in order to successfully schedule pods to them. In this case, since the pod doesn't have a toleration for that taint, it will remain in the pending state until either a toleration is added to the pod, or a new node without a taint is added to the cluster. For simplicity's sake, let's go ahead and remove the taint with the command `kubectl taint no kind-control-plane node-role.kubernetes.io/master-`

```
root@kind-control-plane:/# kubectl taint no kind-control-plane no
node/kind-control-plane untainted
root@kind-control-plane:/# kubectl get po -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP
nginx-85b98978db-fnnb6             1/1     Running   0          20m   10.24
```

As you can see by running the command `kubectl get po -o wide`, the container is now running on the kind-control-plane node. We'll get to joining a node to a cluster later in this chapter, in which we can simulate another scenario where the pod may move out of a pending state without potentially putting the control plane node at risk (by limited resources). For now, you can uncordon the "kind-worker" node with the command `kubectl uncordon kind-worker`. Additionally, if you wanted to, you could also reapply the taint to the control plane node with the command `kubectl taint no kind-control-plane node-role.kubernetes.io/master:NoSchedule`

5.2.2 Adding nodes to the cluster

There may be case within the lifetime of your Kubernetes cluster that you may need to add additional nodes. Whether you need more resources for your application, or you've lost a node due to failure or planned downtime. No matter the reason, adding a node to an existing kubeadm cluster is simplified by enabling bootstrap token authentication in the Kubernetes API. An exam question might say something like this:

EXAM TASK

There's a third node in the cluster named "ik8s", but it is not appearing when

you perform the `kubectl get nodes` command. The name of the node is `node02`. Allow `node02` to join the cluster by re-creating the join command and ensure that the node is in a “Ready” state when you list all the nodes in the cluster.

When you first create a Kubernetes cluster, `kubeadm` creates an initial bootstrap token with a 24-hour TTL, but you can create additional tokens on-demand. You can see the bootstrap token mechanism enabled in our kind kubernetes cluster, by opening the file `/etc/kubernetes/manifests/kube-apiserver.yaml` and looking under “command”. You’ll see an output similar to figure 5.14.

```
root@kind-control-plane:/# cd /etc/kubernetes/manifests/  
root@kind-control-plane:/etc/kubernetes/manifests# ls | grep apis  
kube-apiserver.yaml
```

Figure 5.14 Enable bootstrap token authentication in Kubernetes API

```
apiVersion: v1
kind: Pod /etc/kubernetes/manifests/kube-apiserver.yaml
metadata:
  annotations:
    kubeadm.kubernetes.io/kube-apiserver.advertise-ad
  creationTimestamp: null
  labels:
    component: kube-apiserver
    tier: control-plane
  name: kube-apiserver
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-apiserver
    - --advertise-address=172.18.0.2
    - --allow-privileged=true
    - --authorization-mode=Node,RBAC
    - --client-ca-file=/etc/kubernetes/pki/ca.crt
    - --enable-admission-plugins=NodeRestriction
    - --enable-bootstrap-token-auth=true
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
```

This setting allows you to add a node via authentication token

Bootstrap tokens are essentially bearer tokens used when creating new clusters or joining new nodes to an existing cluster. These tokens were primarily built for kubeadm, but can also be used in other scenarios without kubeadm, such as with third party applications. It works much like a service account token, in that the token allows the third-party app to authenticate with the Kubernetes API and communicate with objects inside of the cluster. In the context of joining a node to the cluster, bootstrap tokens establish bidirectional trust between the node that's joining the cluster and a control-plane node. We can generate a new token in our kind Kubernetes cluster with the command `kubeadm token create --print-join-command`.

```
root@kind-control-plane:/# kubeadm token create --print-join-comm
kubeadm join kind-control-plane:6443 --token l5kotg.hiivo73eu000b
```

EXAM TIP

On exam day, don't be afraid to use the help menu (`kubeadm -help`). The help menu will contain examples, in some cases, that you can copy and paste right into the command-line.

Let's proceed with adding a new node to our kind Kubernetes cluster. To create a new node, we must first create a secondary cluster, then orphan the node from the new cluster, adding it to our original cluster. We do this because the orphaned node will have all the necessary pre-requisites (containerd, kubelet, and kubeadm) that are required to run this node as a Kubernetes node. On the CKA exam, you may come across a node that needs to be added to a cluster after being orphaned, but you most likely will not have to install the pre-requisites. We can create the new kind cluster with the command `kind create cluster --config config2.yaml --name cka` using the "config.yaml" file that we used in chapter 2, but modified slightly. You will get an output similar to figure 5.15.

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
```

Figure 5.15 Create a new cluster named “cka” with a two node configuration

**create two node
cluster**

**create cluster
named "cka"**

```
$ kind create cluster --config config2.yaml --name cka
```

Creating cluster "cka" ...

- ✓ Ensuring node image (kindest/node:v1.32.2) 
- ✓ Preparing nodes  
- ✓ Writing configuration 
- ✓ Starting control-plane 
- ✓ Installing CNI 
- ✓ Installing StorageClass 
- ✓ Joining worker nodes 

Set kubectl context to "kind-cka"

You can now use your cluster with:

```
kubectl cluster-info --context kind-cka
```


Now that we have created a new cluster named “cka”, our context has already been switched to the new cluster for us. We can now simply run the command `kubectl delete no cka-worker` to remove the node from that cluster, like we see in figure 5.16.

Figure 5.16 Deleting the node removes it from context, but it still lives as a Docker container

```
$ kubectl get no
```

NAME	STATUS	ROLES	AGE	VERSION
cka-control-plane	Ready	control-plane	11m	v1.32.2
cka-worker	Ready	<none>	11m	v1.32.2


```
$ kubectl delete no cka-worker ← delete node "cka-worker"
node "cka-worker" deleted
```



```
$ kubectl get no ← no longer shown in cluster
```

NAME	STATUS	ROLES	AGE	VERSION
cka-control-plane	Ready	control-plane	11m	v1.32.2

Even though we deleted “cka-worker”, the node is still running and can be accessed from outside of Kubernetes. You can get a shell to it with the command `docker exec -it cka-worker bash`. Now let’s run the command `kubeadm reset` to revert to a fresh state for this node and clear its affiliation with the “cka” cluster.

```
$ docker exec -it cka-worker bash
root@cka-worker:/# kubeadm reset
[reset] WARNING: Changes made to this host by 'kubeadm init' or '
[reset] Are you sure you want to proceed? [y/N]: y
[preflight] Running pre-flight checks
W0527 01:56:05.673626    3614 removeetcdmember.go:80] [reset] No
[reset] No etcd config found. Assuming external etcd
[reset] Please, manually reset etcd to prevent further issues
[reset] Stopping the kubelet service
[reset] Unmounting mounted directories in "/var/lib/kubelet"
[reset] Deleting contents of config directories: [/etc/kubernetes
[reset] Deleting files: [/etc/kubernetes/admin.conf /etc/kubernet
[reset] Deleting contents of stateful directories: [/var/lib/kube
```

The reset process does not clean CNI configuration. To do so, you

The reset process does not reset or clean up iptables rules or IP
If you wish to reset iptables, you must do so manually by using `t`

If your cluster was setup to utilize IPVS, run `ipvsadm --clear` (o
to reset your system's IPVS tables.

The reset process does not clean your kubeconfig files and you mu
Please, check the contents of the `$HOME/.kube/config` file.

Now we can go ahead and apply that join command that we generated earlier. This command will use `kubeadm` to join the node to our cluster named “kind” and pass the token for authentication and also pass “—discovery-token-ca-cert-hash” to validate the public key of the root certificate authority(CA) presented by the control plane. You’ll see an output similar to figure 5.17.

Figure 5.17 Use the join command to join a node to an existing cluster

```

root@cka-worker:~# kubeadm join kind-control-plane:6443 --token 15kotg.hiivo73eu000bbfu --discovery-token-ca-cert-hash sha256:13b3aac8089081
14d45b6ad91640babd8613d8136b21d405711a1204c68fa8a4
[preflight] Running pre-flight checks
    [WARNING Swap]: swap is enabled; production deployments should disable swap unless testing the NodeSwap feature gate of the kubelet
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

```

Now we can go back over to the kind cluster and look at our nodes from that context.

```

root@cka-worker:~# exit
exit
$ docker exec -it kind-control-plane bash
root@kind-control-plane:/# alias k=kubectl
root@kind-control-plane:/# k get no

```

NAME	STATUS	ROLES	AGE	VERSION
cka-worker	Ready	<none>	15m	v1.23.4
kind-control-plane	Ready	control-plane	2d1h	v1.24.0
kind-worker	Ready	<none>	2d1h	v1.24.0

Congratulations! You've successfully added an additional node to an existing cluster making it a three node cluster. Now you can continue to schedule pods to it and use it just as you would any other worker node in the Kubernetes cluster.

Practice Exercises

From a three node cluster, cordon one of the worker nodes. Schedule a pod without a nodeselector. Uncordon the worker node and edit the pod, applying a new node name to the YAML (set it to the node that was just uncordoned). After replacing the YAML, see if the pod is scheduled to the recently uncordoned node.

Start a basic nginx deployment, remove the taint from the control plane node, so that pods don't need a toleration to be scheduled to it. Add a node selector to the pod spec within the deployment and see if the pod is now running on the control plane node.

5.3 Using Helm and Kustomize

With all the different methods to configure applications for deployment in Kubernetes, therein lies a problem of misconfiguration and also developing re-deployable components. For example, when creating deployments with services, ingress, configMaps, roles, and role bindings, it can get complex and it's not always intuitive how the application developer intended the application to run. This is where Helm and Kustomize come in. Kustomize and Helm both manage Kubernetes YAML files, but they take fundamentally different approaches. Helm is a package manager for Kubernetes that supports templating to deploy applications. Kustomize is a declarative configuration management tool for taking existing YAML manifests to deploy applications. Testing your knowledge of using Helm and Kustomize will be a part of the exam, therefore, it's important to understand what problem they solve and how to use them.

5.3.1 Helm

Instead of having to rebuild each YAML file from scratch, they can be packaged up and stored in a repository for others to use and share. In addition to a template engine, much like a package manager in your favorite Linux distro (i.e., apt in Ubuntu), Helm allows you to pull down Helm charts from a repository (public or private) and install those Helm charts locally.

To install Helm on MacOS, simply perform the command `brew install helm`

To install Helm on Windows, simply perform the command `choco install kubernetes-helm`

To install Helm on Ubuntu or Debian, perform the following commands to add Helm to your package repository and install using apt:

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/master/scripts/get_helm.sh
chmod 700 get_helm.sh
./get_helm.sh
```

Once you get Helm installed, you can run the command `helm version --short` and you'll see an output similar to `v3.17.0+g301108e`, which means that the version of Helm that is currently installed is version 3.17.

Next we can install a Helm chart, which is nothing more than a group of YAML manifests packaged up into an application that runs in Kubernetes. Think of it as the instruction set which controls how the application (comprised of many YAML manifests) is installed and in what order. In order for Helm to know which cluster to install the application on, it uses the same kubeconfig that we have been using so far in the book with `kubectl`. Therefore, it can perform the same operations that us Kubernetes administrators can, which could be a good thing or a bad thing (as we've discussed in Chapter 3). For us, and for the purposes of the CKA exam, it's ok, as we'll need admin permissions to create the resources associated with our Helm chart.

We can really use any Helm chart from the public repository located at <https://artifacthub.io/>. In order to use a Helm chart, we first must add the repository where the Helm chart is stored, or else Helm won't know where to pull the chart from. We can run the command `helm repo add hashicorp https://helm.releases.hashicorp.com` to add the repository to Helm, and then run the command `helm repo list` to see that it's been added.

```
root@kind-control-plane:/# helm repo add hashicorp https://helm.r
"hashicorp" has been added to your repositories
root@kind-control-plane:/# helm repo list
NAME          URL
```

hashicorp <https://helm.releases.hashicorp.com>

EXAM TIP

During the exam look at the weblinks on the top of the question about installing the helm package. These links usually point to URLs that you are allowed to access and they contain information for you to complete the question, in that case, the URL of the helm chart to be added.

Now that a repository has been added, we can search that repository for all available charts with the command `helm search repo vault` and you'll see that within the repo hashicorp, there's a Helm chart named "hashicorp/vault" which we can use.

```
root@kind-control-plane:/# helm search repo vault
```

NAME	CHART VERSION	APP VERSION
hashicorp/vault	0.29.1	1.18.1
hashicorp/vault-secrets-gateway	0.0.2	0.1.0
hashicorp/vault-secrets-operator	0.9.1	0.9.1

Let's add another repo! I thought it would be helpful to install a load balancer in our kind cluster. This will be useful later on when we talk about communication in chapter 6. A load balancer in Kubernetes allows access to our application from outside the cluster. For example, if we were hosting a web application as a part of a deployment in Kubernetes, we'd be able to expose it via the Load Balancer service. This way, visitors of your application could reach the front end of the application via its IP address. For now though, let's focus on adding the repo, and installing the Helm chart.

We can add the repository just as we did with the hashicorp repository with the command `helm repo add metallb`
`https://metallb.github.io/metallb`

```
root@kind-control-plane:/# helm repo add metallb https://metallb.  
"metallb" has been added to your repositories
```

Now that we have the repo added, we can search for metallb with the command `helm search repo metallb`

```
root@kind-control-plane:/# helm search repo metallb
```

NAME	CHART VERSION	APP VERSION	DESCRIPTION
metallb/metallb	0.14.9	v0.14.9	A network load

Sure enough, there it is! Now, before we install it, let's talk about something called the values file. Now, we know that Helm is a templating engine. The templating engine takes a file that has values to plug in to the template, which changes the configuration of our Helm chart. This values file is written in YAML and will look similar to figure 5.18 for the configuration of a pod.

Figure 5.18 How to create a values file and apply values to Helm for a pod manifest.

```
name: nginx ← set the value of name to "nginx"

container:
  name: nginx ← set the value of name in container to "nginx"
  image: nginx ← set the value of image in container to "nginx"
  port: 80 ← set the value of port in container to 80
```

Then, once we establish these values, the templating in the actual Helm chart looks similar to a regular pod YAML file that we've created before, but instead of the values filled in, they use the templating syntax `{{ .values.name }}` as you also see in figure 5.19.

Figure 5.19 What a normal pod YAML would look like compared to a Helm chart with templated values.

POD YAML		HELM CHART
<pre> apiVersion: v1 kind: Pod metadata: name: nginx spec: containers: - image: nginx name: nginx ports: - containerPort: 80 </pre>	-VS-	<pre> apiVersion: v1 kind: Pod metadata: name: {{ .values.name }} spec: containers: - image: {{ .values.container.image }} name: {{ .values.container.name }} ports: - containerPort: {{ .values.container.port }} </pre>

Now that we know what a template file does (it templateizes your YAML files), you may be asking how do we apply the values file to our Helm chart? Well, we simply run the command `helm install` and use the `--values` flag. So, to install the metallb Helm chart, perform the command `helm install metallb metallb/metallb --values values.yaml`. But wait! We haven't created the values file yet. Let's do this now using the below YAML and save it to a file named `values.yaml`.

```

address-pools:
- name: default
  protocol: layer2
  addresses:
  - 172.18.255.200-172.18.255.250

```

The address range (172.18.255.200-250) is within the address range of your Docker bridge network, which may be different for you. To find the “kind” docker bridge CIDR, use the command `docker network inspect -f '{{.IPAM.Config}}' kind` (type “exit” to get out of kind-control-plane shell). Once you've created the `values.yaml` file, now you can run the command `helm install metallb metallb/metallb --values values.yaml`. (type “docker exec -it kind-control-plane bash” to get back into kind-control-plane shell).


```
root@kind-control-plane:/# helm install metallb metallb/metallb -  
NAME: metallb  
LAST DEPLOYED: Thu Jan 23 04:49:11 2025  
NAMESPACE: default  
STATUS: deployed  
REVISION: 1  
TEST SUITE: None  
NOTES:  
MetallB is now running in the cluster.
```

Now you can configure it via its CRs. Please refer to the metallb on how to use the CRs.

EXAM TIP

By default, helm will always install the latest available general available version of the chart. In case the exam requires you to install a particular version of the chart, invoke the command with `--version x.y.z`. You can list all the versions available with `helm search repo metallb` command.

Now that we've deployed the metallb load balancer via Helm, we'll see that it was more than just a single resource. It created a deployment, a daemonset, four secrets, two service accounts, and a configmap, as you see in figure 5.20 by running `kubectl get po,deploy,ds,secret,sa,cm`. All we had to do was run `helm install` and it installed all of that! Pretty cool, huh?

Figure 5.20 Simplifying Kubernetes scheduling and creating multiple Kubernetes objects via Helm Chart

All these
Kubernetes
objects created
by running
"helm install"

```
root@kind-control-plane:/# kubectl get po,deploy,ds,secret,sa,cm
```

NAME	READY	STATUS	RESTARTS	AGE
pod/metallb-controller-dc88974b6-9tkxn	1/1	Running	0	48s
pod/metallb-speaker-2flkl	4/4	Running	0	48s
pod/metallb-speaker-qsbfp	4/4	Running	0	48s
pod/metallb-speaker-vlw7f	4/4	Running	0	48s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/metallb-controller	1/1	1	1	48s

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE
daemonset.apps/metallb-speaker	3	3	3	3	3	kubernetes.io/os=linux	48s

NAME	TYPE	DATA	AGE
secret/metallb-memberlist	Opaque	1	42s
secret/metallb-webhook-cert	Opaque	4	48s
secret/sh.helm.release.v1.metallb.v1	helm.sh/release.v1	1	48s

NAME	SECRETS	AGE
serviceaccount/default	0	21d
serviceaccount/metallb-controller	0	48s
serviceaccount/metallb-speaker	0	48s

NAME	DATA	AGE
configmap/kube-root-ca.crt	1	21d
configmap/metallb-excludel2	1	48s
configmap/metallb-frr-startup	3	48s

It reminds me of when I'm at a restaurant and order a delicious meal. I have no idea how it's made or all of the ingredients used, it just comes out of the kitchen piping hot and tastes so delicious!

To list the Helm charts we have installed, we can run the command `helm ls`

```
root@kind-control-plane:/# helm ls
```

NAME	NAMESPACE	REVISION	UPDATED
metallb	default	1	2025-01-23 04:49:11.896140217

We'll come back to using MetalLB in chapter 6. For now, this was good practice using Helm and installing a Helm chart using custom values.

Much like `kubectl`, Helm will use the namespace of your current context to install the objects. If you want to install the objects on a given namespace, you must pass the parameter `--namespace=your-namespace`. If this

namespace does not exist, helm will fail, unless you pass another parameter, `--create-namespace`. The combination of both parameters will make helm to create a namespace and install the objects on it.

EXAM TIP

Helm provides a command that generates all the objects without having them installed. To do this, run `helm install` with the added flag `--dry-run=client` and you will see this in action. What you get is the YAML output on the screen without actually installing it. You can also use `helm template` instead of `helm install` and achieve the same outcome. During the exam, you might be asked to generate the manifests only, not necessarily install them.

5.3.2 Kustomize

Kustomize and Helm differ in many ways. Some find Kustomize to be simpler, as it is natively integrated into the `kubectl` command-line tool—eliminating the need for a separate CLI like Helm. With Kustomize, you define a `kustomization.yaml` file, which specifies the resources to manage. While it lacks built-in package management and dependency handling, Kustomize allows you to create overlays, making it easy to customize deployments across different environments while keeping your YAML configurations DRY (Don't Repeat Yourself). This is why it's used a lot for GitOps, as the desired state always exists in a Git repository, allowing teams to define base configurations and environment-specific overlays in YAML files without introducing imperative scripts or ad-hoc modifications.

In order to put this into action, let's create the YAML for a simple Kubernetes deployment with the command `kubectl create deploy my-app --image nginx:latest --port 80 --dry-run=client -o yaml > deployment.yaml`. Let's also create the YAML for a Kubernetes service with the command `kubectl create svc clusterip myapp-service --tcp=80:80 --dry-run=client -o yaml > service.yaml`. Now that you have two files `deployment.yaml` and `service.yaml`, let's create the `kustomize.yaml`. In the base directory (run `mkdir -p base` to create), create a file named `kustomization.yaml` with the following contents:

```
resources:
  deployment.yaml
  service.yaml
```

Copy the two other files you created previously, and now that we have our basic Kustomize setup, we can run the command `kubectl apply -k base/` to process the deployment and service, creating them in our Kubernetes cluster. You can run the command `kubectl get deploy,svc` to verify that the deployment and service are now created.

Let's take it one step further and create an overlay for Kustomization. Let's create an overlay for a staging environment. In the overlay directory, we'll create a staging directory, by performing the command `mkdir overlays/staging`. Inside `overlays/staging`, create a new `kustomization.yaml` file with the following contents, modifying the deployment to use 3 replicas instead of 1 (only in the staging environment overlay):

```
resources:
  - ../../base

patches:
  target:
    kind: Deployment
    name: my-app
  patch: |-
op: replace
path: /spec/replicas
value: 3
```

This patch can be applied with the command `kubectl apply -k overlays/staging/`. Verify the deployment now has 3 replicas with the command `kubectl get deploy`. You should see the `my-app` deployment with 3 replicas instead of the default 1 that we started with. The reason why we used a patch is so we didn't have to modify the original deployment YAML. This way, we can keep the base definitions clean and reusable. Also, instead of duplicating entire YAML files for different environments, Kustomize overlays apply patches to modify only specific fields, ensuring that the core configuration remains DRY. On the exam, you can expect to be tasked with creating a deployment and a service like this, so become familiar

with the tree structure of Kustomize and practice with patching multiple environments with overlays.

One of the key differences that sets it apart from Helm, Kustomize does not enforce the state of the removed objects that you once had when installed with Kustomize. In other words, if you remove a section with an object from the Kustomize file and reapplying it, the object will not get removed from the cluster.

There is another command very helpful, that can generate the actual objects to be applied on the cluster without actually applying them, similar to what we learned before as `helm template`.

Try to run `kubectl kustomize overlays/staging/`. You will see all objects to be created, but not applied. Please do an experiment, remove the line “- service” from the `base/kustomization.yaml` file. You will see the service will no longer be installed on the next time you run `kubectl apply -k overlays/staging/` but the service will still exist on the cluster.

EXAM TIP

If the exam asks you to remove any object from the Kustomize files, you will need to delete them manually from the cluster, otherwise you will lose points. One way to compare the state of the Kustomize manifests versus the state of cluster is “piping” the result of `kubectl kustomize <dir>` to the handy subcommand `kubectl diff`. In the example before, remove the service line and run `kubectl kustomize overlays/staging/ | kubectl diff -f -` and pay attention on the output. If you have not deleted the service object yet, you see the diff will not come up clean. This is a handy command for the exam to make sure all the changes on Kustomize were actually in sync with the cluster.

5.4 Custom Resource Definition (CRD) and operators

So far, we have been using built-in Kubernetes resources, like pods, deployments, ReplicaSets, etc. What if we told you that Kubernetes is

flexible enough to allow you to create your own customizable resources and you can manage in the same way as you can any other built-in resource? It's true! Kubernetes is like building with Lego's, in the way that you can truly create any type of resource you want. These "Lego building blocks" to continue the analogy are called Custom Resource Definitions (CRDs). CRDs allow us to extend the Kubernetes API and build an entirely new resource, that can be controlled and managed in the same way as any other resource in Kubernetes.

It is important to note here that it's not as simple as just creating a CRD and being done. There's typically a framework or pattern to follow, commonly in the form of an operator. An operator in Kubernetes uses CRDs to define what to manage and includes a controller that defines how to manage it, automating tasks like deployment, scaling, and healing.

EXAM TIP

The CKA exam does not ask about a particular operator. There are hundreds of them: practically every CNCF project supports at least one operator. For the following sections, we will abstract the concepts about operators, and we picked one real-world operator: CloudNativePG. The CloudNativePG Postgres operator `<https://github.com/cloudnative-pg/cloudnative-pg>` is an open source project created to automate management of Postgres databases (create, scale, backup, etc.), tasks otherwise performed by a database administrator.

5.4.1 Defining a CRD

The CRD is created like any other Kubernetes object, calling a specific resource, `apiextensions.k8s.io/v1`, and defining some key elements of it, such as name, API group, etc. One key configuration is "scope": scope defines if the CRD will be namespaced or cluster-scoped. Namespaced scope means the new resource must reside within a namespace. On other hand, cluster-scoped objects do not need a namespace and it will belong to the entire cluster.

All these settings are arbitrarily to the developer of the operator.

Let's look at one example of a CRD, straight from the CloudNativePG Postgres project. Open your browser and go to: https://github.com/cloudnative-pg/cloudnative-pg/blob/main/config/crd/bases/postgresql.cnpg.io_clusters.yaml and you will see the YAML file. Its initial lines are depicted in figure 5.21.

Figure 5.21 An example of a CRD and its most important fields

```
---
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  annotations:
    controller-gen.kubebuilder.io/version: v0.16.5
  name: clusters.postgresql.cnpg.io
spec:
  group: postgresql.cnpg.io ← New API Group
  names:
    kind: Cluster ← New Kind
    listKind: ClusterList
    plural: clusters ← New Resource
    singular: cluster
    scope: Namespaced ← Scope
```

This is a regular YAML file that, once applied, will create a custom resource definition called Cluster Kind under the postgresql.cnpg.io API group. The

same Kind is mapped to a resource named clusters that will be used for the REST API calls, similar to when we experienced working with the SWAPI in chapter 2.

If you open the CRD files, you will see how long and complicated they are. These CRD files are automatically generated by frameworks that developers use to write operators. The developers of Kubernetes operators define the schema of CRDs that dictate how users will input information for the operators. Each CRD has its own schema and fields, and each field has specific allowed values, if it is an integer or a string or etc., if the field is optional and/or it has a default value, etc.

Most likely, you will not need to navigate inside the schema of CRDs. A good operator will explain the fields leveraging `kubectl explain` command as we will see next section.

5.4.2 Installing operators and CRDs

One can install Kubernetes operators in different ways, from applying files with `kubectl` or via `helm`, like we did as part of the `Helm` section. Let's install the `CloudNativePG` operator using a regular command `kubectl apply --server-side -f \`
`https://raw.githubusercontent.com/cloudnative-pg/cloudnative-pg/release-1.25/releases/cnpg-1.25.0.yaml`

The same command will install all CRDs necessary for this operator. It is standard practice for operators to install and manage their CRDs. The below output shows the CRDs being installed from the `YAML` file (abbreviated for readability).

```
root@kind-control-plane:/# kubectl apply --server-side -f \
https://raw.githubusercontent.com/cloudnative-pg/cloudnative-pg/r
namespace/cnpg-system serverside-applied
customresourcedefinition.apiextensions.k8s.io/backups.postgresql.
customresourcedefinition.apiextensions.k8s.io/clusterimagecatalog
customresourcedefinition.apiextensions.k8s.io/clusters.postgresql
customresourcedefinition.apiextensions.k8s.io/databases.postgresq
customresourcedefinition.apiextensions.k8s.io/imagecatalogs.postg
customresourcedefinition.apiextensions.k8s.io/poolers.postgresql.
customresourcedefinition.apiextensions.k8s.io/publications.postgr
```

Many kubernetes objects were created by this command, including `customresourcedefinition.apiextensions.k8s.io`. Let's see them all with the command `kubectl get crds`. You should see an output like this:

```
root@kind-control-plane:/# kubectl get crds
backups.postgresql.cnpg.io          2025-01-23T05:31:48Z
clusterimagecatalogs.postgresql.cnpg.io 2025-01-23T05:31:48Z
clusters.postgresql.cnpg.io         2025-01-23T05:31:48Z
databases.postgresql.cnpg.io        2025-01-23T05:31:48Z
imagecatalogs.postgresql.cnpg.io     2025-01-23T05:31:48Z
poolers.postgresql.cnpg.io          2025-01-23T05:31:49Z
publications.postgresql.cnpg.io     2025-01-23T05:31:49Z
scheduledbackups.postgresql.cnpg.io  2025-01-23T05:31:49Z
subscriptions.postgresql.cnpg.io     2025-01-23T05:31:49Z
```

As you see, many CRDs were installed, each one belonging to the group `postgresql.cnpg.io`.

EXAM TIP

If you install an operator with helm command, it will install all CRDs for by default. If you do not want to install the CRDs part of helm installation, you need to specify `--skip-crds` parameter.

In chapter 1, we explained how we interact with Kubernetes via a REST API and that all objects are *Resources*. Each CRD from this list was created as an API Resource as well. Figure 5.22 shows the output running `kubectl api-resources --api-group=postgresql.cnpg.io`.

Figure 5.22 The new API Resource created from the installation of Postgres operator and its CRDs

New API Resources created with CRDs

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
backups		postgresql.cnpg.io/v1	true	Backup
clusterimagecatalogs		postgresql.cnpg.io/v1	false	ClusterImageCatalog
clusters		postgresql.cnpg.io/v1	true	Cluster
databases		postgresql.cnpg.io/v1	true	Database
imagecatalogs		postgresql.cnpg.io/v1	true	ImageCatalog
poolers		postgresql.cnpg.io/v1	true	Pooler
publications		postgresql.cnpg.io/v1	true	Publication
scheduledbackups		postgresql.cnpg.io/v1	true	ScheduledBackup
subscriptions		postgresql.cnpg.io/v1	true	Subscription

Resource belongs to a Namespace or Not

From now on, because the Postgres operator was installed and it created its CRDs, our Kubernetes cluster processes new resource types such as clusters, backups, subscriptions, etc (as shown in the first column of figure 5.20). This is one of the key benefits of writing operators: as cloud native applications, they automatically provide a fully functional API accessible over REST, all managed by Kubernetes API.

There is another point to make: please note that each CRD belongs to an API Version, which is composed by an API group and a version. In our example, the API group is `postgresql.cnpg.io` (note how we filtered the `kubectl api-resources` by this field with the command `kubectl api-resources --api-group=postgresql.cnpg.io`) and the version is `v1`. From time to time, the schema of CRDs evolve and developers of operators need to release new versions of CRDs. In this case, they manage these differences over multiple API versions. A Kubernetes cluster can have multiple versions of a given API Group.

But what now? Each operator has its own business logic and it creates its own CRDs. We can run the command `kubectl explain clusters.postgresql.cnpg.io` to see what this particular resource does (please note that we added the API group name as a suffix of the resource name - this is important to differentiate in rare cases there are multiple CRDs using the same resource name):

```
root@kind-control-plane:/# kubectl explain clusters.postgresql.cn
GROUP:      postgresql.cnpg.io
KIND:       Cluster
VERSION:    v1
```

DESCRIPTION:

Cluster is the Schema for the PostgreSQL API

(...)

spec <Object> -required-

Specification of the desired behavior of the cluster.

So it seems we can create a Postgres cluster by creating objects of the Kind “Cluster”. By the way, when you install any operator, most likely on their docs website, they will provide examples on how to start using them. But for the exam, you will need to rely on the `kubectl explain` command. So, now we can understand the syntax and business logic of the operator, let’s jump on using the Postgres operator in the next section!

5.4.3 Using operators

Let’s check if the Postgres operator is running, let’s verify via the command

```
kubectl get deployment -n cnpg-system cnpg-controller-manager:
root@kind-control-plane:/# kubectl get deployment -n cnpg-system
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
cnpg-controller-manager            1/1      1              1            6m27s
```

The Postgres operator is running as a built-in deployment like the examples we saw in previous sections in this chapter. Now this operator is running, it is ready to act upon a creation of a custom object, which we will see next. A CRD *defines the schema* and API for a new type of resource in Kubernetes (e.g., MyApp, Database, BackupJob), whereas, the Custom Resource (CR) is an *instance* of that new type — it’s the actual object you create and manage in your cluster, using YAML or `kubectl`.

Now that our Postgres operator is running, we need to activate it by creating a custom object that the operator is expecting to act upon. Part of the CloudNativePG operator installation, it created a handful of CRDs as we saw in the last section, but which one should we use?. Each operator will have documentation on how to use it. For this particular operator, under the github

project they give some examples <https://github.com/cloudnative-pg/cloudnative-pg/blob/main/docs/src/quickstart.md>.

For the CloudNativePG Postgres operator, its goal is creating a high-available postgres cluster running on Kubernetes cluster, so they need the creation of an object based on the CRD Cluster Kind.

EXAM TIP

The creation of custom resource based on any CRD is referred as “CR”. Many people interchangeably use the acronyms “CR” and “CRD”. This is wrong. There is a one-to-many relationship between CRD and CRs. CRD is the definition that needs to be created before one instantiates multiple CRs out of it.

So, let’s create a cluster CR creating a file containing Cluster Kind and parameters for the postgres cluster. Here is an example YAML file below:

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: my-postgres-cluster
  namespace: default
spec:
  instances: 1
  storage:
    size: 1Gi
```

Let’s break down this object as figure 5.23 shows. First off, note that it is a kind cluster belonging to API Version postgresql.cnpg.io. It specifies an object name, my-postgres-cluster, and it belongs to the namespace default (recall that Postgres cluster is a namespaced object). Finally, based on this configuration, this Postgres cluster will have only one instance with 1Gb of size. As we saw in figure 5.22, when we showed the details of the field instance, we learned that this field is an integer with default value of 1. So, technically, we did not need to necessarily specify this field on our Custom Resource (CR). But we did it anyway for the sake of explanation and declaring explicitly we want to start the Postgres cluster with a single instance.

Figure 5.23 An example of a Custom Resource (CR)

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster ← Kind and API Group
metadata:
  name: my-postgres-cluster ← Name of the CR
  namespace: default ← Namespace of the CR
spec:
  instances: 1
  storage:
    size: 1Gi
```

Specifications
As Defined
By the Operator

One of the benefits of using CRDs is that their specifications (aka “spec”) are self-explanatory as defined by the schema explained in the previous section. In this case, without requiring deep knowledge of Postgres, this file intends to create a simple cluster and we just had to define size and the replica number. The Postgres operator will take care of everything for you to create a database. Let’s create the postgres cluster by running `kubectl apply -f postgres-cluster.yaml`:

```
root@kind-control-plane:/# kubectl apply -f postgres-cluster.yaml
cluster.postgresql.cnpg.io/my-postgres-cluster created
```

Let's now see the status of the Postgres cluster we just created by running `kubect1 get cluster my-postgres-cluster -n default`:

```
root@kind-control-plane:/# kubectl get cluster my-postgres-cluste
```

NAME	AGE	INSTANCES	READY	STATUS
------	-----	-----------	-------	--------

```
my-postgres-cluster    3m19s    1                1        Cluster in heal
```

That's so cool! We have a Postgres cluster running. The output of the command shows the Postgres cluster `my-postgres-cluster` is in a healthy state with one instance. This information is gathered by the Postgres operator and shows it in a structured form.

If you want to get even more information about your Postgres cluster, let's run `kubectl describe cluster my-postgres-cluster -n default`. The `describe` command is a standard way of retrieving more information from a resource. A good operator leverages this command to display advanced information about its resources.

```
root@kind-control-plane:/# kubectl describe cluster my-postgres-c
Name:          my-postgres-cluster
Namespace:     default
(...)
Status:
  Available Architectures:
    Go Arch:    amd64
  (...)
  Switch Replica Cluster Status:
    Target Primary:          my-postgres-cluster-1
    Target Primary Timestamp: 2025-01-23T06:12:35.277745Z
    Timeline ID:             1
    Topology:
      Instances:
        my-postgres-cluster-1:
          Nodes Used:      1
```

Crazy, right? So much information. It is mind-blowing what happened behind the scenes: the Postgres operator acted on the creation of the Custom Resource (CR) and translated the `Cluster` type object specs into a running pod called `my-postgres-cluster-1` with Postgres in it.

You can see the pods created by this operator by running `kubectl get pods -n default`:

```
root@kind-control-plane:/# kubectl get pods -n default
NAME                                READY    STATUS    RESTARTS
my-postgres-cluster-1              1/1      Running   0
```

If you are curious about more features of this Postgres operator, try to edit the object (`kubectl edit cluster my-postgres-cluster -n default`) and increase the number of instances to 2 and see what happens.

Learning Postgres is definitely out of scope of this book and the CKA exam, but we used this as an example of how powerful Kubernetes operators are. By creating a simple YAML, you saw a Kubernetes operator abstracting the complexity of tasks usually performed by DBAs to create a database from scratch.

Practice Exercises

Install operator <https://cert-manager.io/>, a CNCF project to generate TLS certificate for Kubernetes workloads. Navigate the documentation and check different ways to install this operator. Install it on the namespace “my-cert-manager” using the helm method. Please note that installing CRDs is part of the process as long as you pass the helm argument `--set crds.enabled=true`. After installation, check and list all CRDs installed that belongs to the cert-manager API Group. Check if the operator is running.

Install another operator, this time the popular Argo CD, a CNCF project used for GitOps. You can get details on <https://argo-cd.readthedocs.io/>. Install the operator via the manual way following the steps on <https://argo-cd.readthedocs.io/en/stable/operator-manual/installation/>. Choose the installation option of Non High Availability, with the option of only requiring namespace level privileges. In this mode, you will need to install the CRDs separately, which are on <https://github.com/argoproj/argo-cd/blob/master/manifests/crds>. The documentation provides the installation command line using “`kubectl apply -k`” (the parameters `-k` uses the Kustomize option). Once the operator and CRDs are installed, check the CRDs of Argo CD and if the operator is running.

5.5 Summary

- Deployments are a common resource in Kubernetes and the replica sets within are important for keeping the desired number of replicas running
- Rollouts are versioned and you can optionally leave notes for other

Kubernetes administrators or developers

- You can expose a deployment, which creates a service for users to access the application from outside the cluster.
- Maintenance is inevitable and will mean at some point, you must know cordon and drain, in order to perform OS upgrades or add additional resources to a node
- You can add a node to an existing cluster to provide additional resources to applications running on Kubernetes
- Much like a package manager in Linux, Helm is a package manager and templating engine for Kubernetes.
- Much like Helm for deploying complex apps, there's also Kustomize, which is built-in to kubectl (no CLI). Get familiar with using kustomization.yaml files with base and overlays for the exam. Know how to use Helm for the exam, and how to use Helm templates to deploy applications to Kubernetes.
- Know how to extend the Kubernetes cluster with operators and CRDs and take advantage of a breadth of cloud native applications.